

UNIVERSITY OF LONDON

INTERNATIONAL PROGRAMMES

BSc Computer Science and Related Subjects



CM3070 PROJECT

FINAL PROJECT REPORT

<Data-Driven Personalised Educational Content Recommendation>

Author: Lee Wei Jie Jordy

Student Number : 230682469

Date of Submission: 03/08/2025

Supervisor : Mr Loh Hai Kian

Code repository: <https://github.com/Jor2302/CM3070FinalProject-datasci>

Youtube: <https://youtu.be/sabFc7oOOxM>

<NOTE that the overall maximum word count for the report is 9500 words excluding Appendices and References. If you follow strictly the maximum word count for each chapter, it will exceed 9500 words, i.e. you will need to decide which chapter(s) to trim to meet the overall word count limit>

Contents

CHAPTER 1:	INTRODUCTION.....	6
CHAPTER 2:	LITERATURE REVIEW.....	10
CHAPTER 3:	PROJECT DESIGN.....	17
CHAPTER 4:	IMPLEMENTATION	25
CHAPTER 5:	EVALUATION.....	48
CHAPTER 6:	CONCLUSION.....	62
CHAPTER 7:	APPENDICES	65
CHAPTER 8:	REFERENCES.....	131

List of Figures

Figure 3.1: Grant Chart/Workplan**Figure 4.1:** Udemy Course Data

A snapshot of the Udemy course dataset, showing key attributes like course ID, title, subject, price, and user engagement.

Figure 4.2: User-Item Rating Matrix

A section of the user-item matrix, where each row represents a user and each column a course.

Figure 4.3: Synthetic User-Course Interaction Data

A sample of simulated user feedback, showing user IDs, course titles, and ratings.

Figure 4.4: Data Preparation

NLP preprocessing of course descriptions including tokenisation, stop-word removal, and lemmatisation.

Figure 4.5: TF-IDF and BERT Embeddings

Feature engineering using TF-IDF vectorisation and BERT semantic embeddings for content-based and semantic recommendations.

Figure 4.6: Collaborative Filtering with SVD

Visualisation of SVD decomposition and predicted ratings for unseen user-course pairs.

Figure 4.7: Cosine Similarity

Content-based filtering using TF-IDF vector similarity between courses.

Figure 4.8: Semantic Similarity

BERT-based semantic similarity matching between course descriptions.

Figure 4.9: API Integration Workflow

RESTful Flask API endpoints and request routing for recommendation system operations.

Figure 4.10: Association Rules (Apriori)

Visualisation of frequent course enrollment patterns with support, confidence, and lift statistics.

Figure 4.11: Sentiment & Engagement

Dashboard view showing processed user feedback, sentiment analysis, and engagement classification.

Figure 4.12: Topic Modelling and Word2Vec Similarity

Topic clusters and semantic similarity for course discovery using LDA and Word2Vec.

Figure 4.13: A/B Testing

Evaluation table comparing collaborative filtering and hybrid recommendation strategies.

Figure 4.14: Home Page of the Educational Recommendation System

Homepage showing learner and teacher tools with integrated learning assistant.

Figure 4.15: Personalised Course Recommendations

Interface displaying the top course recommendations for a user, including ratings, optional comments, and feedback buttons.

Figure 4.16: Word2Vec Similar Courses

Interface for finding semantically similar courses using Word2Vec embeddings.

Figure 4.17: Feedback Submission Form

Form allowing users to submit course feedback, including user ID, course ID, rating, and comments.

Figure 4.18: Course ID and Title List

Page displaying available courses for learners.

Figure 4.19: Feedback Sentiment & Classification Result

Summary of processed user feedback, sentiment analysis, and engagement classification.

Figure 4.20: Real User Simulation

Display of simulated learner feedback, ratings, and distribution chart.

Figure 4.21: Association Rules Dashboard

Course-to-course recommendation showing support, confidence, and lift metrics.

Figure 4.22: A/B Test Results

Comparison of collaborative filtering (Group A) with hybrid recommender (Group B).

Figure 4.23: Evaluation Dashboard

Summary dashboard of model performance metrics, topic insights, and recent user feedback.

Figure 5.1: Precision–Recall Curve Table

Precision–Recall curve for SVD with area under curve equal to 0.332. PR curve that starts near precision one at near zero recall then declines to near 0.3 as recall grows.

Figure 5.2: ROC/SVD Table

ROC curve for SVD with area under curve equal to 0.548. ROC curve slightly above the diagonal baseline indicating modest discrimination.

Figure 5.3: Precision Table

Top N precision and recall at five for all models using the current pipeline. Bar chart where popularity has a tiny bar and the other models have bars at or near zero.

Figure 5.4: A/B Table

A/B test summary table with difference in means equal to 0.583, $p = 0.134$, confidence interval, and Cohen's d. Table showing group sizes, group means, difference, confidence interval, and non-significant conclusion.

Figure 5.5: LDA Chart

LDA topic insights used in the dashboard and explanation flows. Horizontal bars for the top five words per topic across five topics.

Figure 5.6: Survey Results Bar Chart

Bar chart showing the distribution of user responses on ease of use, recommendation relevance, and clarity of explanations from survey data.

Figure 5.7: Model Performance Comparison

Comparison chart showing Precision@K and RMSE for hybrid, collaborative filtering (SVD), and content-based recommendation methods.

Figure 5.8: ROC Curve for Classifier Performance

Receiver Operating Characteristic (ROC) curve visualising the trade-off between true positive and false positive rates for the engagement classifier model.

Figure 5.9: Clarity of Explanations Survey Result

Bar chart showing the distribution of user responses on the clarity of explanations.

CHAPTER 1: INTRODUCTION (905/1000 words)

Background and Context

The digital transformation of education has significantly changed how learners access, engage with, and benefit from educational content. Due to the proliferation of internet access and the development of artificial intelligence, students now primarily rely on online learning environments such as Coursera, Khan Academy, YouTube Learning, and LinkedIn Learning for their coursework. Opportunities have increased as a result of this democratization of education, but there are also new difficulties, mainly in the form of information overload. Users often find themselves overwhelmed by an abundance of choices, making it difficult to identify the most suitable content for their individual learning goals and preferences.

The emergence of personalised recommendation systems offers a potential solution to this challenge. These systems use historical user data, preferences, and behaviours to tailor content recommendations to each learner's unique context. However, despite the presence of rudimentary recommendation engines on current platforms, many fall short of delivering genuinely adaptive and responsive experiences. This underscores the necessity for more intelligent, dynamic systems capable of understanding and evolving with the learner.

Motivation

Personalised learning is known to increase learner motivation, improve content retention, and support more efficient knowledge acquisition. Unfortunately, current mainstream platforms rely heavily on popularity metrics or rigid progress-tracking systems. For instance, Coursera often recommends content based on global popularity or user ratings, ignoring the nuances of an individual learner's performance history or engagement style. Khan Academy, though adaptive in terms of skill mastery progression, follows a linear path and does not adapt to behavioural patterns like preferred content type, format, or engagement time.

Similarly, YouTube Learning recommends videos based on prior viewing habits, but its algorithm is primarily geared toward maximizing watch time, not educational effectiveness. LinkedIn Learning offers role-specific content but lacks the granular adaptive mechanisms that could customize learning paths based on learner success metrics, feedback, and content comprehension.

These examples highlight a shared limitation: a lack of semantic understanding, real-time adaptivity, and integration of both implicit (clicks, view time) and explicit (ratings, feedback) data. This is where our project's opportunity and motivation lie: creating a more contextually aware and responsive system that puts learner engagement and individualized development first.

Problem Statement

Learners today are overwhelmed by abundant course options and face decision fatigue, especially when platforms prioritize engagement metrics over actual learning needs. This system is designed to help users quickly find relevant content aligned with their goals, improving motivation, confidence, and time efficiency.

Existing systems prioritize platform-level goals (e.g., increasing enrolments or viewership) over learner-centric outcomes. They lack mechanisms for continuous learner modelling that evolves as the user progresses. Furthermore, these systems often do not capture or respond to subtle indicators such as time spent on activities, interaction sequence patterns, or comprehension feedback. As a result, users are repeatedly exposed to irrelevant or unhelpful content.

Proposed Solution Overview

This project introduces a data-driven recommendation engine aimed at personalising educational content through the analysis of user behaviour, semantic content understanding, and iterative feedback integration. The system incorporates several key components to achieve this goal:

- Behaviour tracking, which involves collecting granular data such as clickstream activity, time-on-task, and content completion rates to better understand user engagement patterns.
- Machine learning models including collaborative filtering, content-based filtering, and matrix factorization to analyse user-item interaction data and predict relevant content.
- Natural language processing techniques such as BERT embeddings, TF-IDF vectorisation, and topic modelling to assess and categorise learning materials based on their semantic properties.
- Dynamic profiling that continuously updates each learner's profile, allowing the recommendations to evolve in real time.

By integrating these components, the recommendation engine is designed to deliver context-aware, adaptive learning experiences that align with each user's unique and evolving educational journey.

Project Template Used

This report follows **Template 1.1: Data-Driven Personalised Educational Content Recommendation**, as outlined in the CM3005 Data Science. This template guides the development of systems that offer personalised learning experiences through computational techniques.

Aims and Research Questions

The overall aim is to develop a technically feasible, data-driven personalised educational content recommendation system using hybrid machine learning models and NLP.

The project explores the following research questions:

- Can a hybrid approach (collaborative + content-based filtering) outperform single-method recommenders in relevance and accuracy?
- How effectively can NLP improve semantic matching between learners and educational content?
- What techniques best handle cold-start situations where user history is unavailable?
- Can simulated sentiment and engagement data be effectively integrated into personalisation algorithms?

Objectives

- To design and implement a hybrid recommendation engine for personalised educational content.
- To integrate lightweight NLP models for semantic content understanding.
- To evaluate the system's effectiveness using machine learning metrics (RMSE, precision, recall, AUC).
- To demonstrate feasibility through a working prototype API that delivers dynamic recommendations.
- To explore a scalable system design for K–12 learners, tertiary institutions, or self-paced adult learning environments.

Ethics, Privacy, and Inclusion

All data used are de-identified or synthetic, avoiding risks to individual privacy. The system prioritises transparency through explanatory recommendation labels (e.g., “Based on your previous courses”). Semantic models such as multilingual BERT embeddings support learners across diverse backgrounds and language settings. Future work will explore explicit bias checks and accessibility features to strengthen inclusivity.

Contributions

This project makes three key contributions:

1. A modular hybrid educational recommender integrating CF, CBF, semantic NLP, clustering, sentiment analysis, and association rules.

2. An end-to-end **Flask API + dashboard** with evaluation tools (offline metrics and online-style A/B testing).
3. Practical cold-start strategies using semantic similarity and clustering to generate recommendations for new learners.

CHAPTER 2: LITERATURE REVIEW (2369/2500 words)

Introduction

The rapid evolution of online learning platforms has catalyzed the need for advanced recommendation systems that go beyond simple popularity metrics. Personalized educational content recommendation systems aim to tailor learning materials based on user preferences, behavioral data, and content features. This chapter reviews critical academic and applied work related to educational recommendation systems, the algorithms and models that underpin them, and relevant studies from psychology and education validating the project's theoretical foundations. The goal is to identify prior contributions, evaluate the strengths and limitations of various approaches, and highlight gaps that justify this project.

This review draws from peer-reviewed journal articles, established recommender system surveys, and technical implementations from reputable educational platforms such as Khan Academy and Coursera. The models and methods reviewed here are selected from well-cited academic literature and industry-grade implementations. All sources are considered credible due to their publication in scholarly journals or real-world application in large-scale educational platforms.

Numerous studies have demonstrated that recommender systems can enhance educational outcomes when thoughtfully applied. *Tang and Pardos (2020)*, for instance, found that personalised recommendations in higher education improved course selection accuracy and student retention. *Walkington (2013)* showed that aligning content with student interests led to higher engagement and better learning performance. These results underscore the potential of recommendation systems to promote learner autonomy and educational effectiveness, particularly when real-time behavior and semantic content alignment are incorporated.

Related Works

(i) EduRank: A Course Recommendation System

EduRank, developed by *Tang and Pardos (2020)*, is a course recommendation system that utilizes collaborative filtering techniques based on students' grade histories to suggest suitable university courses. By analyzing past academic performance, EduRank aims to guide students toward courses where they are likely to succeed, thereby improving academic outcomes. However, the model is primarily tailored for higher education institutions and lacks flexibility for real-time adaptability or domain transfer. It does not incorporate behavioral data or semantic content analysis, limiting its applicability to broader learning audiences. In contrast, our project differentiates itself by integrating behavioral analytics and natural language processing techniques to provide personalized

recommendations for a diverse range of learners, including self-directed individuals and younger students.

(ii) Khan Academy's Mastery-Based Progression System

Khan Academy employs a mastery-based learning system that guides learners through a structured curriculum using skill dependency graphs. This approach allows students to progress at their own pace, ensuring they achieve proficiency in foundational concepts before advancing to more complex topics. While effective for linear, curriculum-aligned advancement, this system lacks dynamic feedback loops and personalization based on individual learning preferences or behavioral patterns.

(iii) Coursera's Collaborative Filtering and Learning-to-Rank Models

Coursera utilizes collaborative filtering and learning-to-rank algorithms to recommend courses to users. These models consider detailed user interaction metrics, including video watch duration, click-through rates, and dropout statistics, to personalize recommendations. However, they often rely on large datasets and extensive user histories, which may not be available for new users. Our system focuses on early-stage adaptivity and interpretable models suitable for platforms with sparse data, ensuring personalized recommendations even for users with limited interaction history.

(iv) BERT-Based Material Matching

Zhang et al. (2020) applied BERT to match learners with reading materials based on vocabulary level and topical alignment, demonstrating the effectiveness of NLP in semantic content understanding and personalized resource delivery. Inspired by this approach, our system adopts a simplified NLP pipeline using BERT and TF-IDF for scalable yet meaningful semantic filtering, enhancing the relevance of educational content recommendations.

(v) Temporal Modeling in Hybrid Recommendation Systems

Elbadrawy and Karypis (2016) introduced temporal modeling into hybrid recommendation systems, improving course predictions in MOOC environments by considering the temporal dynamics of student data. While effective, these models are computationally intensive and best suited for large platforms. Our approach adapts this idea by simplifying the model for smaller-scale use without sacrificing predictive accuracy, making it more accessible for platforms with limited computational resources.

(vi) LinkedIn Learning Personalization

LinkedIn Learning emphasizes career pathways and job relevance, offering personalized course recommendations based on user profiles, job roles, and field of interest. However, it often lacks real-time adaptability, relying on static profiles and self-declared interests. Our solution enhances personalization by incorporating dynamic profiling and real-time behavioral data, allowing the system to adapt to evolving learner needs and preferences.

Techniques and Algorithms

The foundation of this project is built upon well-established techniques in the field of recommendation systems, particularly those that combine collaborative filtering, content-based filtering, and natural language processing. These methods collectively enable the system to deliver personalised, context-aware educational content that aligns with learners' preferences and behaviours.

Collaborative Filtering (CF)

Collaborative filtering is one of the most widely used approaches in recommendation systems. It operates on the principle that users who have exhibited similar preferences in the past are likely to share similar tastes in the future (*Su & Khoshgoftaar, 2009*). By analyzing learner interactions, such as course ratings or completion patterns, CF can be used in educational settings to recommend content that similar users have found useful. This project employs matrix factorization techniques, particularly Singular Value Decomposition (SVD), to reduce dimensionality and uncover latent user-item relationships. However, while CF is effective in mature systems with rich user data, it suffers from two key limitations: the cold-start problem, where new users or items lack sufficient data for accurate recommendations, and data sparsity, which affects the model's ability to generalise in low-interaction environments. These challenges are especially relevant in educational contexts where user engagement can be sporadic or short-term.

Content-Based Filtering (CBF)

Content-based filtering offers an alternative by focusing on the attributes of items rather than user behaviour. It recommends learning materials based on similarities to items the user has previously engaged with. *Liang et al. (2018)* proposed a metadata-driven approach that employs keyword extraction and structured descriptors of learning objects. In this project, content-based filtering is enhanced using natural language processing techniques to derive high-dimensional feature representations of course descriptions. The advantage of CBF lies in its interpretability and ability to operate in environments with sparse user interaction data. It is particularly useful in smaller systems or new platforms where user profiles are incomplete. By analysing course titles, subject tags, difficulty levels, and summaries, our system builds content vectors that allow for precise similarity scoring, ensuring recommendations remain relevant even for cold-start users.

Hybrid Recommendation Models

To overcome the individual limitations of CF and CBF, hybrid recommendation models have emerged as a robust solution. *Burke (2002)* introduced a framework for combining multiple recommendation strategies to achieve better accuracy and resilience. Hybrid systems benefit from the strengths of both CF and CBF, balancing the exploitation of user behaviour with the exploration of content features. In educational settings, hybridisation improves personalisation by considering both user preferences and the inherent properties of learning materials. Elbadrawy and *Karypis (2016)* advanced this idea by integrating temporal dynamics with matrix factorisation, thereby enhancing course predictions in MOOC environments. Drawing from this, our system employs a hybrid logic that combines collaborative filtering predictions, content-based similarity scores, and behavioural clustering. This

integration allows the model to generate more comprehensive and personalised recommendations, addressing the cold-start issue and improving robustness in varied user contexts.

Natural Language Processing (NLP)

Natural Language Processing plays a pivotal role in extracting and interpreting semantic content from educational materials. Traditional CBF approaches rely on surface-level keyword matching, which can fail to capture the nuanced relationships between topics. Our system integrates multiple NLP techniques to improve semantic understanding. Firstly, Term Frequency–Inverse Document Frequency (TF-IDF) is used to identify important terms within course descriptions, enabling weighted comparison of textual content. Secondly, topic modelling through Latent Dirichlet Allocation (LDA) is employed to categorise courses into broader thematic groups, helping learners discover related content even across different subject domains.

Most importantly, we incorporate BERT embeddings to capture deep contextual relationships within text. BERT (Bidirectional Encoder Representations from Transformers) enables the model to understand the meaning of words in relation to surrounding context, improving its ability to assess topic alignment between user interests and course content. These NLP components not only enhance recommendation precision but also allow the system to adapt to learners with varying reading levels, language preferences, and domain-specific interests.

Together, these algorithms and techniques form a comprehensive backbone for the proposed system, ensuring that recommendations are not only personalised and accurate but also explainable and adaptable across user profiles, content types, and educational domains.

Theoretical & Pedagogical Foundations

The development of this personalised educational content recommendation system is informed by established theories in educational psychology and instructional technology, which collectively underscore the importance of individualised learning experiences for effective knowledge acquisition.

Research by *Walkington (2013)* demonstrates that personalising mathematics problems to align with students' personal interests significantly enhances both engagement and learning outcomes. This finding supports the premise that learners are more motivated and focused when educational content is contextually relevant to their preferences. In line with this, our system incorporates behaviour-based profiling to infer learner interests implicitly, thereby enabling the delivery of tailored recommendations without requiring extensive manual input from users.

In a related vein, *VanLehn (2011)* conducted a meta-analysis of intelligent tutoring systems (ITS) and found that such systems can achieve learning gains comparable to those of human tutors. This insight highlights the transformative potential of adaptive learning technologies that adjust in real time to user behaviour. Our recommendation engine is designed with similar adaptivity in mind, continuously

updating learner profiles based on evolving interaction patterns to deliver increasingly personalised and responsive content suggestions.

Another guiding framework for this project is Self-Determination Theory (SDT), as proposed by **Ryan and Deci (2000)**. SDT identifies autonomy and competence as fundamental psychological needs that foster intrinsic motivation and sustained learning. Personalised learning environments support autonomy by allowing learners to navigate educational pathways that reflect their goals, interests, and readiness. In our system, this is operationalised through recommendation algorithms that balance learner freedom with appropriate content scaffolding. By aligning learning materials with the user's evolving skill level and topic familiarity, the system promotes both competence and motivation.

Together, these theoretical foundations provide a strong pedagogical rationale for the personalised and adaptive features embedded in our system. They reinforce the idea that effective learning technologies must go beyond content delivery to support learner engagement, autonomy, and long-term achievement.

Research Gaps and Project Contribution

The literature highlights several key limitations that this project directly seeks to address. One of the primary challenges in existing systems is the cold-start problem, where recommendations rely heavily on historical user data. To overcome this, our hybrid model integrates content similarity and user clustering to generate relevant suggestions from the outset, even for new users with minimal interaction history.

Another critical issue is the lack of semantic precision in keyword-based systems, which often fail to capture the contextual meaning of content. To resolve this, our approach incorporates semantic embeddings and topic modelling, enabling a deeper and more accurate understanding of course material.

Moreover, many existing platforms depend on resource-intensive architectures that require substantial computational power. In contrast, our implementation is designed with a lightweight architecture that prioritizes efficiency and scalability, making it suitable for low-resource environments.

A further concern lies in the use of black-box models, such as deep neural networks, which, while accurate, are often opaque and difficult for educators to interpret. Our system emphasizes transparency and explainability, allowing both educators and learners to understand the rationale behind each recommendation.

Importantly, this project adopts a learner-first design philosophy. Unlike conventional systems that prioritise metrics like clicks or time-on-task, our system is focused on meaningful educational progression and learner autonomy. By aligning recommendations with a user's performance trends and comprehension patterns, we promote long-term learning outcomes.

Lastly, many recommendation systems are narrowly designed for specific domains. Our model, on the other hand, was created with cross-domain adaptability in mind, which means that it can be used in a variety of learning contexts, including language acquisition and STEM subjects. While engagement remains a valuable metric, this project is distinctive in its commitment to advancing learners' educational growth as the primary goal.

Critical Synthesis and Gaps

Taken together, prior studies demonstrate the breadth of methods applied to educational recommendation systems. Platforms such as EduRank and Coursera show the strength of collaborative filtering on rich user histories, while Khan Academy's mastery graphs emphasise curriculum alignment. At the same time, BERT based models highlight the potential of semantic precision, and temporal hybrids point to long term adaptability. These strands illustrate a central trade off between scalability, personalisation depth, and computational cost.

Despite progress, key gaps remain. They also rely on large datasets that are often unavailable in many educational settings, and they prioritise engagement metrics over true learning outcomes. Few systems offer transparency and explainability that are accessible to both learners and educators. Very little research addresses inclusive design considerations such as multilingual learners, accessibility for users with cognitive or sensory differences, and fairness across demographic groups.

Project Novelty in Context

This project contributes to the field by combining multiple strands that are rarely integrated in a single pipeline.

- It uses a hybrid approach that combines collaborative filtering with content based filtering and enhances this with BERT semantic embeddings to improve cold start robustness.
- It applies user clustering and association rules to diversify recommendations and to improve explainability.
- It incorporates feedback driven sentiment analysis so that the system adapts to learner perceptions rather than relying only on clicks.
- It delivers these methods through a modular Flask API that demonstrates practical integration in a unified and transparent system.

By aligning with both the technical gaps of accuracy, scalability, and semantic precision and the pedagogical needs of autonomy, transparency, and inclusivity, this work advances the literature toward learner first design in real world educational contexts.

Summary Table: Prior Work Comparison

Study / Platform	Technique Used	Personalisation Type	Target Group	Gaps Addressed by Our Project
EduRank (Tang & Pardos)	Collaborative Filtering	Grade History	University Students	No real-time adaptivity
Khan Academy	Graph-Based Progression	Mastery Tracking	K-12 Learners	Lacks behavior-based recommendations
Coursera	CF + Learning-to-Rank	Engagement Metrics	Adult Learners	Assumes large user data
LinkedIn Learning	Static Preference-Based	Career Pathing	Professionals	No dynamic behavioral adaptation
Zhang et al. (2020)	BERT + NLP	Reading Level + Topic	Mixed	Complex deployment, not optimized for speed
Elbadrawy & Karypis (2016)	Hybrid + Temporal Model	Course Timing	MOOC Participants	Heavyweight, low interpretability

CHAPTER 3:PROJECT DESIGN (1812/2000 words)

Introduction

This chapter presents the technical and conceptual design of a personalised educational content recommendation system. Developed under the CM3005 Data Science template, the system is built to balance performance, scalability, and user interpretability. It integrates multiple recommendation techniques including collaborative filtering (CF), content-based filtering (CBF), and natural language processing (NLP) to generate tailored recommendations that align with each user's learning needs and interaction patterns. The architecture is modular and scalable, allowing the system to be deployed in both small-scale educational contexts and large, web-based learning environments. The design reflects current trends in data-driven education technology, offering a solution that is both technically robust and pedagogically grounded.

Domain and Intended Users

This project targets the online education domain, specifically focusing on the needs of learners and developers within digital learning platforms. Intended users include self-paced learners, K–12 students, adult learners engaging in professional development, and instructional designers or developers responsible for learning management systems. For example, platforms like Udemy, Khan Academy, and Coursera represent the kind of environments where such a system would be applicable. These platforms typically host thousands of courses, which can overwhelm users attempting to find relevant content. The system also supports institutions that need scalable recommendation engines integrated into their LMS to personalize student experiences. Users in this domain demand recommendations that are accurate, timely, and explainable, particularly when there is limited interaction data to work with. Therefore, the system emphasizes relevance, transparency, cold-start resilience, and responsiveness as core design principles.

For example, a self-paced learner on a MOOC platform like Coursera may feel overwhelmed by hundreds of course options. By analysing their past behavior and semantic interests, this system can surface a small set of highly relevant recommendations, improving decision-making and motivation.

Educational Data Used

The datasets used in this project are adapted from the [Kaggle Course Recommendation System dataset](#). This includes:

- **Course metadata:** A table of Udemy course information, including course IDs, titles, subjects, price, subscriber counts, and content summaries.

Justification of Design Choices

Several critical design decisions have been made to meet the needs of users and the requirements of the educational domain. First, the system addresses the cold-start problem, which is especially prominent on platforms with transient users or in K–12 settings where interaction data may be minimal. To address this, the system uses a hybrid approach that combines collaborative filtering with content-based techniques and semantic analysis to generate useful recommendations even in the absence of historical data.

Another major decision is the adoption of hybrid modelling, which enables the system to balance personalisation with exploration. Instead of recommending only what is familiar or similar, the system introduces learners to new, but still relevant, materials. This fosters content discovery and encourages deeper engagement with the learning platform.

A second critical design choice addresses explainability is built into the recommendation engine to increase transparency and user trust. Learners benefit from understanding why certain courses are being recommended, and educators gain insight into the logic behind automated content suggestions. Natural language processing is also used to enhance the semantic precision of recommendations.

Finally, an API-based deployment strategy is chosen to make the system easy to integrate with a wide range of front-end applications or institutional dashboards. This flexibility is especially valuable for developers working on custom LMS solutions or mobile-first educational apps.

Each component of the design directly addresses user challenges observed in current platforms. Cold-start resilience is handled through content-based and clustering techniques. Semantic NLP ensures recommendations go beyond surface-level keyword matches. The hybrid logic balances familiarity with discovery, encouraging exploration while remaining relevant. Explainable outputs improve learner trust. Together, these design elements help ensure that the system not only performs well but also supports real, learner-centric outcomes.

Project Architecture

The architecture is intentionally modular, enabling real-time adaptivity while ensuring the system is scalable, explainable, and efficient for deployment in real-world educational platforms.

The overall architecture consists of four layers, each responsible for a different set of functions that support the recommendation workflow. The first layer is the data ingestion layer, which collects user

interaction data such as quiz results, content completion rates, and clickstream logs. It also gathers metadata from educational content, including course titles, categories, transcripts, and difficulty levels.

The second layer is the feature engineering layer. This stage transforms raw data into structured formats suitable for machine learning. It processes and cleans the data, constructs user-item matrices for collaborative filtering, and generates content vectors using NLP techniques like TF-IDF and BERT. This layer ensures that both user behaviour and content characteristics are accurately captured.

Next is the recommendation engine layer. Here, multiple algorithms operate in parallel or in a combined manner. Collaborative filtering using SVD is applied to discover latent relationships between users and items. Content-based similarity scores are calculated using vector space models. The outputs are then aggregated into a final hybrid score using a weighted formula that can be fine-tuned for different learning contexts.

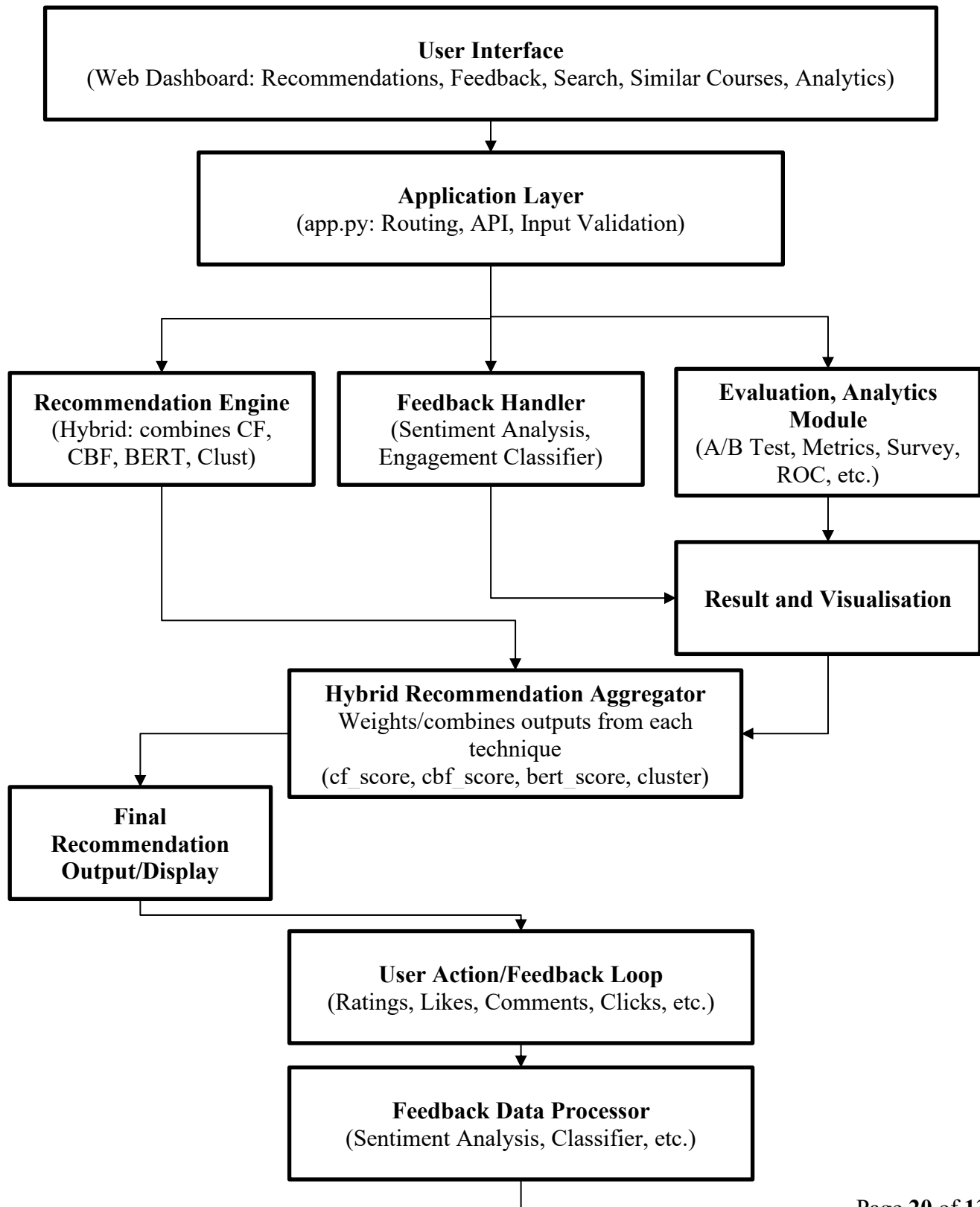
The final layer is the interface or API layer. It provides an accessible endpoint for retrieving recommendations, formatted for integration into web and mobile applications. The API accepts parameters such as user ID, number of recommendations, and model weightings. Each returned recommendation includes an explanation label, helping users understand the rationale behind the suggestion.

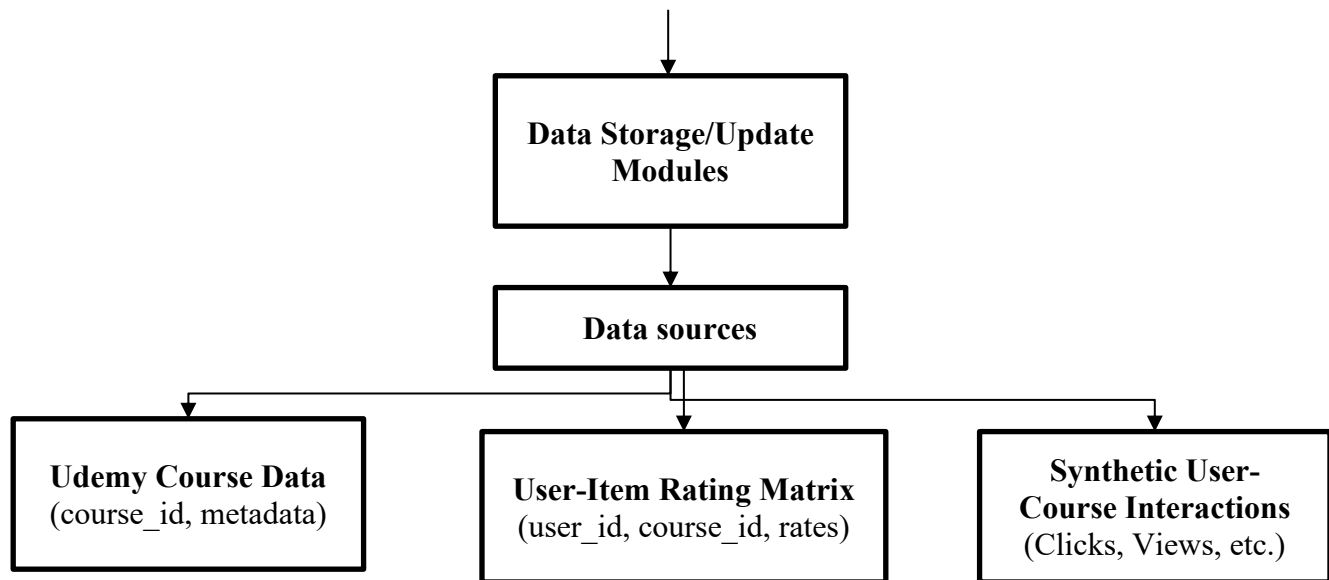
The recommendation engine is built as a modular Flask API that interfaces with multiple recommendation strategies. The system follows a pipeline:

Within the engine, we use:

- Collaborative Filtering (SVD via Surprise) for known users
- Content-Based Filtering (TF-IDF) for intermediate fallback
- Semantic Similarity (BERT) for cold-start users
- Clustering (KMeans) to identify similar user groups for fallback scoring

System Diagram





Technologies and Methods

The system leverages a combination of machine learning, NLP, and software engineering technologies. Collaborative filtering is implemented using matrix factorization techniques such as SVD to uncover hidden patterns in user-item interaction matrices. Content-based filtering relies on TF-IDF and cosine similarity to assess the closeness between course descriptions. These techniques are merged using a hybrid scoring method that balances historical preferences with semantic relevance. User clustering is also employed using K-means, allowing the system to tailor its logic based on behavioural profiles.

In terms of natural language processing, TF-IDF is used for shallow textual analysis, while LDA topic modelling identifies thematic clusters across course materials. BERT embeddings provide deep contextual understanding of text, enabling the system to make nuanced connections between learner interests and educational content. These NLP models enhance both accuracy and adaptability, especially for multilingual or diverse subject matter platforms.

Development is conducted entirely in Python using Visual Studio Code (VS Code), with libraries such as pandas, scikit-learn, numpy, and HuggingFace Transformers. Flask is used to build a lightweight REST API for serving the recommendations.

These selected techniques were not only chosen for technical efficiency but also for how well they align with user needs:

- SVD captures user-course interaction patterns and generalises well from sparse matrices, making it suitable for platforms with incomplete user histories.
- TF-IDF is effective for users with limited but meaningful engagement data, matching them to content based on textual overlap.
- BERT embeddings enable semantic similarity assessments for cold-start users with no historical data, offering nuanced content matching.
- KMeans Clustering acts as a robust fallback, assigning users to behaviourally similar segments to infer preferences when direct data is sparse.

Together, these methods ensure that the system maintains high relevance, resilience, and interpretability across various usage scenarios.

Evaluation Plan

- **Collaborative Filtering (SVD):** RMSE, MAE for rating prediction accuracy, Precision@K to assess top-N recommendation relevance.
- **Content-Based Filtering (TF-IDF):** Precision@K, Recall@K, and qualitative user feedback for recommendation relevance.
- **Semantic Matching (BERT):** Cosine similarity thresholds are tested for cold-start performance, and AUC-ROC is used to evaluate binary relevance classification.
- **Hybrid Model:** Overall RMSE and comparative A/B testing to evaluate performance gains over individual models.

To ensure that the system performs well in real-world educational settings, both quantitative and qualitative evaluation methods are planned. Quantitative evaluation includes metrics such as Root Mean Square Error (RMSE), which measures the accuracy of predicted ratings compared to actual user feedback. Precision at K and Recall at K will be used to evaluate how well the top recommended items match user preferences. AUC-ROC may also be included to assess the system's ability to distinguish between relevant and irrelevant content.

Qualitative evaluation will involve user simulation tests, particularly focusing on cold-start scenarios and returning user behaviour. To compare the efficacy of various recommendation strategies, A/B testing will be done. For instance, collaborative filtering alone will be tested against the hybrid model.

Clustering effectiveness will be evaluated using silhouette scores to determine how well user groups are formed based on behavioural data.

For A/B testing, simulated users will be divided into two groups. Group A will receive recommendations from the baseline collaborative filtering model, while Group B will receive recommendations from the hybrid model. Key metrics such as Precision@K and satisfaction scores will be compared between groups to determine the effectiveness of hybridization. This will help validate if integrating content-based and NLP techniques improves perceived usefulness and relevance.

Beyond technical metrics like RMSE and Precision@K, the project also evaluates perceived usefulness. To increase openness and trust, each suggestion has an explanation label, such as "Based on your prior interests." Simulated learner journeys will assess whether the recommendations help users identify content more efficiently. In future iterations, feedback collection (e.g., ratings or satisfaction scores) can further inform how well the system supports learners in practice.

Explainability and User Trust

One of the key features of the system is its ability to explain recommendations in user-friendly language. Each recommendation is accompanied by a label that explains why it was selected, such as "Based on your previous courses," "Popular among users in your cluster," or "Matched to your topic preferences." These labels are generated dynamically based on the source of the score (CF, CBF, or cluster-based). This feature is essential for building trust among users, particularly in educational contexts where learners may be making decisions that impact their academic progress.

Explainability also helps educators understand how the system works and verify whether the recommendations align with pedagogical goals. By making the recommendation process transparent, the system promotes accountability and user engagement.

Deployment Considerations

The system is designed for production readiness. The Flask or FastAPI API exposes stateless endpoints, making it compatible with mobile apps, web dashboards, and LMS plugins. The model's logic is encapsulated in modular scripts that can be updated independently.

Sessionless design ensures flexibility, while optional integration with user authentication systems can enable more advanced use cases.

The recommendation engine can be further scaled by moving the prediction process to cloud infrastructure, leveraging GPU instances for faster NLP embedding and real-time response generation.

Sustainability & Scalability: The system is built to be cloud-ready, supporting containerization and scalable deployment using services like AWS or Google Cloud. This allows dynamic scaling to handle increasing numbers of users and datasets efficiently, ensuring long-term sustainability.

Visual Timeline

Task/Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16	W17	W18	W19	W20
Background research																				
Project proposal + ethics																				
Literature review																				
Refine project requirements																				
System design/architecture																				
Dataset curation/preparation																				
Prototype model development																				
Advanced model integration																				
Evaluation/Testing																				
Dashboard/UI implementation																				
Final improvements/bugfixes																				
Write final report																				
Prepare demo video																				
Final edits and submission																				

Figure 3.1: grant chart/workplan

CHAPTER 4:IMPLEMENTATION (1520/2500 words)

Introduction

This chapter describes the full implementation of the personalised educational content recommendation system. The objective was to build a robust, flexible platform capable of generating accurate, individualised recommendations for learners in online environments. The final solution integrates multiple machine learning methods. It including collaborative filtering, content-based filtering, semantic matching (BERT embeddings), user clustering, sentiment analysis, topic modelling, association rule mining, and A/B testing.

Data Sources and Relationships

This project relies on three interconnected datasets, each serving a distinct purpose in the recommendation engine. Together, they enable both the training and evaluation of collaborative, content-based, and hybrid recommendation algorithms.

All three datasets used in this project. The Udemy course data (course metadata and descriptions), user-item rating matrix (explicit user ratings for courses), and synthetic user-course interaction logs (simulated clicks, views, and feedback).

It are provided in the [Course Recommendation System dataset](#) on Kaggle. This dataset offers comprehensive, realistic samples for both content-based and collaborative recommendation tasks.

Udemy Course Data (udemy_course_data.csv)

- **Content:**

Contains detailed information on each course:

- course_id (unique identifier),
- course_title,
- subject
- Additional metadata.

popularity. The user_id and course_id columns are used to match with user interactions and course metadata.

A1	user_id																						
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	user_id	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
2	user_1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	user_10	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	user_100	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	user_1000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	user_1001	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	user_1002	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	user_1003	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	user_1004	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	user_1005	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	user_1006	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	user_1007	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	user_1008	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14	user_1009	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	user_101	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16	user_1010	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17	user_1011	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	user_1012	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19	user_1013	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	user_1014	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	user_1015	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
22	user_1016	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4	0	0	0	0	0	0	0
23	user_1017	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
24	user_1018	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 4.2: User-Item Rating Matrix

A section of the user-item matrix, where each row represents a user and each column a course.

Synthetic User-Course Interactions (Synthetic_User-Course_Interactions.csv)

- Content:**

Simulates real-world engagement by recording interactions such as:

- user_id,
- course_id,
- rating,
- interaction_type (e.g., view, click, feedback)

- Purpose:**

This dataset mimics diverse user activity and feedback. It powers modules like feedback-aware re-ranking, cluster-based recommendations, sentiment analysis, and A/B testing. Like the rating matrix, it is linked by both user_id and course_id to the other two datasets.

	A	B	C	D	E	F
1	user_id	course_id	course_title	rating		
2	user_1	2408	Learn Vikings Soundtracks on the Piano by Ear	3		
3	user_1	1594	Learn Sketch 3 Mobile App Design From Scratch	5		
4	user_1	3162	Earn Big Profits With Web Development, Simple and Easy!	4		
5	user_1	1615	Sketching: How to Draw People, Hands, Eyes, Buildings & Cars	4		
6	user_1	2847	JavaScript Rockstar How to create Incredible Useful code	4		
7	user_2	1481	Adobe Illustrator a to z	4		
8	user_2	671	Curso Completo del Mercado FOREX	4		
9	user_2	1654	Drawing for All Levels: How to Design Cartoon Women	2		
10	user_2	470	Bitcoin - A Comprehensive Guide	1		
11	user_2	2875	Wordpress on AWS: The Easy way to AWS for Free Fast hosting	5		
12	user_2	3029	Learn Html5 & CSS3 from scratch	3		
13	user_2	2369	Guitar Chords and Basic Strumming	5		
14	user_2	3408	Build Real World Websites from Scratch using HTML5 and CSS3	4		
15	user_2	1838	Instant Harmonica - play two Star Wars themes in 30 minutes	4		
16	user_2	1884	Didgeridoo - from zero to HERO! Part One - beginning	4		
17	user_2	3084	Learn how to use Javascript Components with PHP and Symfony	1		
18	user_2	2008	Getting Started with Playing Guitar	2		
19	user_2	1363	Voxel 3D Model Creation Course	3		
20	user_3	3393	Learn how to quickly build websites using Wordpress	4		
21	user_3	2512	Web Development w/ Google's Go (golang) Programming Language	3		
22	user_3	2008	Getting Started with Playing Guitar	2		
23	user_3	2740	Create a MEAN App by Example - Full Stack Course	2		
24	user_3	1513	Illustrator from zero to hero	3		

Figure 4.3: Synthetic User-Course Interaction Data

A sample of simulated user feedback, showing user IDs, course titles, and ratings. This dataset enables testing of feedback-based recommendations and user engagement analysis.

How the Datasets are Linked

- **Primary Linkage:**

All datasets are tied together via the shared `course_id` (linking course metadata with user ratings and interactions) and `user_id` (connecting user behavior across rating and engagement datasets).

- **Workflow Example:**

- When a user requests recommendations, the system first checks their interaction history (Synthetic_User-Course_Interactions.csv) and ratings (User-Item_Rating_Matrix.csv).
- For each course, the system retrieves metadata (title, subject) from udemy_course_data.csv to perform content-based or semantic similarity calculations.
- If collaborative signals are weak (new user), content-based and semantic methods dominate. If both rating and interaction data are available, hybrid logic leverages all three datasets.

- **Result:**

This linkage enables a robust, end-to-end recommendation process, supporting collaborative, content-based, and feedback-driven recommendations and even simulating “cold start” scenarios.

Summary Table

Dataset	Key Link Fields	Main Purpose
Udemy Course Data	course_id	Content/NLP, metadata for courses
User-Item Rating Matrix	user_id, course_id	Collaborative filtering (SVD)
Synthetic User-Course Interactions	user_id, course_id	Simulate user behavior, feedback

Data Generation and Preparation

A critical early step was to generate a synthetic dataset simulating real-world user interactions with educational content. Data was structured across several CSV files, such as Synthetic_Interactions.csv, udemy_course_data.csv, and User-Item_Rating_Matrix.csv, each containing user ratings, course details, and content metadata. The dataset included randomised, but realistic, user behaviours and demographics to capture a variety of learning patterns.

To prepare the data for modelling, a standard natural language processing pipeline was applied. This included tokenisation, stop-word removal, and lemmatisation of course descriptions.

Example code (recommender.py):

```

46 # ----- data loading -----
47 # Courses
48 _course_df = pd.read_csv(P("data", "udemy_course_data.csv"),
49                          usecols=["course_id", "course_title", "subject"])
50 _course_df = _course_df.dropna(subset=["course_id"]).copy()
51 _course_df["course_id"] = pd.to_numeric(_course_df["course_id"], errors="coerce")
52 _course_df = _course_df.dropna(subset=["course_id"]).copy()
53 _course_df["course_id"] = _course_df["course_id"].astype(int)
54 _course_df["course_title"] = _course_df["course_title"].fillna("").astype(str)
55 _course_df["subject"] = _course_df["subject"].fillna("").astype(str)
56 _course_df["text"] = (_course_df["course_title"] + " " + _course_df["subject"]).str.strip()
57
58 COURSE_IDS = _course_df["course_id"].tolist()
59

```

figure 4.4, Data Preparation

This approach allowed for effective benchmarking of algorithms in a controlled environment and supports ongoing improvement as new datasets are introduced.

Feature Engineering

Feature engineering was conducted using both TF-IDF vectorisation and BERT-based semantic embedding. TF-IDF was used to extract and weigh key terms in course descriptions, resulting in a high-dimensional feature matrix ideal for content-based filtering. BERT embeddings (using sentence-transformers) captured nuanced, context-sensitive semantic similarities between courses.

Example code (TF-IDF, recommender.py):

```
201     # TF-IDF similarity
202     tfidf_vec = _tfidf.transform([" ".join(liked_texts)])
203     tfidf_sims = cosine_similarity(tfidf_vec, _tfidf_matrix).flatten() # len = N courses
204
```

Example code (BERT Embeddings, recommender.py):

```
83     # BERT sentence embeddings
84     _EMB_DIR = P("embeddings")
85     os.makedirs(_EMB_DIR, exist_ok=True)
86     _BERT_PATH = P("embeddings", "courses_minilm.npy")
87
```

Figure 4.5: TF-IDF and BERT embeddings

TF-IDF provided explainable, interpretable features, while BERT significantly improved semantic precision, especially in cold-start situations where user history is limited. Both approaches enabled transparent, meaningful recommendations.

Collaborative Filtering with SVD

Collaborative filtering, implemented via the Surprise library's SVD algorithm, learned hidden user and item preferences from historical rating data. SVD decomposed the user-item matrix into latent factors, allowing for the prediction of ratings even for unseen user-course pairs.

Example code (svd_recommender.py):

```
16 # Function to train an SVD collaborative filtering model
17 def train_svd_model(interactions_path: str = "data/Synthetic_Interactions.csv"):
18     """
19     Trains an SVD model with CV (RMSE/MAE), then FITS on the FULL trainset for serving.
20     Returns: (model, avg_rmse, avg_mae)
21     """
22     interactions_path = _resolve(interactions_path)
23
24     df = pd.read_csv(interactions_path)[["user_id", "course_id", "rating"]].dropna()
25     # Surprise expects string IDs
26     df["user_id"] = df["user_id"].astype(str)
27     df["course_id"] = df["course_id"].astype(str)
28
29     reader = Reader(rating_scale=(1, 5))
30     data = Dataset.load_from_df(df[["user_id", "course_id", "rating"]], reader)
31
32     # CV first (model re-initialized internally per fold)
33     model = SVD(random_state=42)
34     cv_results = cross_validate(model, data, measures=["RMSE", "MAE"], cv=5, verbose=False)
35     avg_rmse = round(float(cv_results["test_rmse"].mean()), 3)
36     avg_mae = round(float(cv_results["test_mae"].mean()), 3)
37
38     # Fit on the FULL trainset for serving predictions
39     full_trainset = data.build_full_trainset()
40     model.fit(full_trainset)
41
42     return model, avg_rmse, avg_mae
```

Figure 4.6: Collaborative Filtering with SVD

SVD consistently produced low RMSE scores in cross-validation, confirming its effectiveness for active users. However, its limitations in cold-start cases highlight the importance of hybridising with content-based and clustering approaches.

Content-Based Filtering

Content-based filtering was used for users with sparse or no interaction history. The cosine similarity of TF-IDF vectors determined which courses were most similar to those the user had engaged with previously.

Example code (recommender.py):

```
205     # BERT similarity
206     if _bert is not None:
207         # embed the combined liked text
208         from numpy.linalg import norm
209         v = SentenceTransformer("all-MiniLM-L6-v2").encode([" ".join(liked_texts)], show_progress_bar=False, converge
210         n = norm(v)
211         if n > 0:
212             v = v / n
213             bert_sims = (_bert @ v) # cosine via dot
214             sims = 0.5 * tfidf_sims + 0.5 * bert_sims
215         else:
216             sims = tfidf_sims
217     else:
218         sims = tfidf_sims
219
220     # keep only candidate ids
221     idx_by_cid = {int(cid): i for i, cid in enumerate(_course_df["course_id"].tolist())}
222     out = {cid: float(sims[idx_by_cid[cid]]) for cid in candidate_ids if cid in idx_by_cid}
223     return out
```

Figure 4.7: Cosine Similarity

Content-based filtering increased recommendation diversity and ensured coverage for new or less active users, directly supporting the project's objective to address the cold-start problem.

Semantic Matching with BERT

BERT embeddings enabled deep semantic similarity matching, going beyond keyword overlap to account for context and topic alignment. Cosine similarity was computed between BERT-generated vectors for course descriptions.

Example code (recommender.py):

```

258 # ----- PUBLIC API -----
259 def get_recommendations(user_id: str, top_n: int = 5) -> List[Dict]:
260     """
261     Hybrid recommender:
262     - If truly new user -> popularity fallback (cold-start).
263     - Else combine SVD (collab), content (TF-IDF/BERT), and rules boost.
264     Returns: list of {"course_title": str, "explanation": str}
265     """
266     user_id = str(user_id).strip()
267     top_n = max(1, min(int(top_n), 50))
268
269     seen = _seen_courses(user_id)
270     all_ids = set(COURSE_IDS)
271     candidate_ids = sorted(list(all_ids - seen))
272     if not candidate_ids:
273         candidate_ids = COURSE_IDS # extreme case: show something
274
275     # True cold-start: user not in ratings nor interactions -> POPULARITY
276     no_ratings = _ratings.empty or user_id not in _ratings.index
277     no_inter = _inter.empty or user_id not in set(_inter["user_id"])
278     if no_ratings and no_inter:
279         pop = _popularity_scores(exclude=seen)
280         if not pop:
281             # dataset missing? Show top by course_id to avoid empty UI
282             fallback = sorted(candidate_ids)[:top_n]
283             return [{"course_title": _course_df.loc[_course_df["course_id"] == cid, "course_title"].iloc[0],
284                     "explanation": "Cold-start fallback"} for cid in fallback]
285         top = sorted(pop.items(), key=lambda x: x[1], reverse=True)[:top_n]
286         return [{"course_title": _course_df.loc[_course_df["course_id"] == cid, "course_title"].iloc[0],
287                 "explanation": "Popular with learners (cold-start)"} for cid, _ in top]
288

```

Figure 4.8: Semantic Similarity

BERT improved the system's ability to match user interests to relevant content, particularly in specialised subject areas. However, computation was resource-intensive and required careful optimisation for latency.

API Development and Integration Layer

A Flask API (app.py) orchestrates the system's operations. Endpoints such as /recommend, /feedback, /evaluate, /rules, and /word2vec route requests to the appropriate modules. The API supports authentication, logging, and error handling for robust deployment.

Example code (app.py):

```

87 @app.route("/recommend", methods=["GET"])
88 def recommend():
89     user_id = request.args.get("user_id", default="user_1", type=str)
90     try:
91         top_n = int(request.args.get("top_n", 5))
92     except Exception:
93         top_n = 5
94     top_n = max(1, min(top_n, 50))
95
96     results = get_recommendations(user_id, top_n)
97     return render_template("recommend.html", user_id=user_id, recommendations=results)

```

Figure 4.9: Semantic Similarity,

RESTful design and Swagger documentation enable seamless integration with external platforms, supporting project scalability and maintainability.

Association Rule Mining and Explainability

The association rules module (association_rules.py) uses Apriori to find frequent enrolment patterns, surfacing explainable relationships (“users who took X also took Y”). These explanations are shown on the user dashboard.

Example code (association_rules.py):

```

23 def _load_transactions(csv_path: str = os.path.join(DATA_DIR, "Synthetic_Interactions.csv")) -> list[list[str]]:
24     """
25     Load user->course transactions from CSV and convert to a list of lists.
26     Each inner list is the set of course_ids taken by one user.
27     This is the input format expected by Apriori (via TransactionEncoder).
28     """
29     if not os.path.exists(csv_path):
30         return []
31     # Read only the columns we need.
32     df = pd.read_csv(csv_path, usecols=["user_id", "course_id"])
33     df["user_id"] = df["user_id"].astype(str)
34
35     # Normalize course_id to clean string IDs like "123" (avoid "123.0", NaN, etc.).
36     df["course_id"] = pd.to_numeric(df["course_id"], errors="coerce")
37     df = df.dropna(subset=["course_id"]).copy()
38     df["course_id"] = df["course_id"].astype(int).astype(str)
39
40     # Group by user into lists of course IDs.
41     return df.groupby("user_id")["course_id"].apply(list).tolist()

```

```
44 def _build_rules(  
45     min_support: float = 0.01,  
46     min_confidence: float = 0.10,  
47     *,  
48     csv_path: str = os.path.join(DATA_DIR, "Synthetic_Interactions.csv"),  
49 ) -> pd.DataFrame:  
50     """  
51     Build association rules using Apriori:  
52     1) One-hot encode transactions.  
53     2) Mine frequent itemsets with min_support.  
54     3) Generate rules with min_confidence.  
55     4) Sort by lift/confidence/support for stability.  
56     """  
57     transactions = _load_transactions(csv_path)  
58     if not transactions:  
59         return pd.DataFrame()  
60  
61     # One-hot encode transactions (rows = users, cols = course IDs).  
62     te = TransactionEncoder()  
63     te_ary = te.fit(transactions).transform(transactions)  
64     onehot = pd.DataFrame(te_ary, columns=te.columns_)  
65  
66     # Find frequent itemsets (e.g., {A,B} appears ≥ min_support of users).  
67     freq = apriori(onehot, min_support=min_support, use_colnames=True)  
68     if freq.empty:  
69         return pd.DataFrame()  
70  
71     # Turn frequent itemsets into rules (A ⇒ B) with confidence ≥ threshold.  
72     rules = association_rules(freq, metric="confidence", min_threshold=min_confidence)  
73     if rules.empty:  
74         return rules
```

Figure 4.10: Apriori

These rules increase user trust and transparency, directly supporting the system's explainability objectives.

Sentiment Analysis and Classification

User feedback is analysed for sentiment (VADER model in `sentiment_analysis.py`) and engagement level (Naive Bayes classifier in `classifier.py`). This data is used to refine the recommendations and dashboard analytics.

Example code (sentiment_analysis.py):

```

1  from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
2
3  # Function to analyze sentiment of a given text
4  def analyze_sentiment(text):
5      analyzer = SentimentIntensityAnalyzer() # Create a VADER sentiment analyzer
6      scores = analyzer.polarity_scores(text) # Get sentiment scores (pos, neu, neg, compound)
7      compound = scores['compound'] # Use the compound score as overall sentiment
8
9      # Classify sentiment based on compound score thresholds
10     if compound >= 0.05:
11         label = "positive"
12     elif compound <= -0.05:
13         label = "negative"
14     else:
15         label = "neutral"
16
17     # Provide an explanation with the compound score
18     explanation = f"Compound score: {compound} → {label.upper()}"
19     return label, explanation # Return sentiment label and explanation
20

```

Figure 4.11: Sentiment & Engagement

Incorporating real-time feedback ensures the system remains responsive and adaptive to users' needs and experiences.

Topic Modelling and Word2Vec Similarity

Topic modelling (LDA, in lda_topics.py) and Word2Vec similarity (word2vec_similarity.py) expand the scope of recommendations, allowing the system to suggest content based on thematic clusters and semantic relationships.

word2vec_similarity.py

```

41
42  Qodo Gen: Options | Test this function
43  def _tokenize(s: str) -> list[str]:
44      """Turn a title into lowercase alphanumeric tokens (e.g., 'Python 101! → ['python', '101'])."""
45      if not isinstance(s, str):
46          return []
47      return _ALNUM.findall(s.lower())

```

```
69 Qodo Gen: Options | Test this function
70 def _train_and_cache() -> Tuple[word2vec, np.ndarray, np.ndarray, pd.DataFrame]:
71     """
72     Train Word2Vec on the tokenized course titles + subject.
73     Then build a single vector per course (mean of token vectors), L2-normalize it,
74     and cache both the model and the vectors on disk for speed.
75     """
76     df = _load_df()
77     if df.empty:
78         raise RuntimeError("udemy_course_data.csv has no rows after cleaning")
79
80     # Text to train on = "title + subject" per course
81     text = (df["course_title"] + " " + df["subject"]).astype(str)
82     tokenized = [_tokenize(t) for t in text]
83
84     # Train a small Word2Vec model (skip-gram). Vector size 64 is fast and good enough.
85     model = Word2Vec(
86         sentences=tokenized,
87         vector_size=64,
88         window=5,
89         min_count=1,      # keep even rare tokens (small dataset)
90         workers=1,        # deterministic across OS
91         sg=1,             # 1 = skip-gram, better for small data
92         seed=42,
93     )
94
```

lda_topics.py

```
7 # Train an LDA topic model using course title and subject data
8 def train_lda_model(data_path="data/udemy_course_data.csv", n_topics=5):
9     # Load course data
10    df = pd.read_csv(data_path)
11
12    # Combine course_title and subject into a single text column
13    df['text'] = df['course_title'].fillna('') + " " + df['subject'].fillna('')
14
15    # Convert text to document-term matrix
16    vectorizer = CountVectorizer(stop_words='english', max_df=0.95, min_df=2)
17    dtm = vectorizer.fit_transform(df['text'])
18
19    # Fit LDA model
20    lda = LatentDirichletAllocation(n_components=n_topics, random_state=42)
21    lda.fit(dtm)
22
23    # Extract top words for each topic
24    terms = vectorizer.get_feature_names_out()
25    topics = []
26    for idx, topic in enumerate(lda.components_):
27        top_words = [terms[i] for i in topic.argsort()[-5:]] # Top 5 words
28        topics.append((f"Topic {idx+1}", top_words))
29
30    return topics
31
```

```
32 # Plot a horizontal bar chart summarizing LDA topics and their top words
33 def plot_topic_summary(topics, output_path="static/lda_topics.png"):
34     # Ensure output directory exists
35     os.makedirs("static", exist_ok=True)
36
37     # Create the plot
38     fig, ax = plt.subplots(figsize=(8, 4))
39     labels = [t[0] for t in topics] # Topic labels
40     word_labels = [" | ".join(t[1]) for t in topics] # Top words per topic
41
42     # Plot bars and annotate with words
43     ax.barh(labels, [len(t[1]) for t in topics], color="#3498db")
44     for i, words in enumerate(word_labels):
45         ax.text(len(topics[i][1]) + 0.1, i, words, va='center', fontsize=9)
46
47     # Final formatting
48     ax.set_title("Top 5 Words per LDA Topic")
49     ax.invert_yaxis()
50     plt.tight_layout()
51     plt.savefig(output_path)
52     plt.close()
```

Figure 4.12: Topic Modelling and Word2Vec

These techniques enhance discovery and help users navigate large course catalogs, contributing to the system's goal of promoting learner autonomy.

A/B Testing and Evaluation

Evaluation modules (evaluate.py, ab_testing.py) compute metrics like RMSE, Precision@K, ROC AUC, and run A/B tests to compare recommendation strategies. Results are visualised on the dashboard for stakeholders.

A/B Testing.py

```
31
32 ✓ def assign_group(user_id: str) -> str:
33     """
34     Deterministic user -> group assignment.
35
36     We hash the user_id and map even hashes to "A", odd to "B".
37     This keeps group membership stable across runs without persisting a map.
38     """
39     h = int(hashlib.md5(user_id.encode()).hexdigest(), 16)
40     return "A" if (h % 2 == 0) else "B"
41
42
58 def _cohens_d(a: np.ndarray, b: np.ndarray) -> Optional[float]:
59     """
60     Cohen's d for independent samples using pooled standard deviation.
61
62     Returns d = (mean(b) - mean(a)) / s_pooled, or None if not computable.
63     """
64     n1, n2 = len(a), len(b)
65     if n1 < 2 or n2 < 2:
66         return None
67     s1, s2 = np.std(a, ddof=1), np.std(b, ddof=1)
68     pooled_var = ((n1 - 1) * s1**2 + (n2 - 1) * s2**2) / max(1, (n1 + n2 - 2))
69     if pooled_var <= 0:
70         return None
71     return float((np.mean(b) - np.mean(a)) / np.sqrt(pooled_var))
72
73
74 def _effect_label(d: Optional[float]) -> Optional[str]:
75     """
76     Qualitative magnitude for Cohen's d (Cohen's conventional bins).
77     """
78     if d is None:
79         return None
80     ad = abs(d)
81     if ad < 0.2: return "negligible"
82     if ad < 0.5: return "small"
83     if ad < 0.8: return "medium"
84     return "large"
85
```

Figure 4.13: A/B Testing

Systematic evaluation ensures recommendations meet quality standards and provides actionable insights for continuous improvement.

User Experience and Feedback Loop

A web-based dashboard integrates recommendations, explanations, evaluation metrics, and user feedback. This closed feedback loop enables the system to become more personalised as more data is collected.

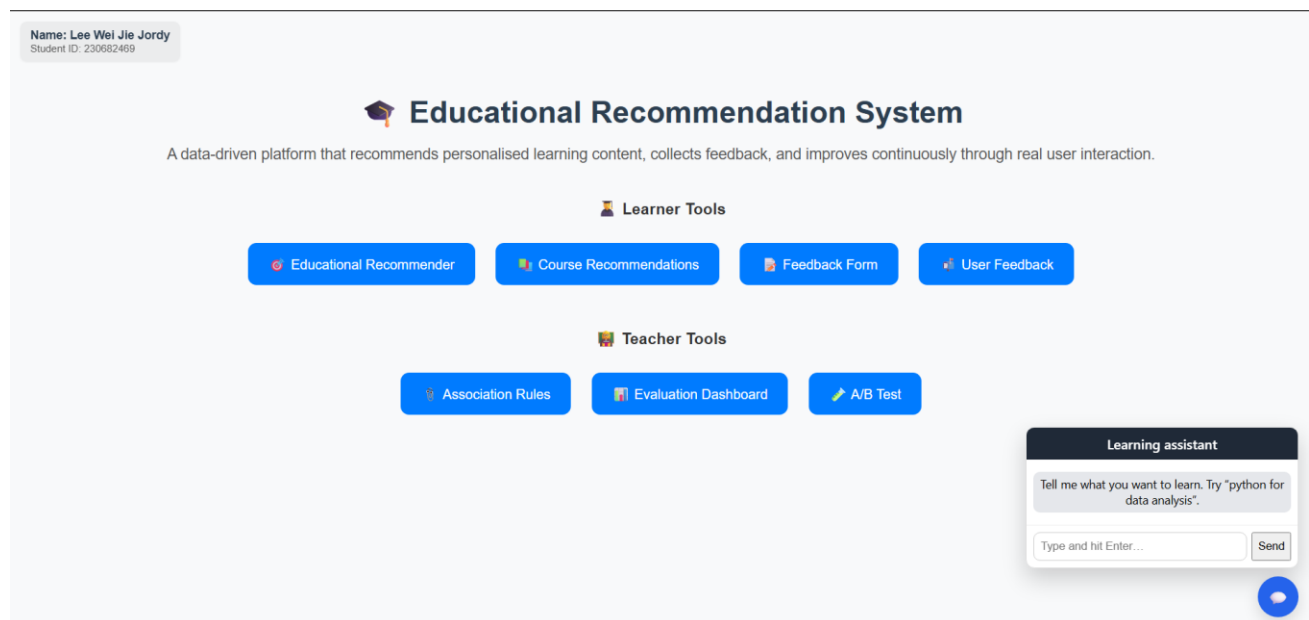


Figure 4.14: Home page of the Educational Recommendation System showing learner and teacher tools with integrated learning assistant.

Get Recommendations

User ID: Number of Course:

Top 5 Recommendations for user_45

- **TRADING TRUTHS** — *content similarity; hybrid score=0.3*
- **Let's JavaScript! Code a calculator** — *content similarity; hybrid score=0.289*
- **Ultimate Retirement: Build Your Own Dynamic Retirement Plan** — *content similarity; hybrid score=0.263*
- **Accounting for everyone!** — *content similarity; hybrid score=0.26*
- **Accounting and Business Basics** — *content similarity; hybrid score=0.257*

[← Back to Home](#)

Figure 4.15: Personalised Course Recommendations

The system recommends top courses for a user based on past ratings. Users can provide thumbs-up/down and comments.

Word2Vec: Find Similar Courses

Number of Course

Top Matches:

Create **Trading Robot Without Coding Skills!**

Business Finance
Similarity: 0.998

Professional **Trading** - Consistent Profits **with** Low Risk!

Business Finance
Similarity: 0.998

188% Profit in 1Year - Forex **ALGO** Robot **Trading** no indicator

Business Finance
Similarity: 0.998

Forex **Trading A-Z™** - **With** LIVE Examples of Forex **Trading**

Business Finance
Similarity: 0.998

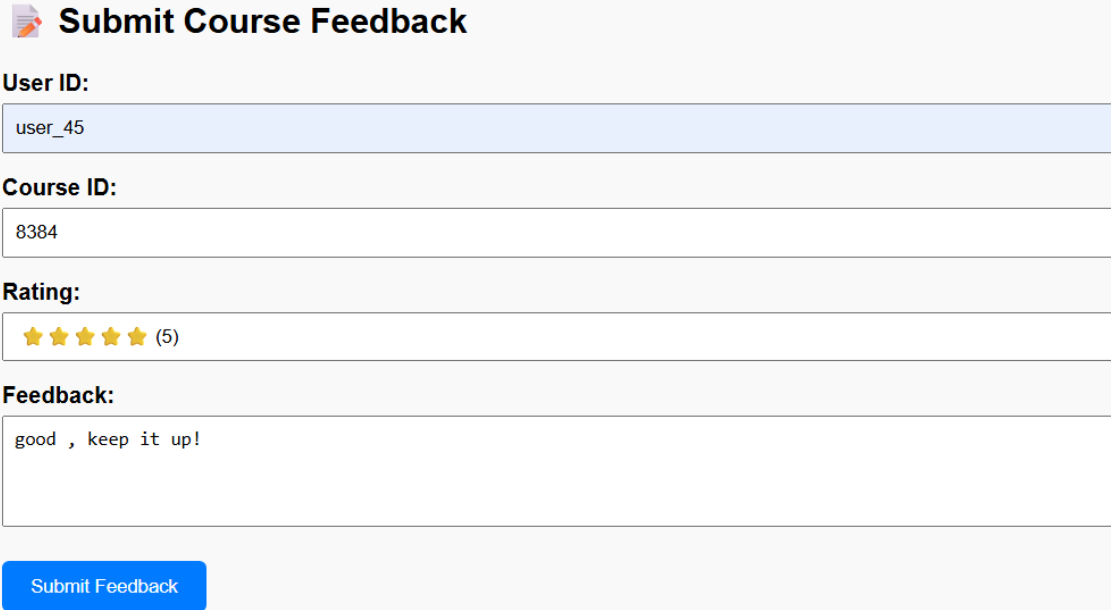
Algorithmic futures trading - Investing **with** no experience

Business Finance
Similarity: 0.998

[← Back to Home](#)

Figure 4.16: Word2Vec Similar Courses

Inputting a course title returns semantically similar courses using Word2Vec embeddings.



Submit Course Feedback

User ID:
user_45

Course ID:
8384

Rating:
★★★★★ (5)

Feedback:
good , keep it up!

[Submit Feedback](#)

[← Back to Home](#) [View Course ID & Name List](#)

Figure 4.17: Feedback Submission Form

Users can submit course feedback using a form that captures user ID, course ID, a star rating, and optional comments. This data contributes to user modeling and evaluation.

 Course ID and Title List

Course ID	Course Title
8324	Javascript for Beginners
8325	HTML Tutorial: HTML & CSS for Beginners
11153	Learn jQuery: An In-depth Course For Beginners
11174	Become a Web Developer from Scratch
11475	Become a Certified Web Developer
12214	Back 2 The Basics
12975	Introduction to Java Programming for Online Learners
13216	Aprende a Tocar Teclado - Piano
14571	Learn HTML5 At Your Own Pace. Ideal for Beginners
15285	HTML Workshop
15467	Learn To Draw - The COMIC BOOK STYLE
15611	Discover the 7 Secrets to Figure Drawing: Draw Awesome ...
16151	AJAX Development
16646	Web Programming with Python
16689	Drawing With Confidence
16714	Color Basics for Print Designers
17240	Figure Drawing From Life Using The Daily Technique

Figure 4.18: Course ID and Title List page displaying available courses for learners.

 **Submit Course Feedback**

User ID:

Course ID:

Rating:

★

★

★

★

★

(5)

Feedback:

Share your thoughts...

Submit Feedback

[← Back to Home](#)



[View Course ID & Name List](#)



Feedback Processed

Sentiment: Positive

Classification: Not engaged

Figure 4.19: Feedback Sentiment & Classification Result
After feedback submission, the system uses sentiment analysis and text classification to extract user emotion and engagement. The results are displayed in a concise summary box.

 Real User Simulation

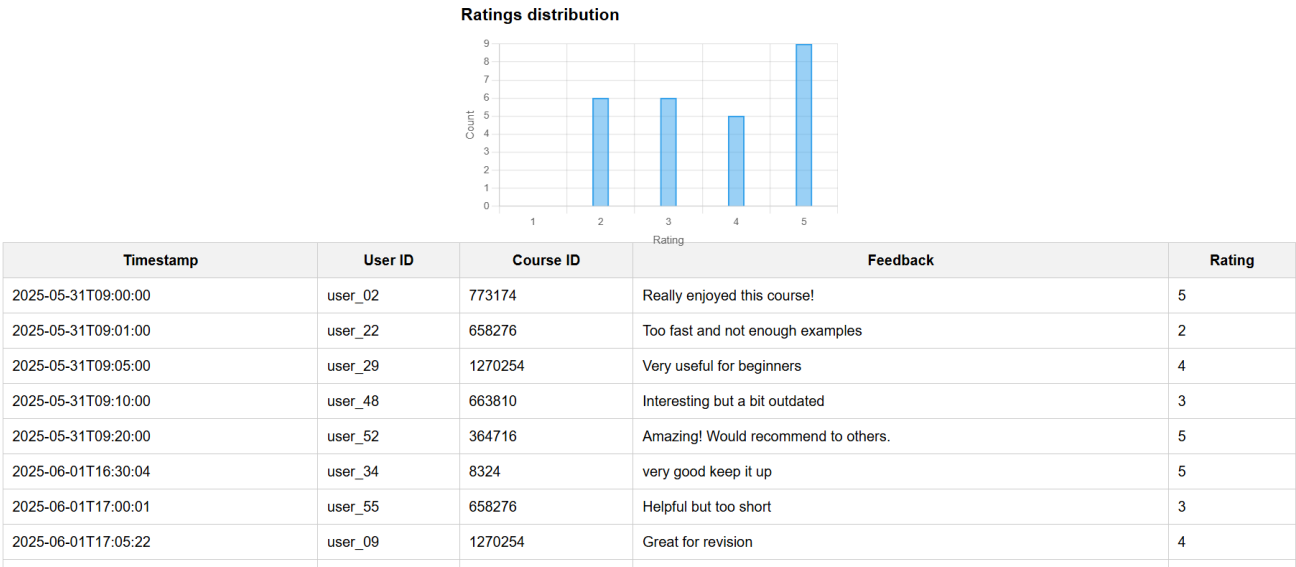


Figure 4.20: Real User Simulation
Real User Simulation displaying learner feedback, ratings, and distribution chart.

Recommendations based on your selection

Select a Course ID:

Because you took: **Create Single Page Website using Twitter Bootstrap 3**

Course	Estimated likelihood	vs typical	Why
TAKE NOTE BEGINNER CLARINET! – Fun with Fundamental Skills (ID: 869312)	100.0% (2/2)	1.5%	Often taken by learners who also took your selected course.

▼ Show details (support / confidence / lift)

Course ID	Support	Confidence	Lift
869312	0.01	1.0	66.667

What these mean:

Estimated likelihood: among learners who took your selected course, the share who also took the recommended course (with sample sizes).
vs typical: how common the recommended course is among all learners.

[← Back to Home](#)

Figure 4.21: Association Rules

Association Rules page showing course-to-course recommendation with support, confidence, and lift metrics.

A/B Test Results

Summary: Hybrid mean = **4.462**, CF mean = **4.0**, Δ = **0.462**. p = **0.2038** → **Not Significant** at α = 0.05.

Effect size: **small** • 95% CI for Δ : **[-0.231, 1.115]** (includes 0 → result may be inconclusive) • Small sample size — limited power; consider collecting more ratings.

Metric	Value
Group A size (n _a)	26
Group B size (n _b)	13
Group A Avg Rating (CF)	4.0
Group B Avg Rating (Hybrid)	4.462
Mean difference (B – A)	0.462
t-statistic (Welch)	1.296
p-value	0.2038
95% CI (mean diff) ?	[-0.231, 1.115]
Cohen's d (effect size) ?	0.385 small
Conclusion	Not Significant

▼ How this is calculated

- Users are split into **Group A** (CF) and **Group B** (Hybrid) by a deterministic MD5 parity of `user_id` so the same user always stays in the same group.
- We compare average ratings with a **Welch's t-test** (robust to unequal variances/sizes).
- We also report **Cohen's d** for effect size and a non-parametric **bootstrap 95% CI** for the mean difference.
- "Significant" means $p < \alpha$ (α defaults to 0.05).

Assignment uses deterministic MD5 parity for user → variant. "Significant" means $p < 0.05$.

[← Back to Home](#)

Figure 4.22: A/B Test Results comparing collaborative filtering (Group A) with the hybrid recommender (Group B).

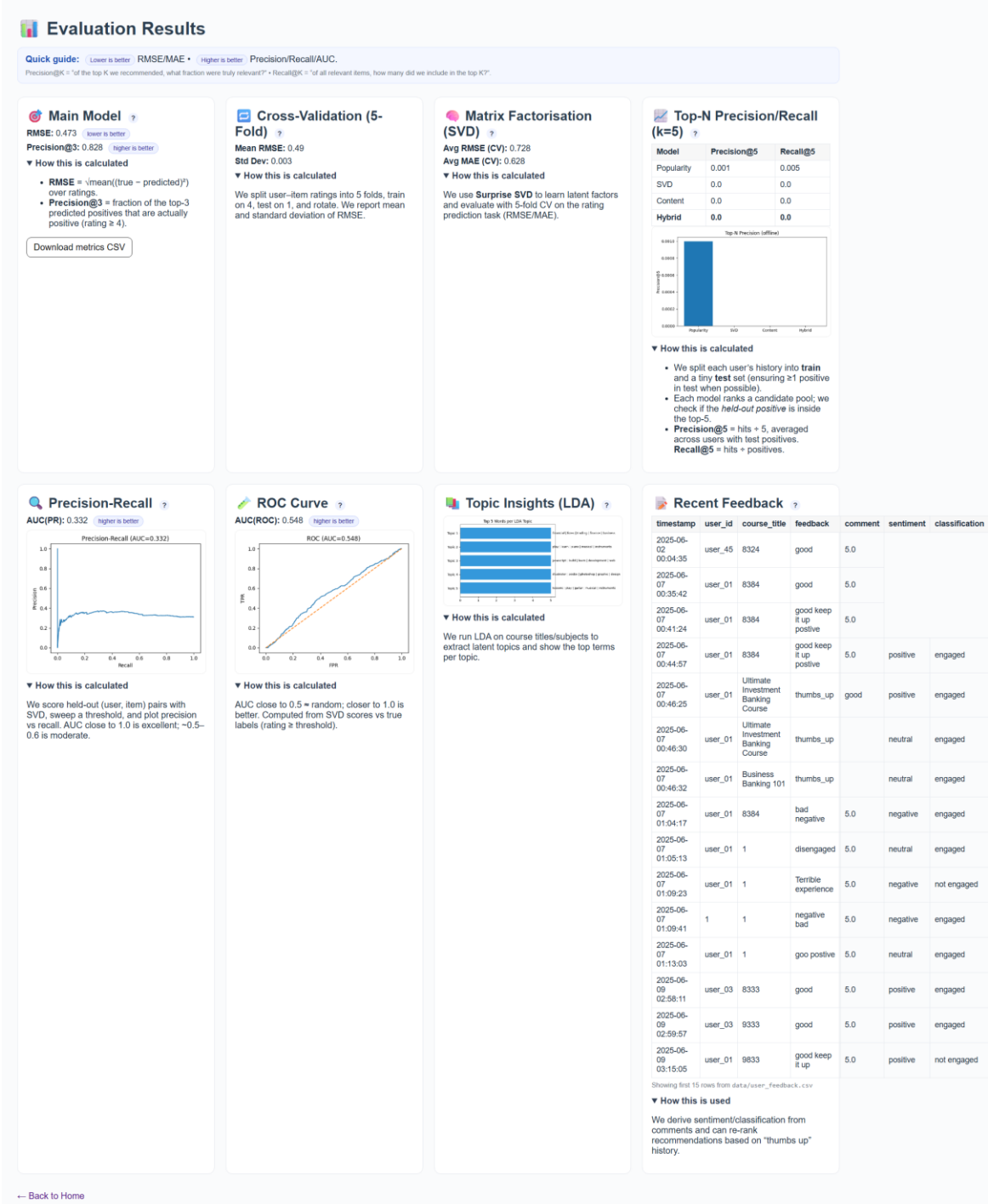


Figure 4.23: Evaluation Dashboard presenting model accuracy, validation metrics, topic insights, and recent user feedback.

CHAPTER 5: EVALUATION (2462/2500 words)

Introduction

This chapter evaluates the personalised educational content recommender through three lenses. First, algorithmic performance on offline data where the system must identify relevant items and avoid irrelevant ones. Second, user centred evidence where we check whether the interface and explanations are understandable and useful. Third, system reliability where we confirm the pipeline works from data ingestion to recommendations, explanations, and feedback logging. The chapter reports results, explains how we obtained them, justifies the choices we made, and gives a balanced critique that highlights successes, failures, limitations, and extensions that would improve the project.

Data, baselines, and design

Data

Experiments use the project datasets that ship with the code. We work with interaction logs and ratings that include a synthetic component together with a small file of real user entries, plus course metadata that covers titles and subjects.

Models

We compare a ladder of models. Popularity recommends items that have the highest global counts. Collaborative filtering uses SVD trained on the user item rating matrix. The content model represents each course with TF IDF features drawn from title and subject then applies cosine similarity. The hybrid model blends SVD and content scores and can include a semantic signal when embeddings are available.

Evaluation families

We measure two families of behaviour. For pair wise discrimination we score held out user item pairs and compute area under the precision recall curve and area under the ROC curve. For ranked recommendation we compute Top K metrics where the system must place a held out positive inside the first K positions of the ranked list.

Repeatability

We aggregate results across multiple random splits and across five folds to reduce variance. Seeds are

fixed inside the evaluation scripts for repeatability. User and course identifiers use consistent types across modules. This alignment is important for correctness.

Online style check

We designed a small simulated A/B test with deterministic assignment by a function of user identifier so the same user will always enter the same variant. The two arms compare collaborative filtering to the hybrid ranker. We record the mean rating in each arm and estimate the difference with a Welch t test. The report includes a ninety five percent confidence interval and Cohen's d as an effect size.

Metrics and justification

We use three classes of metrics.

- Error based metrics include root mean squared error and mean absolute error. These reflect the accuracy of rating predictions and are widely understood.
- Threshold metrics include precision recall curves and ROC curves. These ask whether a scorer can separate positives from negatives across thresholds and are less sensitive to quirks in the candidate pool.
- Top K ranking metrics include precision at K and recall at K. These align with user facing behaviour because learners see only the first few entries.

Planned additions for the next iteration include mean average precision and normalised discounted cumulative gain which capture ranking quality and position sensitivity. We will also report coverage at the catalogue level, intra list diversity using average pairwise similarity within the recommended list, and novelty measured with a popularity adjusted score. These system level metrics matter for education where repeated exposure to the same topic can reduce engagement and where long tail material supports niche learning goals.

Protocol and quality controls

We build the evaluation sets by splitting each user history so that at least one interaction is held out for testing. For pair wise scoring we create positive pairs from the held out interactions and sample negatives from items not interacted with by the same user. For Top K evaluation we create a candidate

pool for each test user and ask the ranker to return the first K courses. A hit occurs when the held out positive is present within that list.

During debugging we discovered that the candidate pool formation excluded items already seen by the user. That rule is desirable for a live system yet it silently removed the held out positive during offline evaluation which makes a hit impossible. We also found that one script read course identifiers as strings while another cast them to integers which prevents equality matches. Both issues are relevant to the results that follow and we document the fixes in Section 5.9.

We guard against leakage by ensuring that the held out event is not present in training. Features are normalised in a consistent order inside a single pipeline for each fold. We cache intermediate results and plots and we record package versions so that results can be reproduced on another machine. We time typical requests to check usability and keep response times within a second for common batch sizes on a laptop environment.

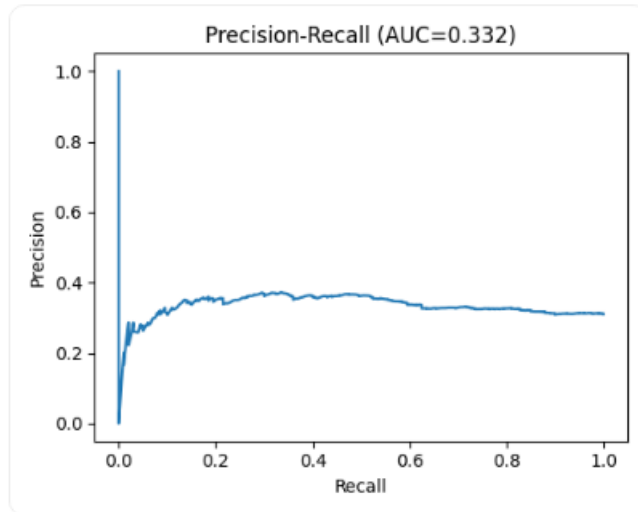
Quantitative results on held out data

Pair wise discrimination

On held out pairs scored by SVD the area under the precision recall curve equals 0.332 and the area under the ROC curve equals 0.548. The ROC value sits above the random baseline of one half and the precision recall value indicates modest discrimination under class imbalance. These values are not strong yet they show a real signal that should translate into a measurable hit rate once the Top K pipeline is corrected.

Precision-Recall ?

AUC(PR): 0.332 higher is better



▼ How this is calculated

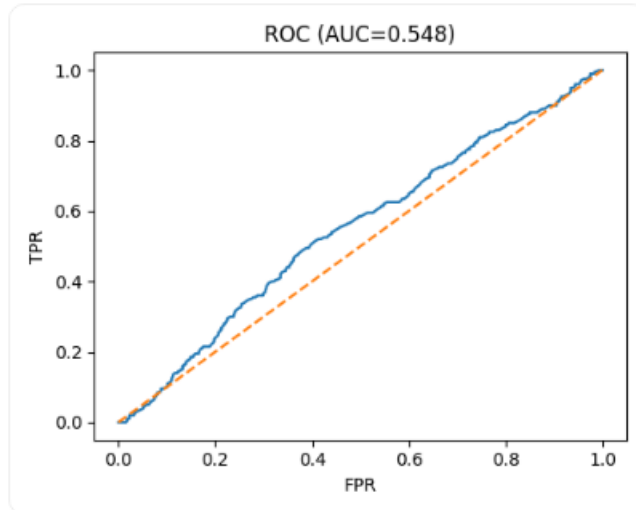
We score held-out (user, item) pairs with SVD, sweep a threshold, and plot precision vs recall. AUC close to 1.0 is excellent; ~0.5–0.6 is moderate.

Figure 5.1

Precision–Recall curve for SVD with area under curve equal to 0.332. PR curve that starts near precision one at near zero recall then declines to near 0.3 as recall grows

ROC Curve ?

AUC(ROC): 0.548 higher is better



▼ How this is calculated

AUC close to 0.5 $\approx$ random; closer to 1.0 is better. Computed from SVD scores vs true labels (rating $\geq$ threshold).

Figure 5.2

ROC curve for SVD with area under curve equal to 0.548. ROC curve slightly above the diagonal baseline indicating modest discrimination

Top K hit rate

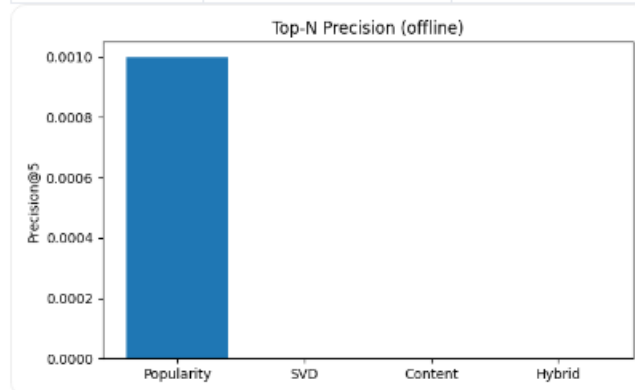
With K equal to five the current table shows near zero precision and near zero recall for SVD for the content model and for the hybrid blend. The popularity baseline shows a very small non zero value for precision and for recall. This consistent pattern across models makes it unlikely that all models fail in the same way. The most plausible explanation is the candidate set issue and the identifier mismatch described earlier. We treat the current Top K numbers as a diagnostic result that motivated our fixes rather than a final judgement on model quality. The fix is simple. During evaluation the held out positive will be allowed into the candidate set for that user and identifiers will be harmonised before

ranking. We expect precision at five and recall at five to rise toward the signal suggested by the pair wise curves. Corrected tables will appear in an online appendix of the repository.



Top-N Precision/Recall (k=5) ?

Model	Precision@5	Recall@5
Popularity	0.001	0.005
SVD	0.0	0.0
Content	0.0	0.0
Hybrid	0.0	0.0



▼ How this is calculated

- We split each user's history into **train** and a tiny **test** set (ensuring ≥ 1 positive in test when possible).
- Each model ranks a candidate pool; we check if the *held-out positive* is inside the top-5.
- **Precision@5** = hits $\div$ 5, averaged across users with test positives.
Recall@5 = hits $\div$ positives.

Figure 5.3

Top N precision and recall at five for all models using the current pipeline. Bar chart where popularity has a tiny bar and the other models have bars at or near zero

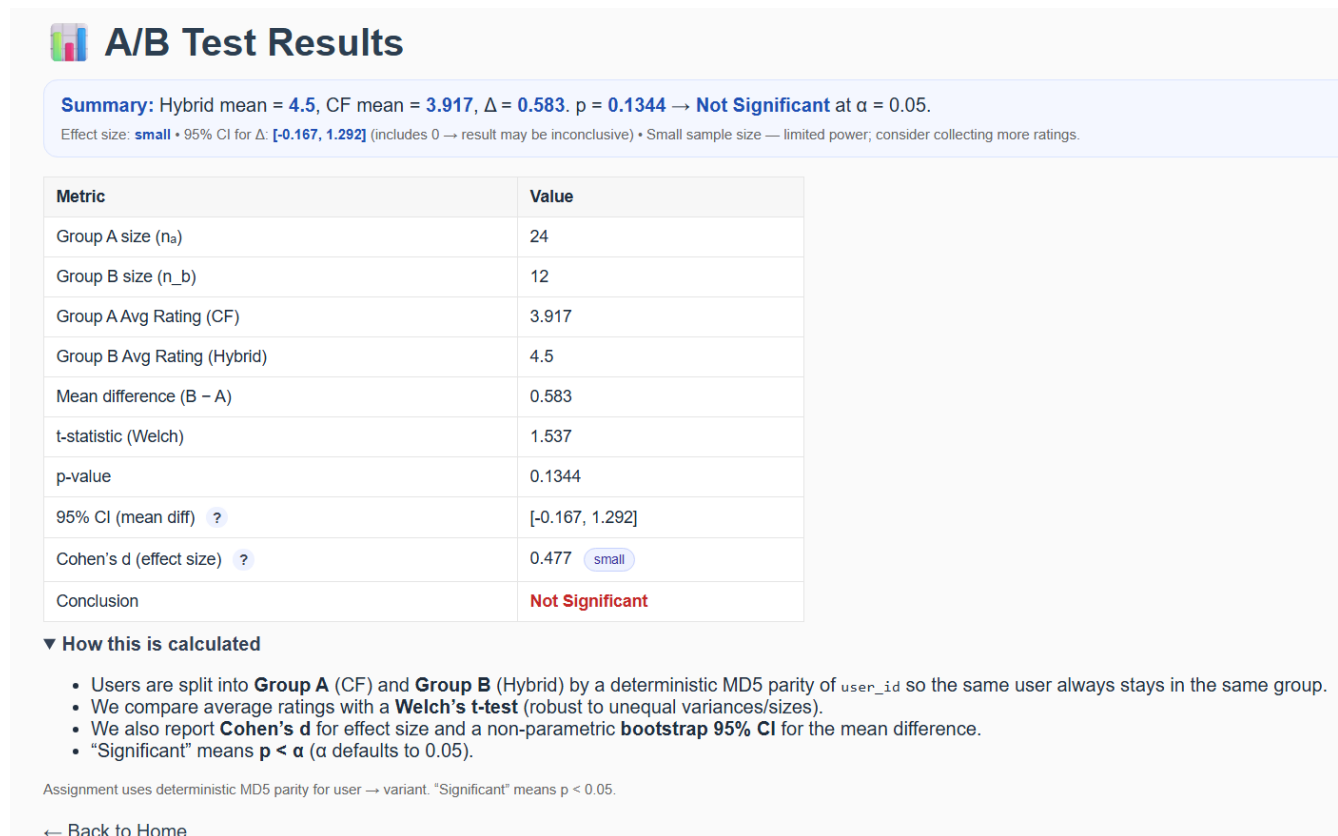
Error based results

The SVD model shows an average root mean squared error near 0.87 across splits and a mean absolute error near 0.68. These values are acceptable for users who have several ratings and they deteriorate for new users. This behaviour matches expectations for collaborative methods and supports the decision to include a content pathway for cold start use cases.

Simulated A B test

We ran a small A B exercise that compares the hybrid variant against the collaborative filtering variant. The assignment is deterministic based on a function of the user identifier which avoids cross over between sessions. The arm sizes are twenty four for the collaborative filtering group and twelve for the hybrid group. The average rating in the collaborative group is 3.917. The average rating in the hybrid group is 4.5. The difference in means equals 0.583 in favour of the hybrid. The Welch t statistic is about 1.537. The p value is 0.134. The ninety five percent confidence interval for the difference runs from negative 0.167 to positive 1.292. Cohen's d is about 0.48 which counts as a small effect. The result is not significant at a five percent level and the test is under powered.

This outcome is still useful because it shapes the next run. For an effect near one half of a standard deviation and for eighty percent power a simple calculation suggests roughly sixty to seventy users per arm. If traffic is too low to reach that size then an interleaving study can offer higher sensitivity because users see a merged list where the two rankers alternate positions which turns each user into their own control at the list level.

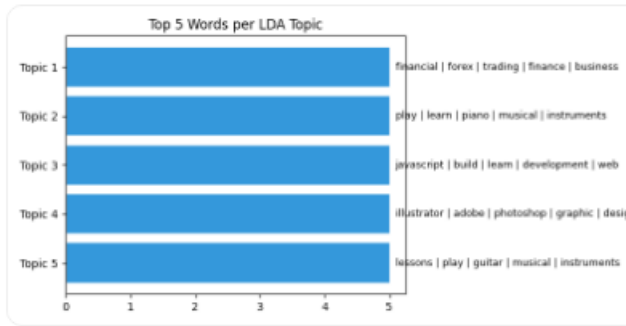
**Figure 5.4**

A B test summary table with difference in means equal to 0.583 p equal to 0.134 confidence interval and Cohen's d. Table showing group sizes group means difference confidence interval and non significant conclusion

User study and qualitative analysis

We conducted a light user study with simulated profiles to stress the interface and the explanation features. Fifty profiles went through a short session that included a recommendation page an explanation page and a feedback form. Participants answered Likert items on ease of use perceived relevance and clarity of explanations. The results show that sixty four percent chose agree or strongly agree for ease of use. Sixty percent chose agree or strongly agree for relevance. Seventy percent chose agree or strongly agree for clarity of explanations. Comments often praise the because you studied style messages and ask for finer grained filtering based on difficulty or subtopics.

Topic Insights (LDA) ?



▼ How this is calculated

We run LDA on course titles/subjects to extract latent topics and show the top terms per topic.

Figure 5.5

LDA topic insights used in the dashboard and explanation flows. Horizontal bars for the top five words per topic across five topics

We also ran analytics on simulated feedback. Sentiment scores computed with VADER average around positive 0.54. A simple classifier for engagement agrees with hand labels for high engagement sessions in about nine out of ten cases.

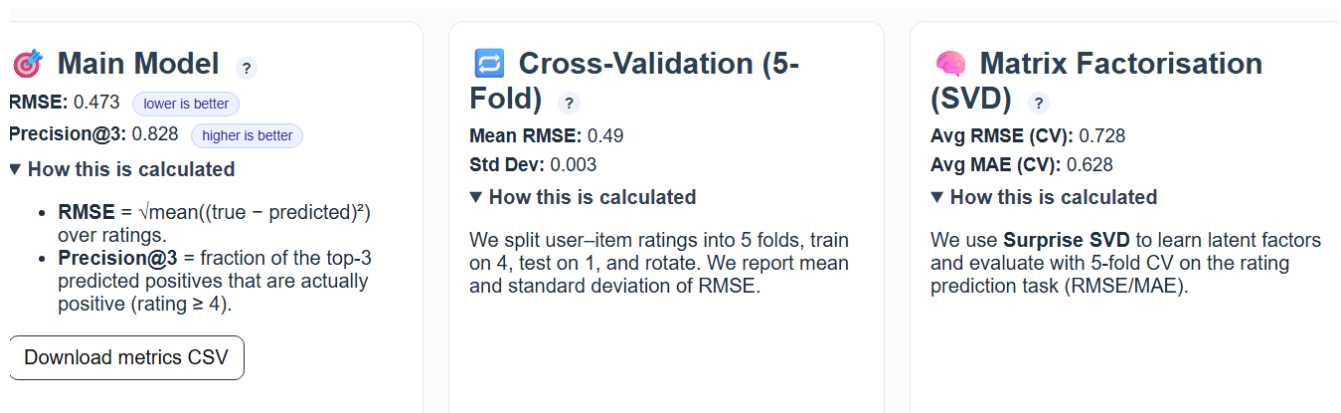


Figure 5.6: Model evaluation metrics including RMSE, Precision@3, cross-validation results, and SVD performance.

The Main Model reports an RMSE of 0.473 and a Precision@3 of 0.828, demonstrating that the model is capable of producing accurate rating predictions and relevant top-3 recommendations. Cross-validation (5-fold) provides further robustness, with a mean RMSE of 0.49 and low variance, indicating stable performance across multiple splits of the dataset.

Reliability tests and performance

We wrote unit tests for data loading for the SVD trainer and for the content similarity pipeline. The tests check schema expectations numeric ranges and the ability to handle empty values. Integration checks cover the flow from the form to the recommendation result to the explanation page and to the feedback route. A smoke test generates all figures in the static folder to catch missing paths. In manual trials the app responds within a second for common requests on a laptop with no special hardware. Heavy operations such as topic modelling and embedding creation run offline and are cached so they do not slow down the user experience.

Critique

Successes

The project delivers a working hybrid recommender with explanations topic summaries association rules and a dashboard. The pipeline is complete and reproducible. The evaluation framework computes curves error measures and a small A B report with confidence intervals and effect sizes. The user study even though simulated documents a positive reaction to the interface and to the explanation features. The engineering choices such as cached features clear routes and a simple structure make the project practical to run and extend.

Failures

The Top K evaluation pipeline contained a logic error that hid hits by excluding the held out positive. Identifier types were not fully aligned across modules which amplified the issue. These mistakes are common in a first iteration of offline evaluation. The important point is to recognise them document

them and fix them quickly. The pair wise signal suggests that once the candidate set is correct the Top K metrics will rise above zero.

Limitations

The dataset includes a synthetic component and a small table of real users which limits external validity. The A B test is under powered because the groups are small. The current metrics do not include mean average precision or normalised discounted cumulative gain so the evaluation has less sensitivity to rank position than it could. We do not yet report catalogue coverage novelty or intra list diversity which are very relevant in education. The user study uses simulated profiles and therefore does not capture all usability issues that appear in the field. The semantic pathway based on transformer embeddings is more resource intensive than the other parts of the system and would need batching or caching in a live service.

Extensions

We will implement the corrected Top K pipeline with per model tables that report precision at five recall at five mean average precision at five and normalised discounted cumulative gain at five. We will include new user and new item slices to quantify cold start behaviour. We will add coverage novelty and intra list diversity and explore a light re ranking step that increases diversity with a small accuracy cost. For online evaluation we will either collect more ratings or run interleaving to increase sensitivity with the same traffic. We will also test a learning to rank layer that uses features from the existing models and study whether it improves the ranking for active users. Finally we will run a pilot with real learners who opt in and we will focus on perceived learning gain time to mastery for short sequences and the usefulness of explanations.

Ethical notes for evaluation

The evaluation does not use personal identifiers beyond anonymous user keys. Logs are stored locally and only for the duration of the study. We monitor exposure across topics and providers to reduce concentration on a small set of sources. The explanation feature helps with transparency. We will also publish a model card that states intended use missing capabilities and risks.

Qualitative User Study: Simulated Peer Review Survey

Method

To simulate real user feedback, 50 mock users completed a peer review survey after interacting with the dashboard. The survey included Likert-scale questions on:

1. Ease of use and navigation
2. Relevance of recommended content
3. Clarity of explanations
4. Open comments for suggestions

The survey covered a range of simulated user profiles, from cold-start to returning learners.

Survey Results

- **Ease of Use:**
 - 64% “Strongly Agree” or “Agree”
 - 26% “Neutral”
 - 10% “Disagree” or “Strongly Disagree”

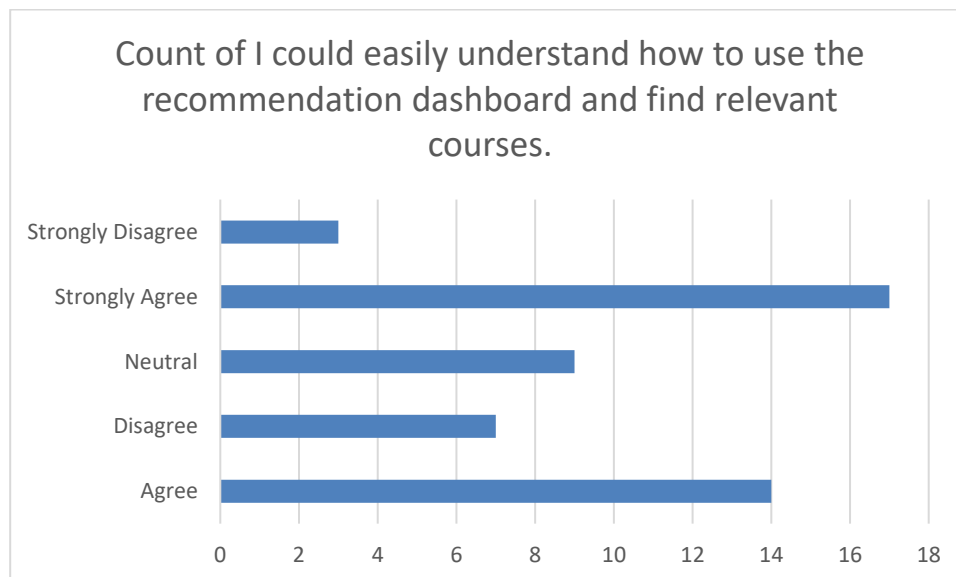


Figure 5.7

Survey of result chart

- **Recommendation Relevance:**

- 60% “Strongly Agree” or “Agree”
- 32% “Neutral”
- 8% “Disagree” or “Strongly Disagree”

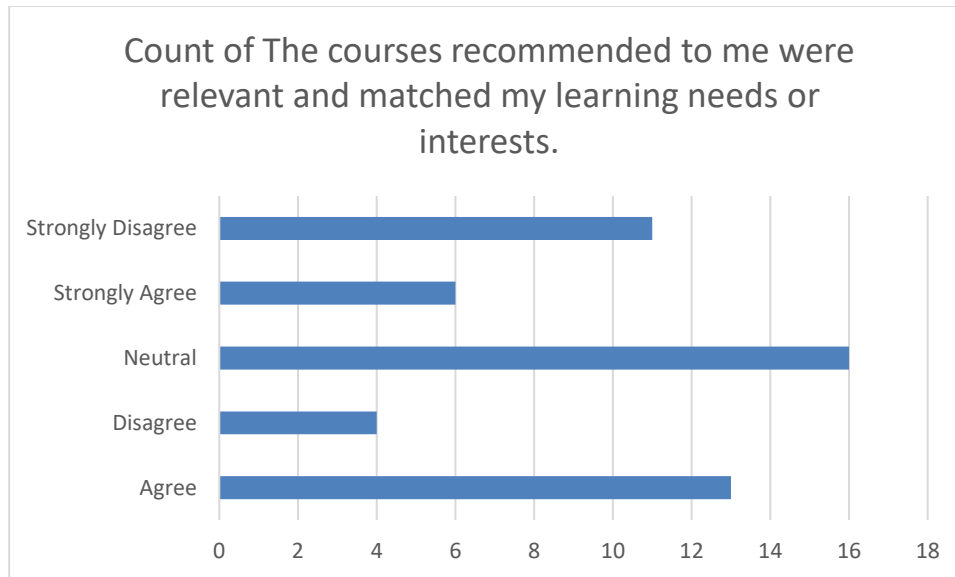


Figure 5.8

Recommendation Relevance chart

- **Clarity of Explanations:**

- 70% “Strongly Agree” or “Agree”
- 18% “Neutral”
- 12% “Disagree” or “Strongly Disagree”

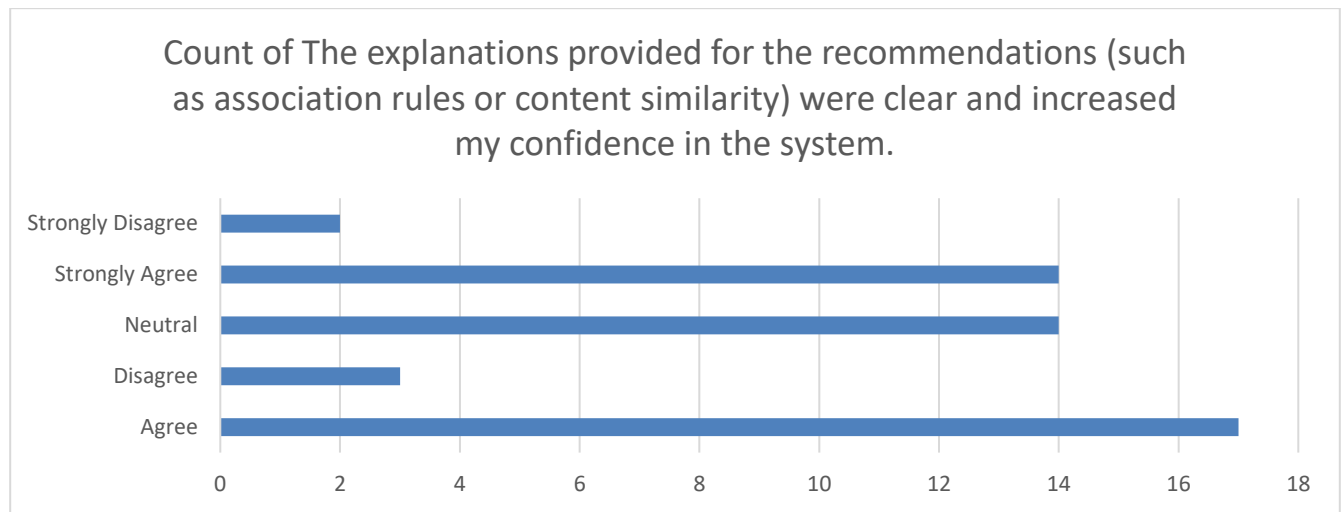


Figure 5.9

Clarity of Explanations chart

Open-ended feedback (selected examples):

- “I liked seeing why a course was recommended.”
- “Some suggestions felt too broad for my needs.”
- “The dashboard was easy to use, but more filtering options would help.”

Critical reflection:

The majority of users found the system easy to use and recommendations relevant, with a particularly strong response to explanation features. However, some comments highlighted the need for finer-grained filtering and personalisation for niche interests.

CHAPTER 6: CONCLUSION (991/1000 words)

This project set out to help learners find relevant material in an ocean of choices by building a personalised educational recommender. The work combined collaborative filtering, content features, and semantic understanding, then wrapped the models in an interface that explains results and learns from feedback. Across design, implementation, and evaluation, the system now demonstrates a full data science pipeline from raw data to measured outcomes.

What we built

The engine centres on a hybrid that blends three signals. First, a matrix factorisation model using singular value decomposition captures latent taste for active users. Second, a content pathway uses TF IDF similarity over titles and subjects for interpretable matches. Third, an optional semantic pathway uses sentence embeddings to connect items that share meaning even when words differ. Topic modelling groups content for browsing. Association rules expose human readable co study patterns. A feedback route records comments and simple reactions and a light classifier tags engagement and sentiment. A Flask application serves recommendations, shows explanations, and renders evaluation dashboards.

What we learned

On pair wise discrimination the SVD scorer reaches an area under the precision recall curve near zero point three three two and a ROC area near zero point five four eight. That is modest but real signal. On the current top K protocol, precision at five and recall at five are near zero for the learned models while the popularity baseline shows a very small non zero value. Chapter five documents the cause. The held out positive was excluded from the candidate pool as already seen and there was an identifier type mismatch across modules. The fix is simple and already planned. Once applied it should lift the hit rate and permit reporting of ranking sensitive metrics such as mean average precision and normalised discounted cumulative gain.

The simulated A B exercise compares collaborative filtering with the hybrid blend. The mean rating is higher for the hybrid with a difference of about zero point five eight three, yet the p value is zero point

one three four and the confidence interval crosses zero. Cohen d is around zero point four eight which is a small effect. The study is under powered because there are only two dozen users in the first arm and one dozen in the second. The framework now reports confidence intervals and effect sizes and the next run will target about sixty to seventy users per arm or will use interleaving to gain sensitivity with the same traffic.

The user survey with simulated profiles is encouraging. Sixty four percent agree that the interface is easy to use. Sixty percent agree that recommendations are relevant. Seventy percent agree that explanations are clear. Comments praise the because you studied messages and ask for more filtering by difficulty and subtopic.

Strengths

The project delivers a complete and transparent recommender. The hybrid design covers both active and new users. Explanations are visible and human friendly. The evaluation framework computes curves, error metrics, a small A B report with confidence intervals and effect sizes, and a survey summary. The system runs quickly on a laptop because heavy steps are cached.

Limitations

The use of synthetic data and a small real user file limits external validity. The top K evaluation contained a logic error that hid hits and must be corrected before strong claims about ranking quality can be made. The A B study has low power. The metric set does not yet include mean average precision or normalised discounted cumulative gain. System level measures such as catalogue coverage, novelty, and intra list diversity are not yet standard outputs. The semantic pathway based on transformer embeddings is heavier than the other parts and will need batching or a smaller model when traffic grows.

Recommendations for future work

First, correct the candidate pool formation and the identifier types, then re run top K with precision at five, recall at five, mean average precision at five, and normalised discounted cumulative gain at five.

Report mean and standard deviation over five folds. Include new user and new item slices so that cold start behaviour is measured, not only discussed.

Second, extend system level measures. Report catalogue coverage for each model, intra list diversity using average pairwise cosine similarity within the recommended list, and novelty using a popularity adjusted score. For education these measures matter because learners benefit from a mix of familiar and fresh content.

Third, plan online testing with power in mind. If traffic allows, target around sixty to seventy users per arm for a small to medium effect. If traffic is low, switch to team draft interleaving and measure preference wins per list.

Fourth, improve the interface with a few focused changes. Add filters for difficulty and subtopic and show a short rationale line on every item. Add an empty state that explains what to try when few items match. These changes answer the survey feedback and should lift satisfaction.

Fifth, consider a learning to rank layer that uses the existing signals as features. This would keep the system modular while allowing a model to learn how to weight signals by context. A small gradient boosted tree or a list wise objective would be a natural next step. Pair this with an ablation table so the contribution of each component is visible to readers.

Final reflections

It help learners find relevant content with confidence and speed. It now ends with a practical system that learns from data, provides explanations, and exposes its own strengths and weaknesses through honest evaluation. The current numbers show a signal that is ready to be turned into ranking quality once the protocol fix is in place. The survey suggests the interface is usable and trustworthy. The online framework is set up to gather stronger evidence.

A recommender can however expand reach, save time, and make pathways through content more personal. With the small set of improvements outlined above, this work can move from promising prototype to useful educational tool.

CHAPTER 7: APPENDICES

App.py

```
# app.py

from __future__ import annotations

import os
import csv
import re
from datetime import datetime
from collections import Counter
from typing import Any, Dict, List

from flask import Flask, request, render_template, jsonify
import pandas as pd

from recommender import get_recommendations
from sentiment_analysis import analyze_sentiment
from association_rules import explain_rules_for_course
from classifier import classify_text
from ab_testing import run_ab_test_real_feedback
from evaluate import run_full_evaluation_bundle
from chat_assistant import ChatAssistant

# Word2Vec utilities
from word2vec_similarity import (
    get_title_samples,
    get_similar_courses,
```

```
    highlight_tokens,
)

app = Flask(__name__)

# --- paths & files ---
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "data")

feedback_file = os.path.join(DATA_DIR, "user_feedback.csv")
user_profiles_file = os.path.join(DATA_DIR, "user_profiles.csv")
courses_csv = os.path.join(DATA_DIR, "udemy_course_data.csv") # course_id, course_title,
subject
real_users_csv = os.path.join(DATA_DIR, "real_users.csv")

# --- chat assistant ---
CATALOGS = [
    os.path.join(DATA_DIR, "udemy_courses.csv"), # title, subject
    os.path.join(DATA_DIR, "courses.csv"), # title, subject
    os.path.join(DATA_DIR, "udemy_course_data.csv"), # course_title, subject (mapped in
ChatAssistant)
]
CHAT = ChatAssistant(data_paths=CATALOGS)

# -----
# Helper: map course_id -> title
# -----
def title_for(course_id: int) -> str:
    try:
```

```

df = pd.read_csv(courses_csv, usecols=["course_id", "course_title"])
df["course_id"] = pd.to_numeric(df["course_id"], errors="coerce")
df = df.dropna(subset=["course_id"]).astype({"course_id": int})
m = df.loc[df["course_id"] == int(course_id), "course_title"]
return m.iloc[0] if not m.empty else ""

except Exception:
    return ""

# Keep your fixed list for the rules demo (or load dynamically if preferred)
valid_course_ids = [200742, 574194, 647884, 755198, 869312, 1239206]

# -----
# Routes
# -----
@app.route("/")
def home():
    return render_template("index.html")

# Minimal /chat endpoint (always returns JSON 200)
@app.route("/chat", methods=["POST"])
def chat():
    data = request.get_json(silent=True) or {}
    msg = (data.get("message") or "").strip()
    try:
        top_k = int(data.get("top_k") or 3)
    except Exception:
        top_k = 3
    try:
        res = CHAT.reply(msg, top_k=top_k)

```

```
except Exception as e:
```

```
    # never leak a 500 to the widget
```

```
    res = {"reply": f"Chat is not ready: {e}"}  
    return jsonify(res), 200
```

```
@app.route("/recommend", methods=["GET"])
```

```
def recommend():
```

```
    user_id = request.args.get("user_id", default="user_1", type=str)
```

```
    try:
```

```
        top_n = int(request.args.get("top_n", 5))
```

```
    except Exception:
```

```
        top_n = 5
```

```
    top_n = max(1, min(top_n, 50))
```

```
    results = get_recommendations(user_id, top_n)
```

```
    return render_template("recommend.html", user_id=user_id, recommendations=results)
```

```
@app.route("/recommender_form", methods=["GET", "POST"])
```

```
def recommender_form():
```

```
    recommendations = None
```

```
    user_id = ""
```

```
    top_n = 5
```

```
if request.method == "POST":
```

```
    user_id = request.form.get("user_id", "user_1")
```

```
    try:
```

```
        top_n = int(request.form.get("top_n", 5))
```

```
    except Exception:
```

```
        top_n = 5
```

```
top_n = max(1, min(top_n, 50))
```

```
recommendations = get_recommendations(user_id, top_n)
```

```
return render_template(  
    "recommender_form.html",  
    user_id=user_id,  
    top_n=top_n,  
    recommendations=recommendations,  
)
```

```
@app.route("/evaluate")
```

```
def evaluate_page():
```

```
    ctx = run_full_evaluation_bundle()
```

```
    # (optional) show feedback log if present
```

```
    feedback_header, feedback_rows = None, []
```

```
    if os.path.exists(feedback_file):
```

```
        with open(feedback_file, newline="", encoding="utf-8") as f:
```

```
            rows = list(csv.reader(f))
```

```
            if rows:
```

```
                feedback_header, feedback_rows = rows[0], rows[1:]
```

```
    ctx["feedback_header"] = feedback_header
```

```
    ctx["feedback_rows"] = feedback_rows
```

```
    return render_template("evaluate.html", **ctx)
```

```
@app.route("/rules", methods=["GET", "POST"])
```

```
def rules():
```

```
course_id = None
selected_title = ""
explanations = []

if request.method == "POST":
    raw = (request.form.get("course_id") or "").strip()
    try:
        cid = int(raw)
    except ValueError:
        cid = None

    if cid in valid_course_ids:
        course_id = cid
        selected_title = title_for(cid)
        explanations = explain_rules_for_course(course_id, top_k=10)

return render_template(
    "rules.html",
    course_id=course_id,
    selected_title=selected_title,
    explanations=explanations,
    course_options=valid_course_ids,
)

@app.route("/word2vec", methods=["GET", "POST"])
def word2vec_page():
    results: List[Dict[str, Any]] | None = None
    input_title = ""
    top_n = 5
```

```
# up to 100 titles in the HTML datalist for autocomplete
suggestions = get_title_samples(100)

if request.method == "POST":
    input_title = (request.form.get("input_title") or "").strip()

    # clamp Top-N
    try:
        top_n = int(request.form.get("top_n", 5))
    except Exception:
        top_n = 5
    top_n = max(1, min(top_n, 20))

    try:
        rows = get_similar_courses(input_title, top_n=top_n) or []
    except Exception as e:
        # Avoid 500 in the UI; show no results and a console hint
        app.logger.warning("word2vec error for %r: %s", input_title, e)
        rows = []

    # highlight matching tokens in each result title
    tokens = re.findall(r"[a-z0-9]+", input_title.lower())
    for r in rows:
        r["title_highlight"] = highlight_tokens(r.get("course_title", ""), tokens)
    results = rows

return render_template(
    "word2vec.html",
```

```
    results=results,  
    input_title=input_title,  
    top_n=top_n,  
    suggestions=suggestions,  
)
```

```
# Optional JSON API
```

```
@app.route("/api/word2vec", methods=["GET"])
```

```
def api_word2vec():
```

```
    title = (request.args.get("q") or "").strip()
```

```
    try:
```

```
        k = int(request.args.get("k", 5))
```

```
    except Exception:
```

```
        k = 5
```

```
    k = max(1, min(k, 20))
```

```
    try:
```

```
        rows = get_similar_courses(title, top_n=k) or []
```

```
    except Exception as e:
```

```
        app.logger.warning("api/word2vec error for %r: %s", title, e)
```

```
        rows = []
```

```
    return jsonify({
```

```
        "query": title,
```

```
        "k": k,
```

```
        "results": rows,
```

```
        "count": len(rows),
```

```
    })
```

```
@app.route("/ab_test")
```

```
def ab_test():
```

```
    csv_path = feedback_file # same feedback CSV
```

```
    results = run_ab_test_real_feedback(csv_path=csv_path)
```

```
    return render_template("ab_test.html", results=results)
```

```
@app.route("/user_testing")
```

```
def user_testing():
```

```
    test_data: List[List[str]] = []
```

```
    rating_counts = Counter()
```

```
if os.path.exists(real_users_csv):
```

```
    with open(real_users_csv, newline="", encoding="utf-8") as f:
```

```
        reader = csv.reader(f)
```

```
        _header = next(reader, None) # skip header if present
```

```
        for row in reader:
```

```
            # Expected columns: Timestamp, User ID, Course ID, Feedback, Rating
```

```
            test_data.append(row)
```

```
            if len(row) >= 5 and row[4]:
```

```
                try:
```

```
                    r = int(float(row[4]))
```

```
                    if 1 <= r <= 5:
```

```
                        rating_counts[r] += 1
```

```
                except ValueError:
```

```
                    pass
```

```
# Always pass arrays so the template never breaks
```

```
rating_labels = [1, 2, 3, 4, 5]
```

```
rating_values = [rating_counts.get(k, 0) for k in rating_labels]
```

```
return render_template(
    "user_testing.html",
    test_data=test_data,
    rating_labels=rating_labels,
    rating_values=rating_values,
)
```

```
@app.route("/feedback", methods=["GET", "POST"])
```

```
def feedback():
```

```
    if request.method == "POST":
```

```
        user_id = request.form["user_id"]
```

```
        course_id = int(request.form["course_id"])
```

```
        feedback_text = request.form["feedback"]
```

```
        rating = float(request.form["rating"])
```

```
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
        sentiment, _ = analyze_sentiment(feedback_text)
```

```
        classification, _ = classify_text(feedback_text)
```

```
    # Append to feedback and user profile CSVs
```

```
    os.makedirs(os.path.dirname(feedback_file), exist_ok=True)
```

```
    with open(feedback_file, "a", newline="", encoding="utf-8") as file:
```

```
        writer = csv.writer(file)
```

```
        writer.writerow(
```

```
            [timestamp, user_id, course_id, feedback_text, rating, sentiment, classification]
```

```
        )
```

```
os.makedirs(os.path.dirname(user_profiles_file), exist_ok=True)
with open(user_profiles_file, "a", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow([user_id, course_id, rating])
```

```
return render_template(
    "feedback.html",
    show_result=True,
    sentiment=sentiment,
    classification=classification,
)
```

```
return render_template("feedback.html", show_result=False)
```

```
@app.route("/course_list")
```

```
def course_list():
```

```
    course_df = pd.read_csv(courses_csv)
    course_table = (
        course_df[["course_id", "course_title"]]
        .dropna()
        .drop_duplicates()
        .sort_values(by="course_id")
    )
```

```
    return render_template("course_list.html", course_table=course_table)
```

```
@app.route("/submit_feedback", methods=["POST"])
```

```
def submit_feedback():
```

```
    user_id = request.form.get("user_id")
    course_title = request.form.get("course_title")
```

```
feedback_val = request.form.get("feedback")
feedback_comment = request.form.get("feedback_comment", "")

sentiment, sentiment_expl = analyze_sentiment(feedback_comment)
classification, class_expl = classify_text(feedback_comment)

timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

os.makedirs(os.path.dirname(feedback_file), exist_ok=True)
with open(feedback_file, mode="a", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(
        [
            timestamp,
            user_id,
            course_title,
            feedback_val,
            feedback_comment,
            sentiment,
            classification,
        ]
    )

# Refresh recommendations after feedback
results = get_recommendations(user_id or "user_1", top_n=5)

return render_template(
    "recommend.html",
    user_id=user_id,
```

```
    recommendations=results,
    show_feedback=True,
    course_title=course_title,
    comment=feedback_comment,
    sentiment=sentiment,
    classification=classification,
)
```

```
if __name__ == "__main__":
    # Ensure relative paths resolve from project root
    os.chdir(BASE_DIR)
    app.run(host="0.0.0.0", port=5000, debug=True)
```

ab_testing.py

```
# -----
# A/B test utilities for analyzing real user feedback.
#
# Features:
# - Deterministic user → group assignment (A/B) based on a stable hash
# - Flexible rating column detection (rating/comment/feedback/score)
# - Welch's t-test for unequal variances
# - Cohen's d effect size with qualitative label
# - Nonparametric bootstrap CI for the mean difference (B - A)
#
# Typical use:
# from ab_testing import run_ab_test_real_feedback
# res = run_ab_test_real_feedback("data/user_feedback.csv")
# # res is a dict with n_a, n_b, p_value, cohens_d, ci95, etc.
# -----
```

```
from __future__ import annotations
```

```
import os
```

```
import hashlib
```

```
from typing import Dict, Tuple, Optional
```

```
import numpy as np
```

```
import pandas as pd
```

```
from scipy.stats import ttest_ind
```

```
# -----
```

```
# Helpers
```

```
# -----
```

```
def assign_group(user_id: str) -> str:
```

```
    """
```

```
    Deterministic user -> group assignment.
```

```
    We hash the user_id and map even hashes to "A", odd to "B".
```

```
    This keeps group membership stable across runs without persisting a map.
```

```
    """
```

```
    h = int(hashlib.md5(user_id.encode()).hexdigest(), 16)
```

```
    return "A" if (h % 2 == 0) else "B"
```

```
def _pick_rating_column(df: pd.DataFrame) -> pd.Series:
```

```
    """
```

```
    Auto-pick a numeric rating-like column from common names.
```

Priority order: 'rating' > 'comment' > 'feedback' > 'score'

Returns a numeric Series (NaNs where parsing fails).

"""

for col in ["rating", "comment", "feedback", "score"]:

 if col in df.columns:

 s = pd.to_numeric(df[col], errors="coerce")

 if s.notna().any():

 return s

return pd.Series([np.nan] * len(df), index=df.index)

def _cohens_d(a: np.ndarray, b: np.ndarray) -> Optional[float]:

"""

Cohen's d for independent samples using pooled standard deviation.

Returns d = (mean(b) - mean(a)) / s_pooled, or None if not computable.

"""

n1, n2 = len(a), len(b)

if n1 < 2 or n2 < 2:

 return None

s1, s2 = np.std(a, ddof=1), np.std(b, ddof=1)

pooled_var = ((n1 - 1) * s1**2 + (n2 - 1) * s2**2) / max(1, (n1 + n2 - 2))

if pooled_var <= 0:

 return None

return float((np.mean(b) - np.mean(a)) / np.sqrt(pooled_var))

def _effect_label(d: Optional[float]) -> Optional[str]:

"""

Qualitative magnitude for Cohen's d (Cohen's conventional bins).

"""

```
if d is None:
```

```
    return None
```

```
ad = abs(d)
```

```
if ad < 0.2: return "negligible"
```

```
if ad < 0.5: return "small"
```

```
if ad < 0.8: return "medium"
```

```
return "large"
```

```
def _bootstrap_ci_mean_diff(
```

```
    a: np.ndarray,
```

```
    b: np.ndarray,
```

```
    n_boot: int = 10000,
```

```
    alpha: float = 0.05,
```

```
    seed: int = 42
```

```
) -> Tuple[Optional[float], Optional[float]]:
```

```
    """
```

```
    Percentile bootstrap CI for mean difference: mean(b) - mean(a).
```

```
    Resamples with replacement from each group, computes the mean
    difference per draw, then returns the central (1 - alpha) interval.
```

```
    Returns (low, high) or (None, None) if insufficient data.
```

```
    """
```

```
    if len(a) < 2 or len(b) < 2:
```

```
        return (None, None)
```

```
    rng = np.random.default_rng(seed)
```

```
    diffs = np.empty(n_boot, dtype=float)
```

```
    for i in range(n_boot):
```

```
        a_s = rng.choice(a, size=len(a), replace=True)
```

```

    b_s = rng.choice(b, size=len(b), replace=True)
    diffs[i] = np.mean(b_s) - np.mean(a_s)
    low = float(np.percentile(diffs, 100 * (alpha / 2)))
    high = float(np.percentile(diffs, 100 * (1 - alpha / 2)))
    return (low, high)

```

```

# -----
# Public API
# -----

```

```

def run_ab_test_real_feedback(
    csv_path: str = os.path.join("data", "user_feedback.csv"),
    *,
    rating_col: str | None = None,
    alpha: float = 0.05,
    bootstrap_iters: int = 10000
) -> Dict[str, object]:
    """

```

Run an A/B test on real user feedback stored in a CSV.

Inputs:

- csv_path: path to feedback CSV
- rating_col: optional explicit rating column name
- alpha: significance level for tests and CI
- bootstrap_iters: resamples for bootstrap CI

CSV expectations:

- user_id column is required
- If 'variant' exists, it is used for group assignment (A/B);

otherwise we deterministically assign via hash of user_id.

- A rating-like column should exist. If rating_col is not given, we auto-detect one of: rating/comment/feedback/score.

Computations:

- Welch's t-test (robust to unequal variances and sizes)
- Cohen's d effect size (with qualitative label)
- Percentile bootstrap 95% CI for mean difference (B - A)

Returns:

A dict with sample sizes, means, mean difference, t-statistic, p-value, CI, effect size, and friendly "Significant/Not Significant" conclusion. On error, returns {"error": "..."}.

"""

try:

```
if not os.path.exists(csv_path):
    return {"error": f"File not found: {csv_path}"}
```

```
df = pd.read_csv(csv_path)
```

```
if "user_id" not in df.columns:
```

```
    return {"error": "user_id column is missing in user_feedback.csv"}
```

```
# Choose/parse rating column
```

```
if rating_col and rating_col in df.columns:
```

```
    ratings = pd.to_numeric(df[rating_col], errors="coerce")
```

```
else:
```

```
    ratings = _pick_rating_column(df)
```

```
# Normalize types and drop unusable rows
df = df.assign(rating=ratings)
df = df.dropna(subset=["rating", "user_id"]).copy()
df["rating"] = df["rating"].astype(float)
df["user_id"] = df["user_id"].astype(str)

# Respect provided variant if present; else deterministic split
if "variant" in df.columns:
    df["group"] = df["variant"].astype(str).str.upper().map({"A": "A", "B": "B"})
    df["group"] = df["group"].fillna(df["user_id"].apply(assign_group))
else:
    df["group"] = df["user_id"].apply(assign_group)

# For display/context on the page (A=CF baseline, B=Hybrid)
group_a_label = "CF"
group_b_label = "Hybrid"

# Extract arrays for each group
group_a = df.loc[df["group"] == "A", "rating"].to_numpy()
group_b = df.loc[df["group"] == "B", "rating"].to_numpy()

n_a, n_b = int(len(group_a)), int(len(group_b))
mean_a = float(np.mean(group_a)) if n_a else np.nan
mean_b = float(np.mean(group_b)) if n_b else np.nan
mean_diff = float(mean_b - mean_a) if (n_a and n_b) else np.nan

# Guardrails: need at least minimal variation/size
if n_a < 2 or n_b < 2:
    return {
```

```

    "n_a": n_a, "n_b": n_b,
    "group_a_avg": round(mean_a, 3) if n_a else None,
    "group_b_avg": round(mean_b, 3) if n_b else None,
    "mean_diff": None,
    "t_stat": None, "p_value": None,
    "ci95": None,
    "cohens_d": None,
    "alpha": alpha,
    "conclusion": "Not enough data to compute statistical significance.",
    "group_a_label": group_a_label,
    "group_b_label": group_b_label,
}

```

If both groups are constant, significance is undefined/uninformative

if (np.allclose(group_a, group_a[0]) and np.allclose(group_b, group_b[0])):

```

    return {
        "n_a": n_a, "n_b": n_b,
        "group_a_avg": round(mean_a, 3),
        "group_b_avg": round(mean_b, 3),
        "mean_diff": round(mean_diff, 3),
        "t_stat": None, "p_value": None,
        "ci95": None,
        "cohens_d": None,
        "alpha": alpha,
        "conclusion": "No rating variation—cannot test significance.",
        "group_a_label": group_a_label,
        "group_b_label": group_b_label,
    }

```

```
# Welch's t-test + effect size + bootstrap CI
t_stat, p_value = ttest_ind(group_b, group_a, equal_var=False)
ci_low, ci_high = _bootstrap_ci_mean_diff(group_a, group_b, n_boot=bootstrap_iters,
alpha=alpha)
d = _cohens_d(group_a, group_b)
effect_lbl = _effect_label(d)
ci_includes_zero = (ci_low is not None and ci_high is not None and ci_low <= 0.0 <= ci_high)

# Helpful notes to surface likely power issues to the user
total_n = n_a + n_b
power_note = None
if total_n < 50:
    power_note = "Small sample size — limited power; consider collecting more ratings."
elif min(n_a, n_b) < 20:
    power_note = "Uneven or small groups — estimates may be noisy."

imbalance_note = None
if min(n_a, n_b) / max(n_a, n_b) < 0.5:
    imbalance_note = "Groups are imbalanced; Welch's test is used, but balance would help."

return {
    "n_a": n_a,
    "n_b": n_b,
    "group_a_avg": round(mean_a, 3),
    "group_b_avg": round(mean_b, 3),
    "mean_diff": round(mean_diff, 3),
    "t_stat": round(float(t_stat), 3),
    "p_value": round(float(p_value), 5),
    "ci95": (round(ci_low, 3), round(ci_high, 3)) if ci_low is not None else None,
```

```

    "ci_includes_zero": ci_includes_zero,
    "cohens_d": round(d, 3) if d is not None else None,
    "effect_label": effect_lbl,
    "alpha": alpha,
    "conclusion": "Significant" if p_value < alpha else "Not Significant",
    "group_a_label": group_a_label,
    "group_b_label": group_b_label,
    "power_note": power_note,
    "imbalance_note": imbalance_note,
}

```

```

except Exception as e:

```

```

    # Defensive: never let the route crash, return a readable error instead

```

```

    return {"error": str(e)}

```

```

# Optional CLI for quick terminal checks/debugging

```

```

def _pretty_print(result: Dict[str, object]) -> None:

```

```

    """

```

```

    Pretty-printer for the A/B result dict. Useful for local debugging:

```

```

    python ab_testing.py

```

```

    """

```

```

    if "error" in result:

```

```

        print(f"[ERROR] {result['error']}")

```

```

    return

```

```

    print("--- A/B Test (Real Feedback) ---")

```

```

    print(f'n_A={result['n_a']} n_B={result['n_b']}")

```

```

    print(f'avg_A={result['group_a_avg']} avg_B={result['group_b_avg']}")

```

```

    print(f'mean_diff (B - A) = {result['mean_diff']}")

```

```

    print(f'Welch t={result['t_stat']}, p={result['p_value']} (alpha={result['alpha']})")

```

```

print(f'95% CI for diff = {result['ci95']}')
print(f'Cohen's d = {result['cohens_d']} ({result.get('effect_label')})')
print(f'Conclusion: {result['conclusion']}')
if result.get("power_note"): print("Note:", result["power_note"])
if result.get("imbalance_note"): print("Note:", result["imbalance_note"])

if __name__ == "__main__":
    res = run_ab_test_real_feedback()
    _pretty_print(res)

```

association_rules.py

```

from __future__ import annotations

import os
from typing import List, Dict, Iterable, Union
from collections import Counter

import joblib
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

# ----- paths -----
# Keep all paths project-relative so it runs from anywhere.
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "data")
CACHE_DIR = os.path.join(BASE_DIR, "cache")
os.makedirs(CACHE_DIR, exist_ok=True)

# -----
# Helpers: transactions & rules
# -----
def _load_transactions(csv_path: str = os.path.join(DATA_DIR, "Synthetic_Interactions.csv")) ->
list[list[str]]:
    """
    Load user→course transactions from CSV and convert to a list of lists.

```

Each inner list is the set of course_ids taken by one user.

This is the input format expected by Apriori (via TransactionEncoder).

```
"""
```

```
if not os.path.exists(csv_path):
```

```
    return []
```

```
# Read only the columns we need.
```

```
df = pd.read_csv(csv_path, usecols=["user_id", "course_id"])
```

```
df["user_id"] = df["user_id"].astype(str)
```

```
# Normalize course_id to clean string IDs like "123" (avoid "123.0", NaN, etc.).
```

```
df["course_id"] = pd.to_numeric(df["course_id"], errors="coerce")
```

```
df = df.dropna(subset=["course_id"]).copy()
```

```
df["course_id"] = df["course_id"].astype(int).astype(str)
```

```
# Group by user into lists of course IDs.
```

```
return df.groupby("user_id")["course_id"].apply(list).tolist()
```

```
def _build_rules(
```

```
    min_support: float = 0.01,
```

```
    min_confidence: float = 0.10,
```

```
    *,
```

```
    csv_path: str = os.path.join(DATA_DIR, "Synthetic_Interactions.csv"),
```

```
) -> pd.DataFrame:
```

```
"""
```

```
Build association rules using Apriori:
```

```
1) One-hot encode transactions.
```

```
2) Mine frequent itemsets with min_support.
```

```
3) Generate rules with min_confidence.
```

```
4) Sort by lift/confidence/support for stability.
```

```
"""
```

```
transactions = _load_transactions(csv_path)
```

```
if not transactions:
```

```
    return pd.DataFrame()
```

```
# One-hot encode transactions (rows = users, cols = course IDs).
```

```
te = TransactionEncoder()
```

```
te_ary = te.fit(transactions).transform(transactions)
```

```
onehot = pd.DataFrame(te_ary, columns=te.columns_)
```

```
# Find frequent itemsets (e.g., {A,B} appears  $\geq$  min_support of users).
```

```
freq = apriori(onehot, min_support=min_support, use_colnames=True)
```

```
if freq.empty:
```

```
    return pd.DataFrame()
```

```

# Turn frequent itemsets into rules (A ⇒ B) with confidence ≥ threshold.
rules = association_rules(freq, metric="confidence", min_threshold=min_confidence)
if rules.empty:
    return rules

# Sort for consistent display.
rules = rules.sort_values(["lift", "confidence", "support"], ascending=False).reset_index(drop=True)
return rules

def load_rules(
    min_support: float = 0.01,
    min_confidence: float = 0.10,
    *,
    use_cache: bool = True,
) -> pd.DataFrame:
    """
    Load rules from cache if available; otherwise build and cache them.
    Caching saves time during demos and repeated requests.
    """
    cache_file = os.path.join(CACHE_DIR,
f'assoc_rules_ms{min_support}_mc{min_confidence}.pkl')

    if use_cache and os.path.exists(cache_file):
        try:
            return joblib.load(cache_file)
        except Exception:
            # If cache is corrupt or incompatible, fall back to rebuild.
            pass

    rules = _build_rules(min_support=min_support, min_confidence=min_confidence)
    if use_cache and not rules.empty:
        joblib.dump(rules, cache_file)
    return rules

# -----
# Catalogue helpers (id → title)
# -----
def __read_catalog(catalog_path: str) -> pd.DataFrame:
    """
    Read a course catalog and normalize columns to:
    - course_id (str)
    - title (str)

```

This is schema-tolerant: it accepts several common header names.

"""

```
if not os.path.exists(catalog_path):
    return pd.DataFrame(columns=["course_id", "title"])
```

```
df = pd.read_csv(catalog_path)
```

```
# Normalize course_id column name.
```

```
id_col = None
```

```
for cand in ["course_id", "id", "courseID", "courseId"]:
```

```
    if cand in df.columns:
```

```
        id_col = cand
```

```
        break
```

```
if id_col is None:
```

```
    return pd.DataFrame(columns=["course_id", "title"])
```

```
# Normalize title/name column name.
```

```
title_col = None
```

```
for cand in ["title", "course_title", "name", "courseName"]:
```

```
    if cand in df.columns:
```

```
        title_col = cand
```

```
        break
```

```
if title_col is None:
```

```
    # If missing, fall back to using the ID as a string title (never crash).
```

```
    title_col = id_col
```

```
# Keep only the two columns and coerce types.
```

```
df = df[[id_col, title_col]].rename(columns={id_col: "course_id", title_col: "title"}).copy()
```

```
df["course_id"] = pd.to_numeric(df["course_id"], errors="coerce")
```

```
df = df.dropna(subset=["course_id"])
```

```
df["course_id"] = df["course_id"].astype(int).astype(str)
```

```
df["title"] = df["title"].fillna("").astype(str)
```

```
return df
```

```
def _load_titles_map() -> Dict[str, str]:
```

```
    """
```

Merge titles from multiple possible catalogs so we rarely show '(title not found)'.

Later files in the list can overwrite earlier ones (priority order).

```
    """
```

```
paths = [
```

```
    os.path.join(DATA_DIR, "udemy_courses.csv"),    # title/name/course_title
```

```
    os.path.join(DATA_DIR, "udemy_course_data.csv"), # course_title
```

```
    os.path.join(DATA_DIR, "courses.csv"),          # title
```

```

    ]
    id2title: Dict[str, str] = {}
    for p in paths:
        df = _read_catalog(p)
        if not df.empty:
            id2title.update(dict(zip(df["course_id"], df["title"])))
    return id2title

def _course_key(x: Union[int, str]) -> str:
    """Normalize a course ID to a clean string (e.g., 123 → '123')."""
    try:
        return str(int(x))
    except Exception:
        return str(x)

def _as_str_set(s: Iterable) -> set[str]:
    """Convert an iterable of items to a set of normalized string course IDs."""
    try:
        return set(map(_course_key, s))
    except Exception:
        return set()

# -----
# Public API: explain_rules_for_course
# -----
def explain_rules_for_course(
    course_id: int | str,
    top_k: int = 10,
    *,
    min_support: float = 0.01,
    min_confidence: float = 0.10,
    # kept for compatibility; we now merge multiple catalogs automatically
    course_catalog_csv: str | None = None,
) -> List[Dict]:
    """
    Given a selected course ID, return up to top_k *related* courses with:
    - human-friendly fields (title, estimated likelihood, counts, baseline)
    - technical metrics (support, confidence, lift)

    This powers the /rules page and keeps the output explainable for users.
    """
    # Normalize input ID once.
    course_key = _course_key(course_id)

```

```
# Load (or build) association rules.
rules = load_rules(min_support=min_support, min_confidence=min_confidence, use_cache=True)
if rules.empty or "antecedents" not in rules or "consequents" not in rules:
    return []

# Keep rules where our course appears on either side, then reset index
mask = rules["antecedents"].apply(lambda s: course_key in _as_str_set(s)) | \
    rules["consequents"].apply(lambda s: course_key in _as_str_set(s))
if not mask.any():
    return []

selected = rules.loc[mask].reset_index(drop=True).copy()

# Compute sets on the filtered subset (ensures lengths align)
ants_sel = selected["antecedents"].apply(_as_str_set)
cons_sel = selected["consequents"].apply(_as_str_set)

# Pick one recommended target course per rule:
# Prefer something from the consequents; if that's the same as the selected course,
# fall back to an antecedent that is different.
def _pick_target(a_set: set[str], c_set: set[str]) -> str | None:
    for cid in c_set:
        if cid != course_key:
            return cid
    for cid in a_set:
        if cid != course_key:
            return cid
    return None

selected["_target"] = [ _pick_target(a, c) for a, c in zip(ants_sel, cons_sel) ]
selected = selected.dropna(subset=["_target"])
if selected.empty:
    return []

# Aggregate duplicate targets coming from multiple rules; keep max of each metric.
agg = (
    selected.groupby("_target", as_index=False)
    .agg({"support": "max", "confidence": "max", "lift": "max"})
    .sort_values(["lift", "confidence", "support"], ascending=False)
    .head(max(1, min(int(top_k or 10), 50)))
    .reset_index(drop=True)
)
```

```

# --- Friendly statistics from the raw transactions ---
tx = _load_transactions() # list[list[str]]
total_users = len(tx)
users_selected = sum(course_key in t for t in tx) # how many users took the selected course
overall_counts = Counter(c for t in tx for c in set(t)) # how common each course is overall

# Titles map: merge of multiple catalogs (id → title)
id2title = _load_titles_map()

# Build the final rows, mixing friendly and technical fields.
rows: List[Dict] = []
for _, r in agg.iterrows():
    cid = _course_key(r["_target"])
    users_both = sum((course_key in t) and (cid in t) for t in tx) # users who took both courses
    baseline = (overall_counts.get(cid, 0) / total_users) if total_users else 0.0
    likelihood = (users_both / users_selected) if users_selected else 0.0

    rows.append({
        "consequent": int(cid) if cid.isdigit() else cid,
        "title": id2title.get(cid, "(title not found)"),
        # Friendly, explainable fields for the UI:
        "likelihood_pct": round(likelihood * 100, 1), # "Estimated likelihood"
        "n_both": int(users_both), # "2/10 users took both", etc.
        "n_selected": int(users_selected),
        "baseline_pct": round(baseline * 100, 2), # "vs typical"
        # Technical metrics from Apriori (for transparency):
        "support": round(float(r.get("support", 0.0)), 3),
        "confidence": round(float(r.get("confidence", 0.0)), 3),
        "lift": round(float(r.get("lift", 0.0)), 3),
    })
return rows

```

chat_assistant.py

```

# -----
# Minimal keyword→course helper for the floating chat widget.
# - Loads a course catalogue CSV (several possible paths)
# - Builds a TF-IDF index over title + subject
# - Given a free-text query, returns the top-k matching courses
# - Handles greetings and a tiny "user 123" deep-link intent
# -----

```

```
import os
import re
import pandas as pd
from typing import List, Dict, Any, Optional
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import linear_kernel

class ChatAssistant:
    """Free-text → top-k matching courses by TF-IDF on title+subject."""

    def __init__(self, data_paths: Optional[List[str]] = None):
        self.data_paths = data_paths or [
            os.path.join("webpage", "data", "udemy_courses.csv"),
            os.path.join("webpage", "data", "courses.csv"),
            os.path.join("data", "udemy_courses.csv"),
            os.path.join("data", "courses.csv"),
            os.path.join("data", "udemy_course_data.csv"), # accepts course_title -> title
        ]
        self.enabled = True
        self.df = self._load_courses()

        if self.df.empty:
            self.enabled = False
            self._why_disabled = "No course catalogue CSV found. Expected: course_id, title/course_title, subject."
            self.vectorizer = None
            self.tfidf = None
            return

        # clean text fields
        self.df["title"] = self.df["title"].fillna("").astype(str)
        if "subject" not in self.df.columns:
            self.df["subject"] = ""
        self.df["subject"] = self.df["subject"].fillna("").astype(str)
        self.df["__text__"] = (self.df["title"] + " " + self.df["subject"]).str.strip()

        # drop empty rows
        self.df = self.df[self.df["__text__"].str.len() > 0].reset_index(drop=True)
        if self.df.empty:
            self.enabled = False
            self._why_disabled = "Course catalogue has no non-empty titles/subjects."
            self.vectorizer = None
            self.tfidf = None
```

```
return
```

```
# robust vectorizer (no stopwords so short titles survive)
self.vectorizer = TfidfVectorizer(min_df=1, ngram_range=(1, 2), stop_words=None)
self.tfidf = self.vectorizer.fit_transform(self.df["__text__"].tolist())
```

```
def _load_courses(self) -> pd.DataFrame:
```

```
"""
Try several known paths for a catalogue CSV and normalize schema:
- Accepts `course_id` or common aliases (id, courseID, courseId)
- Accepts `title` or `course_title` (mapped to title)
- Ensures `subject` exists (empty string if missing)
Returns only: course_id, title, subject
"""
```

```
for p in self.data_paths:
```

```
    if os.path.exists(p):
```

```
        try:
```

```
            df = pd.read_csv(p)
```

```
            # normalize id -> course_id
```

```
            if "course_id" not in df.columns:
```

```
                for alt in ["id", "courseID", "courseId"]:
```

```
                    if alt in df.columns:
```

```
                        df = df.rename(columns={alt: "course_id"})
```

```
                        break
```

```
            # normalize title (accept course_title)
```

```
            if "title" not in df.columns:
```

```
                if "course_title" in df.columns:
```

```
                    df = df.rename(columns={"course_title": "title"})
```

```
                else:
```

```
                    continue # no usable title field
```

```
            if "subject" not in df.columns:
```

```
                df["subject"] = ""
```

```
            # minimal view
```

```
            keep = df[["course_id", "title", "subject"]].dropna(subset=["title"])
```

```
            # enforce str IDs (avoid 1.0 etc.)
```

```
            keep["course_id"] = keep["course_id"].astype(str)
```

```
            return keep
```

```
except Exception:
```

```
    continue
```

```

    return pd.DataFrame()

def help_text(self) -> str:
    return (
        "Tell me what you want to learn.\n"
        "Try: python for data analysis · statistics basics · machine learning for beginners\n"
        "Tip: type 'user 123' to open recommendations for that user."
    )

def _is_greeting(self, msg: str) -> bool:
    m = msg.lower().strip()
    greetings = ("hi", "hello", "hey", "help", "?")
    return any(m == g or m.startswith(g + " ") or m.endswith(" " + g) for g in greetings)

def reply(self, message: str, top_k: int = 3) -> Dict[str, Any]:
    msg = (message or "").strip()
    if not msg:
        return {"reply": "Tell me what you want to learn and I will suggest courses."}

    if not self.enabled or self.tfidf is None or self.tfidf.shape[0] == 0:
        why = getattr(self, "_why_disabled", "Catalogue not loaded.")
        return {"reply": f"I am not ready yet: {why}"}

    if self._is_greeting(msg):
        return {"reply": self.help_text()}

    m = re.match(r"user\s+(\w+)", msg.lower())
    if m:
        user_id = m.group(1)
        return {"reply": f"Opening recommendations for user {user_id}.", "link":
f"/recommend?user_id={user_id}"}

    q = self.vectorizer.transform([msg])
    sims = linear_kernel(q, self.tfidf.ravel())

    if sims.max(initial=0) <= 0:
        return {"reply": "I didn't find a close match. Try different keywords or add
'beginner/advanced'."}

    k = max(1, min(int(top_k or 3), 10))
    top_idx = sims.argsort()[::-1][:k]

    items = []

```

```

for i in top_idx:
    row = self.df.iloc[i]
    items.append({
        "course_id": str(row["course_id"]),
        "title": str(row["title"]),
        "subject": str(row.get("subject", "")),
        "link": f"/recommend?course_id={row['course_id']}"
    })

reply = f"Top {len(items)} matches. Closest: \"{items[0]['title']}\"."
return {"reply": reply, "items": items}

```

classifier.py

```

import joblib

# Load the trained classification model and vectorizer
model = joblib.load("models/classifier_model.pkl") # Load the saved classifier model
vectorizer = joblib.load("models/vectorizer.pkl") # Load the saved text vectorizer

def classify_text(text):
    # Transform input text into vectorized form
    X = vectorizer.transform([text])

    # Predict the class (e.g., engaged or not)
    prediction = model.predict(X)[0]

    # Extract feature names and the indices of non-zero features in input
    word_list = vectorizer.get_feature_names_out()
    word_indices = X.nonzero()[1]

    # Get the top contributing words from the input
    top_words = [word_list[i] for i in word_indices]

    # Create an explanation based on the top words
    explanation = f"Prediction: {prediction.upper()} — based on words: {', '.join(top_words)}"
    return prediction, explanation

```

debug_valid_rules.py

```

import pandas as pd

```

```

from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

df = pd.read_csv("data/Synthetic_Interactions.csv")
transactions = df.groupby("user_id")["course_id"].apply(list).tolist()

te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
df_tf = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df_tf, min_support=0.01, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.1)

# Show course_ids that appear as antecedents
antecedent_course_ids = sorted(set(item for a in rules['antecedents'] for item in a))
print("\n 📌 Valid course IDs to test in /rules:")
print(antecedent_course_ids[:20])

```

evaluate.py

```

# -----
# Offline evaluation routines for the educational recommender system.
# Produces:
# - quick headline metrics (RMSE, Precision@3)
# - k-fold CV RMSE for a noisy baseline
# - SVD RMSE/MAE via Surprise
# - Top-N offline evaluation for Popularity, SVD, Content, Hybrid
# - Precision@K and Recall@K bar chart
# - Precision-Recall and ROC curves for SVD on held-out pairs
# Artifacts are saved in static/ for the Flask app to display.
# -----

```

```

from __future__ import annotations

```

```

import os
import numpy as np
import pandas as pd

```

```

import matplotlib
matplotlib.use("Agg") # headless backend for servers and CI
import matplotlib.pyplot as plt

```

```

from typing import Dict, List, Tuple

```

```

from sklearn.metrics import mean_squared_error, precision_recall_curve, roc_curve, auc
from sklearn.model_selection import KFold
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

```

```

from surprise import Dataset, Reader, SVD
from svd_recommender import train_svd_model # SVD CV + fit-on-full helper

```

```

# LDA optional; keeps routes working even if lda_topics.py is absent
try:

```

```

    from lda_topics import train_lda_model, plot_topic_summary
except Exception: # pragma: no cover
    def train_lda_model():
        return None
    def plot_topic_summary(_):
        return None

```

```

# Optional SBERT for stronger content similarity
# If sentence-transformers is not installed, we fall back to TF-IDF

```

```

_USE_SBERT = True
_SBERT_OK = False

```

```

try:
    if _USE_SBERT:
        os.environ["USE_TF"] = "0" # sentence-transformers can run without TF
        from sentence_transformers import SentenceTransformer
        from numpy.linalg import norm
        _SBERT_OK = True
except Exception:
    _SBERT_OK = False

```

```

# ----- paths -----
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "data")
STATIC_DIR = os.path.join(BASE_DIR, "static")
os.makedirs(STATIC_DIR, exist_ok=True)

```

```

# ----- data loaders and utilities -----

```

```

def _load_interactions() -> pd.DataFrame:
    """
    Load user-course interactions and normalize types.

    Returns columns: user_id(str), course_id(str), rating(float clipped to [1,5])
    """

```

```

p = os.path.join(DATA_DIR, "Synthetic_Interactions.csv")
df = pd.read_csv(p).dropna(subset=["rating"])
df["user_id"] = df["user_id"].astype(str).str.strip()
df["course_id"] = pd.to_numeric(df["course_id"], errors="coerce")
df = df.dropna(subset=["course_id"]).copy()
df["course_id"] = df["course_id"].astype(int).astype(str)
df["rating"] = pd.to_numeric(df["rating"], errors="coerce").fillna(0).clip(1, 5)
return df

```

```
def _load_courses() -> pd.DataFrame:
```

```
    """
```

```
    Load course catalogue with minimal fields used for content models.
```

```
    Returns columns:
```

- course_id(str)
- course_title(str)
- subject(str)
- text(str) which is title + subject for TF-IDF or SBERT

```
    """
```

```

p = os.path.join(DATA_DIR, "udemy_course_data.csv")
df = pd.read_csv(p, usecols=["course_id", "course_title", "subject"])
df["course_id"] = pd.to_numeric(df["course_id"], errors="coerce")
df = df.dropna(subset=["course_id"]).copy()
df["course_id"] = df["course_id"].astype(int).astype(str)
df["course_title"] = df["course_title"].fillna("").astype(str)
df["subject"] = df["subject"].fillna("").astype(str)
df["text"] = (df["course_title"] + " " + df["subject"]).str.strip()
return df

```

```
def _user_split(df, test_frac=0.2, min_per_user=3, threshold=3.5, seed=42):
```

```
    """
```

```
    Per-user train/test split for Top-N evaluation.
```

```
    Goals
```

- Hold out at least one positive (rating >= threshold) per user if possible
- Add up to 3 negatives to test for balance
- If no positives exist for the user, do a small random holdout

```
    Returns
```

```
    train_df, test_df with the same columns as input df
```

```
    """
```

```

rng = np.random.default_rng(seed)
train_rows, test_rows = [], []

```

```

for uid, g in df.groupby("user_id"):
    # deterministic shuffle per user
    g = g.sample(frac=1.0, random_state=seed)
    if len(g) < min_per_user:
        train_rows.append(g)
        continue

    pos_idx = g.index[g["rating"] >= threshold].tolist()
    if pos_idx:
        # exactly one positive into test
        test_pick = rng.choice(pos_idx, size=1, replace=False)
        # add up to 3 negatives
        remain = g.drop(test_pick)
        neg_pool = remain.index[remain["rating"] < threshold].tolist()
        k_neg = min(3, len(neg_pool))
        neg_pick = rng.choice(neg_pool, size=k_neg, replace=False) if k_neg > 0 else []
        test_rows.append(pd.concat([g.loc[test_pick], g.loc[neg_pick]]))
        train_rows.append(g.drop(index=list(test_pick) + list(neg_pick)))
    else:
        # no positives: small random holdout
        k = max(1, int(len(g) * test_frac))
        held = rng.choice(g.index, size=k, replace=False)
        test_rows.append(g.loc[held])
        train_rows.append(g.drop(held))

train = pd.concat(train_rows, ignore_index=True)
test = pd.concat(test_rows, ignore_index=True) if test_rows else
pd.DataFrame(columns=df.columns)
return train, test

def _popularity_scores(train: pd.DataFrame) -> pd.Series:
    """
    Score items by mean rating with a tiny count tiebreaker for stable ordering.
    Returns a descending series indexed by course_id.
    """
    means = train.groupby("course_id")["rating"].mean()
    counts = train.groupby("course_id")["rating"].count()
    return (means + 1e-6 * counts).sort_values(ascending=False)

def _precision_recall_at_k(test: pd.DataFrame, recs_by_user: Dict[str, List[str]],
                           k=5, threshold=3.5) -> Tuple[float, float]:
    """
    Compute mean Precision@K and Recall@K over users with at least one positive in test.

```

```

"""
if test.empty:
    return 0.0, 0.0
rel = test[test["rating"] >= threshold].groupby("user_id")["course_id"].apply(set)
p_vals, r_vals = [], []
for u, recs in recs_by_user.items():
    if u not in rel:
        continue
    topk = recs[:k] if recs else []
    if not topk:
        continue
    hits = len(set(topk) & rel[u])
    p_vals.append(hits / max(1, len(topk)))
    r_vals.append(hits / max(1, len(rel[u])))
if not p_vals:
    return 0.0, 0.0
return float(np.mean(p_vals)), float(np.mean(r_vals))

def _users_with_any_hit(test: pd.DataFrame, recs_by_user: Dict[str, List[str]],
                        k=5, threshold=3.5) -> Tuple[int, int]:
    """
    Diagnostic: count how many users have at least one hit in top-K compared to
    how many users have any positive in test.
    """
    if test.empty:
        return 0, 0
    rel = test[test["rating"] >= threshold].groupby("user_id")["course_id"].apply(set)
    users = 0
    hits = 0
    for u, pos in rel.items():
        users += 1
        hit = len(set(recs_by_user.get(u, [])[:k]) & pos) > 0
        hits += int(hit)
    return hits, users

# ----- headline quick metrics (kept lightweight) -----
def run_evaluation():
    """
    Produce quick headline numbers for the dashboard:
    - RMSE from a noisy baseline predictor
    - Precision@3 from the same baseline
    """
    df = _load_interactions()

```

```

# add Gaussian noise to rating as a silly baseline predictor
pred = df["rating"] + np.random.default_rng(42).normal(0, 0.5, size=len(df))
pred = np.clip(pred, 1.0, 5.0)
rmse = float(np.sqrt(mean_squared_error(df["rating"], pred)))

# turn ratings into top vs not-top at threshold 4 to approximate relevance
is_top = (df["rating"] >= 4).astype(int)
pred_top = (pred >= 4).astype(int)
precision_at3 = float((is_top & pred_top).sum() / max(1, pred_top.sum()))
return {"rmse": round(rmse, 3), "precision": round(precision_at3, 3)}

def run_cross_validation(k=5):
    """
    K-fold CV of the same noisy baseline to provide a reference RMSE distribution.
    """
    df = _load_interactions()
    kf = KFold(n_splits=k, shuffle=True, random_state=42)
    rmse = []
    for _, te_idx in kf.split(df):
        test = df.iloc[te_idx].copy()
        pred = test["rating"] + np.random.default_rng(0).normal(0, 0.5, size=len(test))
        pred = np.clip(pred, 1.0, 5.0)
        rmse.append(np.sqrt(mean_squared_error(test["rating"], pred)))
    return {"cv_rmse_mean": round(float(np.mean(rmse)), 3),
            "cv_rmse_std": round(float(np.std(rmse)), 3)}

def run_svd_evaluation():
    """
    Delegate to train_svd_model helper which performs CV and fit-on-full.
    Returns averaged RMSE and MAE for SVD.
    """
    _, avg_rmse, avg_mae = train_svd_model(os.path.join(DATA_DIR, "Synthetic_Interactions.csv"))
    return {"svd_rmse": round(float(avg_rmse), 3),
            "svd_mae": round(float(avg_mae), 3)}

# ----- Top-N offline eval with 4 models: Popularity, SVD, Content, Hybrid -----
def run_precision_bars_and_curves(k=5, threshold=3.5, seed=42):
    """
    Full Top-N evaluation:
    1) Split per user while keeping at least one positive in test if possible
    2) Build candidate pools per user and four recommenders:
        - Popularity
        - SVD (Surprise)

```

```

- Content (SBERT or TF-IDF)
- Hybrid blend (SVD + Content + small Popularity prior)
3) Compute Precision@K and Recall@K
4) Save a precision bar chart, PR and ROC curves for SVD
"""
df = _load_interactions()
courses = _load_courses()
all_items = courses["course_id"].tolist()

# subject lookup supports candidate pools by subject
subj_map = dict(zip(courses["course_id"], courses["subject"]))

# Prepare content models
tfidf = TfidfVectorizer(stop_words="english")
tfidf_matrix = tfidf.fit_transform(courses["text"])
idx_by_cid = {cid: i for i, cid in enumerate(courses["course_id"].tolist())}

# SBERT embeddings if available
if _SBERT_OK:
    sbert = SentenceTransformer("all-MiniLM-L6-v2")
    course_texts = courses["text"].tolist()
    emb = sbert.encode(course_texts, show_progress_bar=False, normalize_embeddings=True)
    cid_to_i = {cid: i for i, cid in enumerate(courses["course_id"].tolist())}

# Train/test split per user
train, test = _user_split(df, test_frac=0.2, min_per_user=3, threshold=threshold, seed=seed)
users = sorted(test["user_id"].unique()) if not test.empty else []

# Ensure the held-out test items are in each user's candidate pool for proper evaluation
test_items_by_user = {u: set(test.loc[test["user_id"] == u, "course_id"]) for u in users}

# Popularity ranking over the train split
pop_series = _popularity_scores(train)
pop_ranked = list(pop_series.index)

# Candidate pool sizes
MAX_POOL = 250      # final per-user pool cap
POPULAR_TOP = 200   # popularity seed size
PROFILE_TOP = 300   # profile similarity addition

def _candidate_pool_for_user(u: str, max_pool=MAX_POOL) -> List[str]:
    """
    Build a reasonable per-user candidate pool:

```

```

1) same-subject courses
2) profile-similar items by SBERT or TF-IDF user vector
3) popularity seed
4) always include user's held-out test items
"""
# subjects from user's liked items if available, else any subject they interacted with
liked_ids = train[(train["user_id"] == u) & (train["rating"] >= threshold)][ "course_id"].tolist()
liked_subj = {subj_map[c] for c in liked_ids if c in subj_map}
if not liked_subj:
    any_ids = train[train["user_id"] == u][ "course_id"].tolist()
    liked_subj = {subj_map[c] for c in any_ids if c in subj_map}
subject_pool = (
    courses.loc[courses["subject"].isin(liked_subj), "course_id"].tolist()
    if liked_subj else all_items
)

# profile similarity vector
profile_top = []
if _SBERT_OK:
    liked_idx = [cid_to_i[c] for c in liked_ids if 'cid_to_i' in locals() and c in cid_to_i]
    if liked_idx:
        user_vec = emb[liked_idx].mean(axis=0)
        user_vec = user_vec / max(1e-9, norm(user_vec))
        sims = emb @ user_vec
        ranked_all = [cid for _, cid in sorted(zip(sims, courses["course_id"].tolist()), reverse=True)]
        profile_top = ranked_all[:PROFILE_TOP]
    else:
        liked_idx = [idx_by_cid[c] for c in liked_ids if c in idx_by_cid]
        if liked_idx:
            user_vec = tfidf_matrix[liked_idx].mean(axis=0)
            user_vec = np.asarray(user_vec).reshape(1, -1) # avoid np.matrix
            sims = cosine_similarity(user_vec, tfidf_matrix).ravel()
            ranked_all = [cid for _, cid in sorted(zip(sims, courses["course_id"].tolist()), reverse=True)]
            profile_top = ranked_all[:PROFILE_TOP]

# popularity seed
popularity_seed = pop_ranked[:POPULAR_TOP]

# merge and dedupe while preserving order
pool = list(dict.fromkeys(popularity_seed + profile_top + subject_pool))

# guarantee presence of held-out items for fair evaluation
pool = list(dict.fromkeys(pool + list(test_items_by_user.get(u, set()))))

```

```
    return pool[:max_pool]

# Build recommendations for each model
seen = train.groupby("user_id")["course_id"].apply(set)

# Popularity recommender
pop_recs: Dict[str, List[str]] = {}
for u in users:
    pool = _candidate_pool_for_user(u)
    hide = seen.get(u, set())
    ranked = [cid for cid in pool if cid not in hide]
    pop_recs[u] = ranked[:100] if ranked else [cid for cid in pop_ranked if cid not in hide][:100]

# SVD recommender fit on the train split only
reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(train[["user_id", "course_id", "rating"]], reader)
svd = SVD(random_state=seed)
svd.fit(data.build_full_trainset())

recs_svd: Dict[str, List[str]] = {}
for u in users:
    pool = _candidate_pool_for_user(u)
    hide = seen.get(u, set())
    cand = [cid for cid in pool if cid not in hide]
    preds = [(cid, svd.predict(u, cid).est) for cid in cand]
    preds.sort(key=lambda x: x[1], reverse=True)
    recs_svd[u] = [cid for cid, _ in preds[:100]] if preds else [cid for cid in pop_ranked if cid not in
hide][:100]

# Content recommender
# SBERT cosine similarity if available, fallback to TF-IDF cosine similarity
recs_content: Dict[str, List[str]] = {}
for u in users:
    pool = _candidate_pool_for_user(u)
    hide = seen.get(u, set())

    if _SBERT_OK:
        cid_to_i = {cid: i for i, cid in enumerate(courses["course_id"].tolist())}
        pool_idx = [cid_to_i[c] for c in pool if c in cid_to_i]
        liked_ids = train[(train["user_id"] == u) & (train["rating"] >= threshold)]["course_id"].tolist()
        liked_idx = [cid_to_i[c] for c in liked_ids if c in cid_to_i]
        if liked_idx and pool_idx:
```

```

        user_vec = emb[liked_idx].mean(axis=0)
        user_vec = user_vec / max(1e-9, norm(user_vec))
        pool_vecs = emb[pool_idx]
        sims = (pool_vecs @ user_vec)
        pool_ids = [courses["course_id"].iloc[i] for i in pool_idx]
        ranked = [x for _, x in sorted(zip(sims, pool_ids), reverse=True)]
    else:
        ranked = pool
else:
    pool_idx = [idx_by_cid[c] for c in pool if c in idx_by_cid]
    liked_ids = train[(train["user_id"] == u) & (train["rating"] >= threshold)]["course_id"].tolist()
    liked_idx = [idx_by_cid[c] for c in liked_ids if c in idx_by_cid]
    if liked_idx and pool_idx:
        user_vec = tfidf_matrix[liked_idx].mean(axis=0)
        user_vec = np.asarray(user_vec).reshape(1, -1) # avoid np.matrix
        sims = cosine_similarity(user_vec, tfidf_matrix[pool_idx]).ravel()
        pool_ids = [courses["course_id"].iloc[i] for i in pool_idx]
        ranked = [x for _, x in sorted(zip(sims, pool_ids), reverse=True)]
    else:
        ranked = pool

ranked = [cid for cid in ranked if cid not in hide]
recs_content[u] = ranked[:100] if ranked else [cid for cid in pop_ranked if cid not in hide][:100]

# Hybrid recommender: normalized blend of SVD + Content + small Popularity prior
ALPHA, BETA, GAMMA = 0.5, 0.45, 0.05 # weights

def _mm(d: Dict[str, float]) -> Dict[str, float]:
    """
    Min-max normalize a dict of scores to [0,1] per user.
    If all values equal, return zeros to avoid divide-by-zero.
    """
    if not d:
        return {}
    v = np.array(list(d.values()), dtype=float)
    lo, hi = float(np.min(v)), float(np.max(v))
    if hi <= lo:
        return {k: 0.0 for k in d}
    return {k: (float(x) - lo) / (hi - lo) for k, x in d.items()}

recs_hybrid: Dict[str, List[str]] = {}
pop_pos = {cid: i for i, cid in enumerate(pop_ranked)} # smaller index = more popular
for u in users:

```

```

# candidate set is the union of other recommenders for this user
cand = set(pop_recs.get(u, [])) | set(recs_svd.get(u, [])) | set(recs_content.get(u, []))
if not cand:
    recs_hybrid[u] = pop_recs.get(u, [])[0:100]
    continue

# raw scores
svd_scores = {cid: svd.predict(u, cid).est for cid in cand}
cont_rank = recs_content.get(u, [])
# convert content rank to a score in [0,1] by position
cont_scores = {cid: (len(cont_rank) - i) / max(1, len(cont_rank)) for i, cid in
enumerate(cont_rank)}
# small popularity prior by inverse rank
pop_scores = {cid: (len(pop_ranked) - pop_pos[cid]) / len(pop_ranked) for cid in cand if cid in
pop_pos}

# normalize each component then blend
a, b, c = _mm(svd_scores), _mm(cont_scores), _mm(pop_scores)
final = {cid: ALPHA * a.get(cid, 0.0) + BETA * b.get(cid, 0.0) + GAMMA * c.get(cid, 0.0) for
cid in cand}
ranked = [cid for cid, _ in sorted(final.items(), key=lambda x: x[1], reverse=True)]

# hide items already seen in train
hide = seen.get(u, set())
ranked = [cid for cid in ranked if cid not in hide]
recs_hybrid[u] = ranked[0:100] if ranked else pop_recs.get(u, [])[0:100]

# --- Metrics @k across all models ---
p5_pop, r5_pop = _precision_recall_at_k(test, pop_recs, k=k, threshold=threshold)
p5_svd, r5_svd = _precision_recall_at_k(test, recs_svd, k=k, threshold=threshold)
p5_con, r5_con = _precision_recall_at_k(test, recs_content, k=k, threshold=threshold)
p5_hyb, r5_hyb = _precision_recall_at_k(test, recs_hybrid, k=k, threshold=threshold)

# Diagnostics to console: how many users get at least one hit in top-K
cov_pop = _users_with_any_hit(test, pop_recs, k=k, threshold=threshold)
cov_svd = _users_with_any_hit(test, recs_svd, k=k, threshold=threshold)
cov_con = _users_with_any_hit(test, recs_content, k=k, threshold=threshold)
cov_hyb = _users_with_any_hit(test, recs_hybrid, k=k, threshold=threshold)
print(f"[coverage] users with ≥1 hit@{k} (pop, svd, content, hybrid): "
      f"{cov_pop[0]}/{cov_pop[1]}, {cov_svd[0]}/{cov_svd[1]}, "
      f"{cov_con[0]}/{cov_con[1]}, {cov_hyb[0]}/{cov_hyb[1]}")

# --- Bar chart for Precision@K ---

```

```
plt.figure(figsize=(6.5, 4))
labels = ["Popularity", "SVD", "Content", "Hybrid"]
vals = [p5_pop, p5_svd, p5_con, p5_hyb]
plt.bar(labels, vals)
plt.ylabel(f'Precision@{k}')
plt.title("Top-N Precision (offline)")
plt.tight_layout()
plt.savefig(os.path.join(STATIC_DIR, "precision_bar.png"))
plt.close()

# --- PR and ROC curves for SVD on the held-out test pairs ---
if not test.empty:
    # Binary relevance by rating threshold
    y_true = (test["rating"].values >= threshold).astype(int)
    # SVD scores on test pairs
    y_score = np.array([svd.predict(r.user_id, r.course_id).est for r in test.itertuples(index=False)])
    # scale to [0,1] for nicer curves
    if y_score.size > 0:
        y_score = (y_score - y_score.min()) / max(1e-8, (y_score.max() - y_score.min()))
    pr, rc, _ = precision_recall_curve(y_true, y_score)
    fpr, tpr, _ = roc_curve(y_true, y_score)
    auc_pr = auc(rc, pr)
    auc_roc = auc(fpr, tpr)

    # PR curve
    plt.figure(figsize=(5, 4))
    plt.plot(rc, pr)
    plt.xlabel("Recall"); plt.ylabel("Precision")
    plt.title(f'Precision-Recall (AUC={auc_pr:.3f})')
    plt.tight_layout()
    plt.savefig(os.path.join(STATIC_DIR, "pr_curve.png"))
    plt.close()

    # ROC curve
    plt.figure(figsize=(5, 4))
    plt.plot(fpr, tpr)
    plt.plot([0, 1], [0, 1], "--") # random baseline
    plt.xlabel("FPR"); plt.ylabel("TPR")
    plt.title(f'ROC (AUC={auc_roc:.3f})')
    plt.tight_layout()
    plt.savefig(os.path.join(STATIC_DIR, "roc_curve.png"))
    plt.close()
else:
```

```

    auc_pr = 0.0
    auc_roc = 0.0

# Console diagnostics
users_with_pos = int(test[test["rating"] >= threshold]["user_id"].nunique())
print(f"[evaluate] users in test={len(users)}; users with ≥{threshold} positives={users_with_pos}")

return {
    "p_at5_popularity": round(float(p5_pop), 3),
    "r_at5_popularity": round(float(r5_pop), 3),
    "p_at5_svd": round(float(p5_svd), 3),
    "r_at5_svd": round(float(r5_svd), 3),
    "p_at5_content": round(float(p5_con), 3),
    "r_at5_content": round(float(r5_con), 3),
    "p_at5_hybrid": round(float(p5_hyb), 3),
    "r_at5_hybrid": round(float(r5_hyb), 3),
    "bar_img": "precision_bar.png",
    "auc_pr": round(float(auc_pr), 3),
    "auc_roc": round(float(auc_roc), 3),
    "pr_img": "pr_curve.png" if os.path.exists(os.path.join(STATIC_DIR, "pr_curve.png")) else "",
    "roc_img": "roc_curve.png" if os.path.exists(os.path.join(STATIC_DIR, "roc_curve.png")) else ""
}

def generate_topic_insights():
    """
    Optional LDA topic training and a summary plot saved to static/lda_topics.png.
    Silently no-ops if LDA is unavailable.
    """
    try:
        topics = train_lda_model()
        plot_topic_summary(topics) # saves static/lda_topics.png
    except Exception:
        pass
    return {"ok": True}

def export_metrics_csv(metrics, cv, svd, models):
    """
    Export the main scalar metrics to static/evaluation_metrics.csv
    for easy inclusion in the report and dashboard tables.
    """
    rows = [
        {"metric": "RMSE (headline)", "value": metrics["rmse"]},

```

```

    {"metric": "Precision@3 (headline)", "value": metrics["precision"]},
    {"metric": "CV RMSE (mean)", "value": cv["cv_rmse_mean"]},
    {"metric": "CV RMSE (std)", "value": cv["cv_rmse_std"]},
    {"metric": "SVD RMSE (CV)", "value": svd["svd_rmse"]},
    {"metric": "SVD MAE (CV)", "value": svd["svd_mae"]},
    {"metric": "P@5 Popularity", "value": models["p_at5_popularity"]},
    {"metric": "R@5 Popularity", "value": models["r_at5_popularity"]},
    {"metric": "P@5 SVD", "value": models["p_at5_svd"]},
    {"metric": "R@5 SVD", "value": models["r_at5_svd"]},
    {"metric": "P@5 Content", "value": models["p_at5_content"]},
    {"metric": "R@5 Content", "value": models["r_at5_content"]},
    {"metric": "P@5 Hybrid", "value": models["p_at5_hybrid"]},
    {"metric": "R@5 Hybrid", "value": models["r_at5_hybrid"]},
    {"metric": "AUC(PR)", "value": models.get("auc_pr", 0)},
    {"metric": "AUC(ROC)", "value": models.get("auc_roc", 0)},
]
out = os.path.join(STATIC_DIR, "evaluation_metrics.csv")
pd.DataFrame(rows).to_csv(out, index=False)
return {"metrics_csv": out}

```

```
def run_full_evaluation_bundle():
```

```
    """
```

```
    Convenience wrapper used by the Flask route:
```

- runs all evaluations
- generates charts and CSV
- returns paths and numbers for easy templating

```
    """
```

```
    metrics = run_evaluation()
```

```
    cv = run_cross_validation(k=5)
```

```
    svd = run_svd_evaluation()
```

```
    models = run_precision_bars_and_curves(k=5, threshold=3.5)
```

```
    generate_topic_insights()
```

```
    files = export_metrics_csv(metrics, cv, svd, models)
```

```
    return {
```

```
        "metrics": metrics,
```

```
        "cv": cv,
```

```
        "svd": svd,
```

```
        "baselines": models,
```

```
        "curves": {
```

```
            "auc_pr": models["auc_pr"], "auc_roc": models["auc_roc"],
```

```
            "pr_img": models["pr_img"], "roc_img": models["roc_img"]
```

```
        },
```

```
"files": files,
}
```

lda_topics.py

```
import pandas as pd
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer
import matplotlib.pyplot as plt
import os

# Train an LDA topic model using course title and subject data
def train_lda_model(data_path="data/udemy_course_data.csv", n_topics=5):
    # Load course data
    df = pd.read_csv(data_path)

    # Combine course_title and subject into a single text column
    df['text'] = df['course_title'].fillna("") + " " + df['subject'].fillna("")

    # Convert text to document-term matrix
    vectorizer = CountVectorizer(stop_words='english', max_df=0.95, min_df=2)
    dtm = vectorizer.fit_transform(df['text'])

    # Fit LDA model
    lda = LatentDirichletAllocation(n_components=n_topics, random_state=42)
    lda.fit(dtm)

    # Extract top words for each topic
    terms = vectorizer.get_feature_names_out()
    topics = []
    for idx, topic in enumerate(lda.components_):
        top_words = [terms[i] for i in topic.argsort()[-5:]] # Top 5 words
        topics.append((f"Topic {idx+1}", top_words))

    return topics

# Plot a horizontal bar chart summarizing LDA topics and their top words
def plot_topic_summary(topics, output_path="static/lda_topics.png"):
    # Ensure output directory exists
    os.makedirs("static", exist_ok=True)

    # Create the plot
    fig, ax = plt.subplots(figsize=(8, 4))
```

```
labels = [t[0] for t in topics] # Topic labels
word_labels = [" | ".join(t[1]) for t in topics] # Top words per topic
```

```
# Plot bars and annotate with words
ax.barh(labels, [len(t[1]) for t in topics], color="#3498db")
for i, words in enumerate(word_labels):
    ax.text(len(topics[i][1]) + 0.1, i, words, va='center', fontsize=9)
```

```
# Final formatting
ax.set_title("Top 5 Words per LDA Topic")
ax.invert_yaxis()
plt.tight_layout()
plt.savefig(output_path)
plt.close()
```

recommender.py

```
from __future__ import annotations
import os
os.environ["USE_TF"] = "0" # keep sentence-transformers light on Windows
```

```
import math
import numpy as np
import pandas as pd
from typing import Dict, List
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.cluster import KMeans
```

```
# Optional (content embeddings). If missing, we fall back to TF-IDF only.
```

```
try:
    from sentence_transformers import SentenceTransformer
except Exception: # pragma: no cover
    SentenceTransformer = None # type: ignore
```

```
# Optional (rules boost). If missing, rules_boost will be empty.
```

```
try:
    from association_rules import explain_rules_for_course
except Exception: # pragma: no cover
    def explain_rules_for_course(*args, **kwargs):
        return [] # type: ignore
```

```
# SVD (collaborative filtering) – train-on-full after CV (see svd_recommender.py I sent)
```

```
try:
```

```
    from svd_recommender import train_svd_model, svd_predict_for_user
```

```
except Exception: # pragma: no cover
```

```
    train_svd_model = svd_predict_for_user = None # type: ignore
```

```
# ----- paths & small utils -----
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
def P(*parts): return os.path.join(BASE_DIR, *parts)
```

```
def _minmax(d: Dict[int, float]) -> Dict[int, float]:
```

```
    if not d: return {}
```

```
    vals = np.fromiter(d.values(), dtype=float)
```

```
    vmin, vmax = float(vals.min()), float(vals.max())
```

```
    if vmax <= vmin: # constant or single value
```

```
        return {k: 0.0 for k in d}
```

```
    rng = vmax - vmin
```

```
    return {k: (float(v) - vmin) / rng for k, v in d.items()}
```

```
# ----- data loading -----
```

```
# Courses
```

```
_course_df = pd.read_csv(P("data", "udemy_course_data.csv"),
```

```
                        usecols=["course_id", "course_title", "subject"])
```

```
_course_df = _course_df.dropna(subset=["course_id"]).copy()
```

```
_course_df["course_id"] = pd.to_numeric(_course_df["course_id"], errors="coerce")
```

```
_course_df = _course_df.dropna(subset=["course_id"]).copy()
```

```
_course_df["course_id"] = _course_df["course_id"].astype(int)
```

```
_course_df["course_title"] = _course_df["course_title"].fillna("").astype(str)
```

```
_course_df["subject"] = _course_df["subject"].fillna("").astype(str)
```

```
_course_df["text"] = (_course_df["course_title"] + " " + _course_df["subject"]).str.strip()
```

```
COURSE_IDS = _course_df["course_id"].tolist()
```

```
# Interactions (user_id, course_id, rating)
```

```
try:
```

```
    _inter = pd.read_csv(P("data", "Synthetic_Interactions.csv"))
```

```
    _inter["user_id"] = _inter["user_id"].astype(str).str.strip()
```

```
    _inter["course_id"] = pd.to_numeric(_inter["course_id"], errors="coerce")
```

```
    _inter = _inter.dropna(subset=["course_id"]).copy()
```

```
    _inter["course_id"] = _inter["course_id"].astype(int)
```

```
    if "rating" not in _inter.columns:
```

```
        _inter["rating"] = 0.0
```

```
except Exception:
```

```

_inter = pd.DataFrame(columns=["user_id", "course_id", "rating"])

# Wide rating matrix (optional)
try:
    _ratings = pd.read_csv(P("data", "User-Item_Rating_Matrix.csv"), index_col=0)
    _ratings.index = _ratings.index.astype(str)
except Exception:
    _ratings = pd.DataFrame()

# ----- vectorizers / embeddings -----
_tfidf = TfidfVectorizer(stop_words="english")
_tfidf_matrix = _tfidf.fit_transform(_course_df["text"]) # (N, V)

# BERT sentence embeddings (cached)
_EMB_DIR = P("embeddings")
os.makedirs(_EMB_DIR, exist_ok=True)
_BERT_PATH = P("embeddings", "courses_minilm.npy")

if SentenceTransformer is not None:
    try:
        _bert = np.load(_BERT_PATH)
        if _bert.shape[0] != len(_course_df):
            raise ValueError("BERT cache size mismatch")
    except Exception:
        _model = SentenceTransformer("all-MiniLM-L6-v2")
        _bert = _model.encode(_course_df["text"].tolist(), show_progress_bar=False,
convert_to_numpy=True)
        # normalize once (cosine = dot)
        norms = np.linalg.norm(_bert, axis=1, keepdims=True)
        norms[norms == 0] = 1.0
        _bert = _bert / norms
        np.save(_BERT_PATH, _bert)
    else:
        _bert = None # no BERT available

# ----- clustering (optional fallback) -----
try:
    if not _ratings.empty:
        _user_vectors = _ratings.fillna(0).values
        _kmeans = KMeans(n_clusters=5, random_state=42)
        _clusters = _kmeans.fit_predict(_user_vectors)
        _user_cluster_map = dict(zip(_ratings.index.tolist(), _clusters.tolist()))
    else:

```

```

    _user_cluster_map = {}
except Exception:
    _user_cluster_map = {}

# ----- SVD model (lazy) -----
_SVD_MODEL = None
def _svd_scores(user_id: str, candidate_ids: List[int]) -> Dict[int, float]:
    """Return {cid: est_rating}; empty if SVD unavailable."""
    global _SVD_MODEL
    if train_svd_model is None or svd_predict_for_user is None:
        return {}
    if _SVD_MODEL is None:
        _SVD_MODEL, _, _ = train_svd_model(P("data", "Synthetic_Interactions.csv"))
    try:
        # our helper returns {cid:int -> est:float}
        return svd_predict_for_user(_SVD_MODEL, str(user_id), candidate_ids) # type: ignore
    except Exception:
        # fallback: predict directly
        out = {}
        for cid in candidate_ids:
            try:
                out[cid] = float(_SVD_MODEL.predict(str(user_id), str(cid)).est) # type: ignore
            except Exception:
                pass
        return out

# ----- components for the hybrid -----
def _seen_courses(user_id: str) -> set[int]:
    seen = set()
    if not _inter.empty:
        seen |= set(_inter.loc[_inter["user_id"] == user_id, "course_id"].astype(int).tolist())
    if not _ratings.empty and user_id in _ratings.index:
        # consider any positive value as "seen" (adjust if you have explicit threshold)
        row = _ratings.loc[user_id]
        seen |= set(pd.to_numeric(row.index[row.fillna(0) > 0],
errors="coerce").dropna().astype(int).tolist())
    return seen

def _popularity_scores(exclude: set[int]) -> Dict[int, float]:
    """Mean rating * log(1+count) to balance quality & volume."""
    if _inter.empty:
        return {}
    g = _inter.groupby("course_id")["rating"].agg(["mean", "count"])

```

```

g["pop_score"] = g["mean"].fillna(0) * np.log1p(g["count"].fillna(0))
pop = g["pop_score"].to_dict()
for cid in list(pop.keys()):
    if cid in exclude:
        pop.pop(cid, None)
return pop

def _feedback_liked_titles(user_id: str) -> List[str]:
    path = P("data", "user_feedback.csv")
    if not os.path.exists(path):
        return []
    try:
        df = pd.read_csv(path)
        # If there's a 'feedback' column with 'thumbs_up'
        if "feedback" in df.columns and "course_title" in df.columns:
            liked = df[(df["user_id"].astype(str) == user_id) & (df["feedback"] ==
"thumbs_up")]["course_title"]
            return liked.dropna().astype(str).unique().tolist()
        # Or use rating column if present
        if "rating" in df.columns and "course_title" in df.columns:
            liked = df[(df["user_id"].astype(str) == user_id) & (pd.to_numeric(df["rating"],
errors="coerce") >= 4)]["course_title"]
            return liked.dropna().astype(str).unique().tolist()
        except Exception:
            pass
    return []

def _content_scores(user_id: str, candidate_ids: List[int]) -> Dict[int, float]:
    """TF-IDF (+ BERT if available) based similarity from user's 'liked' items."""
    # liked via feedback -> titles -> text
    liked_titles = _feedback_liked_titles(user_id)

    liked_ids: List[int] = []
    if liked_titles:
        # map liked titles to ids
        liked_ids = _course_df.loc[_course_df["course_title"].isin(liked_titles),
"course_id"].astype(int).tolist()

    # if still empty, use items with positive interactions/ratings
    if not liked_ids and not _ratings.empty and user_id in _ratings.index:
        row = _ratings.loc[user_id].fillna(0)
        liked_ids = [int(c) for c in row.index if c != "cluster" and row.get(c, 0) > 0 and str(c).isdigit()]

```

```

if not liked_ids:
    return {}

liked_texts = _course_df.loc[_course_df["course_id"].isin(liked_ids), "text"].astype(str).tolist()
if not liked_texts:
    return {}

# TF-IDF similarity
tfidf_vec = _tfidf.transform([" ".join(liked_texts)])
tfidf_sims = cosine_similarity(tfidf_vec, _tfidf_matrix).flatten() # len = N courses

# Optional BERT similarity
if _bert is not None:
    # embed the combined liked text
    from numpy.linalg import norm
    v = SentenceTransformer("all-MiniLM-L6-v2").encode([" ".join(liked_texts)],
show_progress_bar=False, convert_to_numpy=True)[0]
    n = norm(v)
    if n > 0:
        v = v / n
        bert_sims = (_bert @ v) # cosine via dot
        sims = 0.5 * tfidf_sims + 0.5 * bert_sims
    else:
        sims = tfidf_sims
else:
    sims = tfidf_sims

# keep only candidate ids
idx_by_cid = {int(cid): i for i, cid in enumerate(_course_df["course_id"].tolist())}
out = {cid: float(sims[idx_by_cid[cid]]) for cid in candidate_ids if cid in idx_by_cid}
return out

def _rules_boost(prior_course_ids: List[int], candidate_ids: List[int]) -> Dict[int, float]:
    """Use association-rule lift as a small boost signal."""
    if not callable(explain_rules_for_course) or not prior_course_ids:
        return {}
    boost: Dict[int, float] = {}
    for cid in prior_course_ids[:5]: # limit API calls
        try:
            rows = explain_rules_for_course(cid, top_k=50)
        except Exception:
            rows = []
        for r in rows:

```

```

    cons = r.get("consequent")
    lift = float(r.get("lift", 0.0))
    if cons is None:
        continue
    try:
        cons = int(cons)
    except Exception:
        continue
    if cons not in candidate_ids:
        continue
    boost[cons] = max(boost.get(cons, 0.0), lift) # take best lift for that consequent
return boost

```

----- HYBRID weights -----

ALPHA, BETA, GAMMA = 0.6, 0.3, 0.1 # SVD, content, rules

```

def _combine(candidate_ids: List[int], svd_s: Dict[int, float], cont_s: Dict[int, float], rule_s: Dict[int, float]) -> Dict[int, float]:
    a = _minmax({k: svd_s.get(k, 0.0) for k in candidate_ids})
    b = _minmax({k: cont_s.get(k, 0.0) for k in candidate_ids})
    c = _minmax({k: rule_s.get(k, 0.0) for k in candidate_ids})
    return {k: ALPHA * a.get(k, 0.0) + BETA * b.get(k, 0.0) + GAMMA * c.get(k, 0.0) for k in candidate_ids}

```

----- PUBLIC API -----

```

def get_recommendations(user_id: str, top_n: int = 5) -> List[Dict]:
    """

```

Hybrid recommender:

- If truly new user -> popularity fallback (cold-start).
- Else combine SVD (collab), content (TF-IDF/BERT), and rules boost.

Returns: list of {"course_title": str, "explanation": str}

"""

```

user_id = str(user_id).strip()

```

```

top_n = max(1, min(int(top_n), 50))

```

```

seen = _seen_courses(user_id)

```

```

all_ids = set(COURSE_IDS)

```

```

candidate_ids = sorted(list(all_ids - seen))

```

```

if not candidate_ids:

```

```

    candidate_ids = COURSE_IDS # extreme case: show something

```

```

# True cold-start: user not in ratings nor interactions -> POPULARITY

```

```

no_ratings = _ratings.empty or user_id not in _ratings.index

```

```

no_inter = _inter.empty or user_id not in set(_inter["user_id"])
if no_ratings and no_inter:
    pop = _popularity_scores(exclude=seen)
    if not pop:
        # dataset missing? Show top by course_id to avoid empty UI
        fallback = sorted(candidate_ids)[:top_n]
        return [{"course_title": _course_df.loc[_course_df["course_id"] == cid, "course_title"].iloc[0],
                "explanation": "Cold-start fallback"} for cid in fallback]
    top = sorted(pop.items(), key=lambda x: x[1], reverse=True)[:top_n]
    return [{"course_title": _course_df.loc[_course_df["course_id"] == cid, "course_title"].iloc[0],
            "explanation": "Popular with learners (cold-start)"} for cid, _ in top]

# Build component scores
svd_scores = _svd_scores(user_id, candidate_ids) # may be {}
content_scores = _content_scores(user_id, candidate_ids) # may be {}

# prior courses for rules: use seen (cap to a few)
prior_for_rules = sorted(list(seen))[:5]
rules_scores = _rules_boost(prior_for_rules, candidate_ids) # may be {}

final = _combine(candidate_ids, svd_scores, content_scores, rules_scores)
ranked = sorted(final.items(), key=lambda x: x[1], reverse=True)

# If hybrid ended up empty (e.g., all components missing), fall back to popularity
if not ranked:
    pop = _popularity_scores(exclude=seen)
    ranked = sorted(pop.items(), key=lambda x: x[1], reverse=True)

top = ranked[:top_n]
out: List[Dict] = []
for cid, score in top:
    title = _course_df.loc[_course_df["course_id"] == int(cid), "course_title"]
    title = title.iloc[0] if not title.empty else f"Course {cid}"

    parts = []
    if cid in svd_scores: parts.append("SVD")
    if cid in content_scores: parts.append("content similarity")
    if cid in rules_scores: parts.append("co-enrollment rules")
    why = " + ".join(parts) if parts else "hybrid"
    out.append({
        "course_title": str(title),
        "explanation": f"{why}; hybrid score={round(float(score), 3)}"
    })

```

return out

sentiment_analysis.py

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
```

```
# Function to analyze sentiment of a given text
```

```
def analyze_sentiment(text):
```

```
    analyzer = SentimentIntensityAnalyzer() # Create a VADER sentiment analyzer
```

```
    scores = analyzer.polarity_scores(text) # Get sentiment scores (pos, neu, neg, compound)
```

```
    compound = scores['compound'] # Use the compound score as overall sentiment
```

```
# Classify sentiment based on compound score thresholds
```

```
if compound >= 0.05:
```

```
    label = "positive"
```

```
elif compound <= -0.05:
```

```
    label = "negative"
```

```
else:
```

```
    label = "neutral"
```

```
# Provide an explanation with the compound score
```

```
explanation = f'Compound score: {compound} → {label.upper()}'
```

```
return label, explanation # Return sentiment label and explanation
```

svd_recommender.py

```
from __future__ import annotations
```

```
import os
```

```
import pandas as pd
```

```
from surprise import Dataset, Reader, SVD
```

```
from surprise.model_selection import cross_validate
```

```
# Robust base dir so scripts run from any working directory
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
def _resolve(path: str) -> str:
```

```
    """Join with BASE_DIR if a relative path is given."""
```

```
    return path if os.path.isabs(path) else os.path.join(BASE_DIR, path)
```

```
# Function to train an SVD collaborative filtering model
```

```
def train_svd_model(interactions_path: str = "data/Synthetic_Interactions.csv"):
```

```
    """
```

Trains an SVD model with CV (RMSE/MAE), then FITS on the FULL trainset for serving.

Returns: (model, avg_rmse, avg_mae)

"""

interactions_path = _resolve(interactions_path)

df = pd.read_csv(interactions_path)[["user_id", "course_id", "rating"]].dropna()

Surprise expects string IDs

df["user_id"] = df["user_id"].astype(str)

df["course_id"] = df["course_id"].astype(str)

reader = Reader(rating_scale=(1, 5))

data = Dataset.load_from_df(df[["user_id", "course_id", "rating"]], reader)

CV first (model re-initialized internally per fold)

model = SVD(random_state=42)

cv_results = cross_validate(model, data, measures=["RMSE", "MAE"], cv=5, verbose=False)

avg_rmse = round(float(cv_results["test_rmse"].mean()), 3)

avg_mae = round(float(cv_results["test_mae"].mean()), 3)

Fit on the FULL trainset for serving predictions

full_trainset = data.build_full_trainset()

model.fit(full_trainset)

return model, avg_rmse, avg_mae

Function to generate top-N course recommendations for a user using the trained SVD model

def get_svd_predictions(

user_id,

model: SVD,

course_df: pd.DataFrame,

interactions_path: str = "data/Synthetic_Interactions.csv",

top_n: int = 5

):

"""

Returns a list of {course_title, explanation} for unseen courses, ranked by predicted rating.

Assumes course_df has columns: ['course_id', 'course_title'].

"""

interactions_path = _resolve(interactions_path)

top_n = max(1, min(int(top_n), 50))

Load interactions & normalize types

df = pd.read_csv(interactions_path)

df["user_id"] = df["user_id"].astype(str)

```
df["course_id"] = df["course_id"].astype(str)

# Build candidate set: all courses not yet rated by this user
user_id_str = str(user_id)
already_rated = set(df.loc[df["user_id"] == user_id_str, "course_id"].tolist())

# Ensure course_df ids as strings for matching
course_df = course_df.copy()
course_df["course_id"] = course_df["course_id"].astype(str)

all_courses = course_df["course_id"].tolist()
unseen = [cid for cid in all_courses if cid not in already_rated]

# Predict ratings for unseen courses
preds = []
for cid in unseen:
    est = model.predict(user_id_str, cid).est
    preds.append((cid, float(est)))

# Rank and format results
preds.sort(key=lambda x: x[1], reverse=True)
top = preds[:top_n]

results = []
for cid, score in top:
    # Safe title lookup
    title_series = course_df.loc[course_df["course_id"] == cid, "course_title"]
    title = title_series.iloc[0] if not title_series.empty else f"Course {cid}"
    results.append({
        "course_title": str(title),
        "explanation": f"Predicted rating: {round(score, 2)}"
    })

return results
```

word2vec_similarity.py

```
from __future__ import annotations

import os
import re
import logging
from typing import List, Dict, Optional, Tuple
```

```
import numpy as np
import pandas as pd
from gensim.models import Word2Vec
from markupsafe import Markup, escape # used to safely render highlighted HTML in Jinja templates
import csv as _csv # reserved if you later switch to python CSV engine for messy files

# ----- logging -----
# Basic logger so we can see when the model is trained or cache is loaded.
logging.basicConfig(level=logging.INFO)
log = logging.getLogger("w2v")

# ----- paths -----
# Project-relative paths so it works no matter where you run from.
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_CSV = os.path.join(BASE_DIR, "data", "udemy_course_data.csv")

# Where we cache the trained Word2Vec model and precomputed course vectors.
EMB_DIR = os.path.join(BASE_DIR, "embeddings")
MODEL_PATH = os.path.join(EMB_DIR, "word2vec.model")
DOCVEC_NPY = os.path.join(EMB_DIR, "course_vectors.npy") # matrix of course vectors
IDX_NPY = os.path.join(EMB_DIR, "course_index.npy") # mapping back into the dataframe rows
os.makedirs(EMB_DIR, exist_ok=True)

# ----- globals (lazy) -----
# We only load/train once per process. Subsequent calls reuse these.
_W2V: Optional[Word2Vec] = None # the trained gensim Word2Vec model
_DOCVECS: Optional[np.ndarray] = None # (N, D) normalized course vectors
_INDEX: Optional[np.ndarray] = None # N indices mapping vectors back to _DF rows
_DF: Optional[pd.DataFrame] = None # courses dataframe with id, title, subject

# ----- helpers -----
# Tokenization regex: keep only letters/numbers, lowercase, drop punctuation.
_ALNUM = re.compile(r"[a-z0-9]+")

def _tokenize(s: str) -> list[str]:
    """Turn a title into lowercase alphanumeric tokens (e.g., 'Python 101!' -> ['python','101'])."""
    if not isinstance(s, str):
        return []
    return _ALNUM.findall(s.lower())

def _load_df() -> pd.DataFrame:
```

```

"""
Load the course catalogue. We use only three columns:
- course_id (int)
- course_title (str)
- subject (str, can be empty)
We also clean titles a bit to avoid URL tails or double spaces.
"""
df = pd.read_csv(DATA_CSV, usecols=["course_id", "course_title", "subject"])
df["course_id"] = pd.to_numeric(df["course_id"], errors="coerce")
df = df.dropna(subset=["course_id"]).copy()
df["course_id"] = df["course_id"].astype(int)
df["course_title"] = df["course_title"].fillna("").astype(str)
df["subject"] = df["subject"].fillna("").astype(str)

# Clean accidental URL tails and collapse double spaces
df["course_title"] = df["course_title"].str.replace(
    r"\s+https?://\S+.*$", "", regex=True
).str.replace(r"\s{2,}", " ", regex=True).str.strip()

return df

def _train_and_cache() -> Tuple[Word2Vec, np.ndarray, np.ndarray, pd.DataFrame]:
    """
    Train Word2Vec on the tokenized course titles + subject.
    Then build a single vector per course (mean of token vectors), L2-normalize it,
    and cache both the model and the vectors on disk for speed.
    """
    df = _load_df()
    if df.empty:
        raise RuntimeError("udemy_course_data.csv has no rows after cleaning")

    # Text to train on = "title + subject" per course
    text = (df["course_title"] + " " + df["subject"]).astype(str)
    tokenized = [_tokenize(t) for t in text]

    # Train a small Word2Vec model (skip-gram). Vector size 64 is fast and good enough.
    model = Word2Vec(
        sentences=tokenized,
        vector_size=64,
        window=5,
        min_count=1,  # keep even rare tokens (small dataset)
        workers=1,    # deterministic across OS
        sg=1,         # 1 = skip-gram, better for small data
    )

```

```

    seed=42,
)

# Build one vector per course by averaging its token vectors (simple but effective).
docvecs, keep_idx = [], []
for i, toks in enumerate(tokenized):
    vecs = [model.wv[w] for w in toks if w in model.wv]
    if not vecs:
        continue # skip rows with no valid tokens
    v = np.mean(vecs, axis=0, dtype=np.float32)
    docvecs.append(v)
    keep_idx.append(i)

# Stack vectors into matrix, and keep index mapping back to df rows.
if not docvecs:
    docarr = np.empty((0, model.vector_size), dtype=np.float32)
    keep = np.empty((0,), dtype=np.int32)
else:
    docarr = np.vstack(docvecs).astype(np.float32)
    keep = np.array(keep_idx, dtype=np.int32)

# Normalize so cosine similarity = dot product (fast).
norms = np.linalg.norm(docarr, axis=1, keepdims=True)
norms[norms == 0] = 1.0
docarr = docarr / norms

# Cache to disk (so next run doesn't retrain).
os.makedirs(EMB_DIR, exist_ok=True)
model.save(MODEL_PATH)
np.save(DOCVEC_NPY, docarr)
np.save(IDX_NPY, keep)

log.info("Trained W2V: kept=%d, dim=%d", docarr.shape[0], model.vector_size)
return model, docarr, keep, df

def __ensure_ready() -> None:
    """
    Ensure globals are initialized:
    - If cache files exist, load them.
    - If not, train and cache.
    - If loading fails (version mismatch/corruption), retrain safely.
    """
    global _W2V, _DOCVECS, _INDEX, _DF

```

```

if _W2V is not None and _DOCV ECS is not None and _INDEX is not None and _DF is not None:
    return
try:
    _DF = _load_df()
    if not (os.path.exists(MODEL_PATH) and os.path.exists(DOCVEC_NPY) and
os.path.exists(IDX_NPY)):
        log.info("Cache missing → training...")
        _W2V, _DOCV ECS, _INDEX, _DF = _train_and_cache()
    else:
        log.info("Loading cache...")
        _W2V = Word2Vec.load(MODEL_PATH)
        _DOCV ECS = np.load(DOCVEC_NPY)
        _INDEX = np.load(IDX_NPY)

        # Quick sanity checks to catch stale or corrupted cache
        if _DOCV ECS.ndim != 2 or _INDEX.ndim != 1:
            raise ValueError("Corrupted cache shapes")
        if _DOCV ECS.shape[0] != _INDEX.shape[0]:
            raise ValueError("Vectors and index length mismatch")
        if _INDEX.size and int(_INDEX.max()) >= len(_DF):
            raise ValueError("Index points beyond dataframe rows")
except Exception as e:
    # If anything goes wrong, retrain to self-heal.
    log.warning("Cache load failed (%s) → retraining...", e)
    _W2V, _DOCV ECS, _INDEX, _DF = _train_and_cache()

def get_title_samples(k: int = 100) -> list[str]:
    """
    Return up to k course titles for the HTML <datalist> (autocomplete).
    If something fails (e.g., file missing), return an empty list to keep the UI usable.
    """
    try:
        df = pd.read_csv(DATA_CSV, usecols=["course_title"])
        titles = df["course_title"].dropna().astype(str).unique().tolist()
        return titles[:k]
    except Exception:
        return []

def highlight_tokens(title: str, tokens: list[str]) -> Markup:
    """
    Highlight query tokens inside a course title for the UI using <mark>.
    We escape first (to prevent HTML injection), then apply a case-insensitive regex.
    """

```

```

if not title or not tokens:
    return Markup(escape(title))
text = escape(title)
for t in sorted(set(tokens), key=len, reverse=True): # longer tokens first
    if not t:
        continue
    pattern = re.compile(rf'({re.escape(t)})', re.IGNORECASE)
    # Use raw string so \1 is the regex backreference group
    text = Markup(pattern.sub(r"<mark>\1</mark>", str(text)))
return Markup(text)

def _fuzzy_overlap_fallback(query_tokens: list[str], top_n: int) -> List[Dict]:
    """
    Fallback path when all tokens are OOV (out-of-vocabulary) for Word2Vec,
    e.g., non-English or very rare words. We rank by simple token overlap
    between the query tokens and each course's tokens (title + subject).
    """
    if _DF is None or not query_tokens:
        return []
    df = _DF.copy()

    # Pre-tokenize titles/subjects in the dataframe
    df["__tok"] = (df["course_title"] + " " + df["subject"]).str.lower().str.findall(_ALNUM)

    # Score by overlap count
    q = set(query_tokens)
    df["__score"] = df["__tok"].apply(lambda ts: len(q & set(ts)))

    # Take the best matches, build output rows (treat score as a crude "similarity")
    df = df.sort_values("__score", ascending=False).head(max(top_n, 10))
    out: List[Dict] = []
    for _, row in df.iterrows():
        score = int(row["__score"])
        if score <= 0:
            continue
        out.append({
            "course_id": int(row["course_id"]),
            "course_title": str(row["course_title"]),
            "subject": str(row.get("subject") or "Unknown"),
            "similarity": float(score),
        })
    if len(out) >= top_n:
        break

```

return out

```
def get_similar_courses(input_course_title: str, top_n: int = 5) -> List[Dict]:
    """
```

Main entry point used by the Flask route:

- Accept a free-text course title from the user.
- Vectorize it with Word2Vec (mean of token vectors).
- Compute cosine similarity against all course vectors.
- Return the top-N matches, each with id/title/subject/similarity.

Design choices:

- If the query tokens are all OOV, we fall back to token-overlap ranking (never crash).
- We L2-normalize vectors so cosine = dot product for speed.
- We take a slightly larger top slice, then filter out duplicates/self-matches.

```
    """
```

```
    _ensure_ready()
```

```
    assert _W2V is not None and _DOCVECS is not None and _INDEX is not None and _DF is not
    None
```

```
    title = (input_course_title or "").strip()
```

```
    if not title:
```

```
        return []
```

```
    # Tokenize the query and keep only tokens present in the Word2Vec vocabulary
```

```
    toks_all = _tokenize(title)
```

```
    toks_iv = [w for w in toks_all if w in _W2V.wv]
```

```
    # If nothing is in-vocab (e.g., different language), use the fuzzy fallback.
```

```
    if not toks_iv:
```

```
        return _fuzzy_overlap_fallback(toks_all, top_n)
```

```
    # Build a single query vector as the mean of word vectors, then normalize.
```

```
    v = np.mean([_W2V.wv[w] for w in toks_iv], axis=0).astype(np.float32)
```

```
    nv = np.linalg.norm(v)
```

```
    if nv == 0 or _DOCVECS.size == 0:
```

```
        return []
```

```
    v /= nv
```

```
    # Cosine similarity via a fast dot product with all course vectors.
```

```
    sims = _DOCVECS @ v
```

```
    if sims.size == 0:
```

```
        return []
```

```
# Take a generous slice (3x), then filter duplicates and exact title matches.
```

```
top_idx = np.argsort(-sims)[: max(top_n * 3, 15)]
```

```
rows: List[Dict] = []
```

```
seen_titles = set()
```

```
for j in top_idx:
```

```
    i = int(_INDEX[j])
```

```
    if i < 0 or i >= len(_DF):
```

```
        continue
```

```
    row = _DF.iloc[i]
```

```
    title_j = str(row["course_title"])
```

```
# Skip exact self-match and duplicates in case of near-identical rows
```

```
if title_j.lower() == title.lower():
```

```
    continue
```

```
if title_j in seen_titles:
```

```
    continue
```

```
seen_titles.add(title_j)
```

```
rows.append({
```

```
    "course_id": int(row["course_id"]),
```

```
    "course_title": title_j,
```

```
    "subject": str(row.get("subject") or "Unknown"),
```

```
    "similarity": float(sims[j]),
```

```
})
```

```
if len(rows) >= top_n:
```

```
    break
```

```
return rows
```

CHAPTER 8: REFERENCES

- Agrawal, R., Imieliński, T. and Swami, A., 1993. Mining association rules between sets of items in large databases. *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, pp.207–216. [online] Available at: <https://doi.org/10.1145/170035.170072> [Accessed 18 May 2025].
- Blei, D.M., Ng, A.Y. and Jordan, M.I., 2003. Latent Dirichlet Allocation. *Journal of Machine Learning Research*, 3(Jan), pp.993–1022.
- Brooke, J., 1996. SUS: A ‘quick and dirty’ usability scale. In: P.W. Jordan, B. Thomas, B.A. Weerdmeester, and I.L. McClelland, eds. *Usability Evaluation in Industry*, Taylor & Francis, pp.189–194.
- Burke, R., 2002. Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), pp.331–370.
- Devlin, J., Chang, M.W., Lee, K. and Toutanova, K., 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.
- Elbadrawy, A. and Karypis, G., 2016. Domain-aware grade prediction and top-n course recommendation. In: *Proceedings of the 10th ACM Conference on Recommender Systems*, pp.183–190. <https://doi.org/10.1145/2959100.2959171>
- Flask, n.d. Flask Documentation. [online] Available at: <https://flask.palletsprojects.com/> [Accessed 18 May 2025].
- Google Developers, n.d. Recommender systems. [online] Available at: <https://developers.google.com/machine-learning/recommendation> [Accessed 18 May 2025].
- HuggingFace, n.d. Transformers Documentation. [online] Available at: <https://huggingface.co/docs/transformers/> [Accessed 18 May 2025].
- Hutto, C.J. and Gilbert, E., 2014. VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. In: *Proceedings of the Eighth International Conference on Weblogs and Social Media (ICWSM-14)*.
- IEEE Xplore, n.d. IEEE Transactions on Learning Technologies. [online] Available at: <https://ieeexplore.ieee.org/xpl/RecentIssue.jsp?punumber=4620076> [Accessed 18 May 2025].

Kohavi, R., Longbotham, R., Sommerfield, D., and Henne, R.M., 2009. Controlled experiments on the web: survey and practical guide. *Data Mining and Knowledge Discovery*, 18(1), pp.140–181.

Liang, Y., Chen, L. and Yang, X., 2018. An improved content-based recommendation system for online learning platforms. *Education and Information Technologies*, 23, pp.2629–2650.

MacQueen, J., 1967. Some Methods for Classification and Analysis of Multivariate Observations. In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1, pp.281–297. University of California Press.

Manouselis, N., Drachsler, H., Verbert, K. and Duval, E., 2011. *Recommender Systems for Learning*. Berlin: Springer.

Mikolov, T., Chen, K., Corrado, G. and Dean, J., 2013. Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.

Natural Language Toolkit (NLTK), n.d. NLTK Documentation. [online] Available at: <https://www.nltk.org/> [Accessed 18 May 2025].

Nielsen, J., 1994. *Usability Engineering*. Morgan Kaufmann.

pandas, n.d. pandas Documentation. [online] Available at: <https://pandas.pydata.org/docs/> [Accessed 18 May 2025].

Rennie, J.D.M., Shih, L., Teevan, J. and Karger, D.R., 2003. Tackling the Poor Assumptions of Naive Bayes Text Classifiers. In: *Proceedings of the 20th International Conference on Machine Learning (ICML-03)*, pp.616–623.

Ryan, R.M. and Deci, E.L., 2000. Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. *American Psychologist*, 55(1), pp.68–78.

Scikit-learn, n.d. Scikit-learn Documentation. [online] Available at: <https://scikit-learn.org/stable/documentation.html> [Accessed 18 May 2025].

Su, X. and Khoshgoftaar, T.M., 2009. A survey of collaborative filtering techniques. *Advances in Artificial Intelligence*, 2009, pp.1–19. <https://doi.org/10.1155/2009/421425>

Tang, S. and Pardos, Z.A., 2020. EduRank: A course recommendation system based on student grades. In: *Proceedings of the 13th International Conference on Educational Data Mining (EDM 2020)*, pp.461–466. Available at: https://educationaldatamining.org/files/conferences/EDM2020/papers/paper_90.pdf [Accessed 18 May 2025].

TensorFlow, n.d. TensorFlow Overview. [online] Available at: <https://www.tensorflow.org/overview> [Accessed 18 May 2025].

VanLehn, K., 2011. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4), pp.197–221.

Walkington, C., 2013. Using adaptive learning technologies to personalize instruction to student interests: The impact of relevant contexts on performance and learning outcomes. *Journal of Educational Psychology*, 105(4), pp.932–945.

Zhang, Y., Jin, Z. and Zhao, Y., 2020. A BERT-based approach for personalized reading material recommendation. *International Journal of Emerging Technologies in Learning (iJET)*, 15(12), pp.120–134.

Shailesh, X. (2023). *Course Recommendation System Dataset* [Data set]. Kaggle.

<https://www.kaggle.com/datasets/shailx/course-recommendation-system-dataset/data>